

בעיה ב-DP

בהינתן n , כמה אפשרויות שונות אפשר לחלק את הקבוצה $\{1, \dots, n\}$ ל-2 קבוצות שונות (שהאיחוד שלהם הוא הקבוצה $\{1, \dots, n\}$) שסכומם שווה.
תנאים: סיבוכיות זמן $O(n^3)$, סיבוכיות מקום $O(n^3)$.

פיתרון

בהינתן n , כמה אפשרויות שונות אפשר לחלק את הקבוצה $\{1, \dots, n\}$ ל-2 קבוצות שונות (שהאיחוד שלהם הוא הקבוצה $\{1, \dots, n\}$) שסכומם שווה.

תנאים: סיבוכיות זמן $O(n^3)$, סיבוכיות מקום $O(n^3)$.

- אבחנה: אם $1+2+\dots+n$ אי זוגי אזי אין פיתרון.
- נתחיל מלבדוק זאת, אם הפיתרון אי זוגי נחזיר 0, אחרת נמשיך.

פיתרון

בהינתן n , כמה אפשרויות שונות אפשר לחלק את הקבוצה $\{1, \dots, n\}$ ל-2 קבוצות שונות (שהאיחוד שלהם הוא הקבוצה $\{1, \dots, n\}$) שסכומם שווה.

תנאים: סיבוכיות זמן $O(n^3)$, סיבוכיות מקום $O(n^3)$.

- תכנון דינאמי באופן הבא: נגדיר מערך דו מימדי $A[n][\frac{n(n+1)}{4}]$.

פיתרון

בהינתן n , כמה אפשרויות שונות אפשר לחלק את הקבוצה $\{1, \dots, n\}$ ל-2 קבוצות שונות (שהאיחוד שלהם הוא הקבוצה $\{1, \dots, n\}$) שסכומם שווה.

תנאים: סיבוכיות זמן $O(n^3)$, סיבוכיות מקום $O(n^3)$.

- תכנון דינאמי באופן הבא: נגדיר מערך דו מימדי $A[n][\frac{n(n+1)}{4}]$.
- נבנה את המערך באופן הבא (מלמעלה למטה, משמאל לימין):
- נסתכל על $A[i][k]$: אם I שווה ל-1 ו- k גדול מ-2 אזי $A[1][k] = 0$, לכל i מתקיים $A[i][1] = 1$.

פיתרון

בהינתן n , כמה אפשרויות שונות אפשר לחלק את הקבוצה $\{1, \dots, n\}$ ל-2 קבוצות שונות (שהאיחוד שלהם הוא הקבוצה $\{1, \dots, n\}$) שסכומם שווה.

תנאים: סיבוכיות זמן $O(n^3)$, סיבוכיות מקום $O(n^3)$.

- תכנון דינאמי באופן הבא: נגדיר מערך דו מימדי $A[n][\frac{n(n+1)}{4}]$.

- נבנה את המערך באופן הבא (מלמעלה למטה, משמאל לימין):

- נסתכל על $A[i][k]$: אם I שווה ל-1 ו- k גדול מ-2 אזי $A[1][k] = 0$, לכל i מתקיים $A[i][1] = 1$.

- נוסחה רקורסיבית:

$$A[i][k] = \begin{cases} 0, & \text{if } k > \frac{i(i+1)}{2} \\ 1, & \text{if } k = \frac{i(i+1)}{2} \\ A[i-1][k] + a[i-1][k-i], & \text{if } i < k < \frac{i(i+1)}{2} \\ A[i-1][k] & \text{if } 0 < k \leq i \end{cases}$$

-

תוצאה + סיבוכיות

- אנחנו נחזיר: $A[n][\frac{n(n+1)}{4}]/2$.
- סיבוכיות מקום: ברור כי סיבוכיות המקום היא $O(n^3)$.
- סיבוכיות זמן: כל תא אנו עושים פעולה של $O(1)$ זמן (אפשרי כי אנו עושים סריקה מלמעלה למטה ומשמאל לימין), יש לנו $O(n^3)$ תאים, לכן סה"כ סיבוכיות תצא $O(n^3)$.

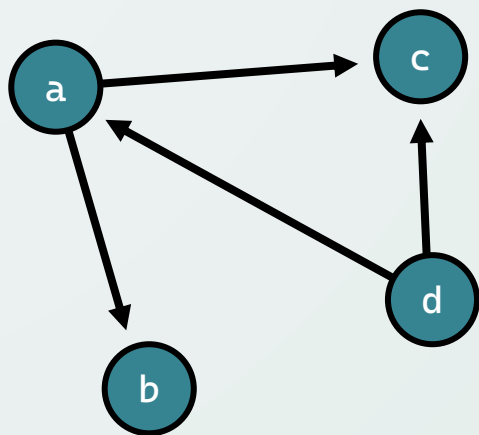
גרפים ואלגוריתם החיפוש לרוחב

תכנון וניתוח אלגוריתמים - תרגול שישי
סמסטר תשפ"ו א' (אתגר)
המצגת באדיבות אלון קרימגנד

דרכים שונות לייצוג גרפים

Graph representations

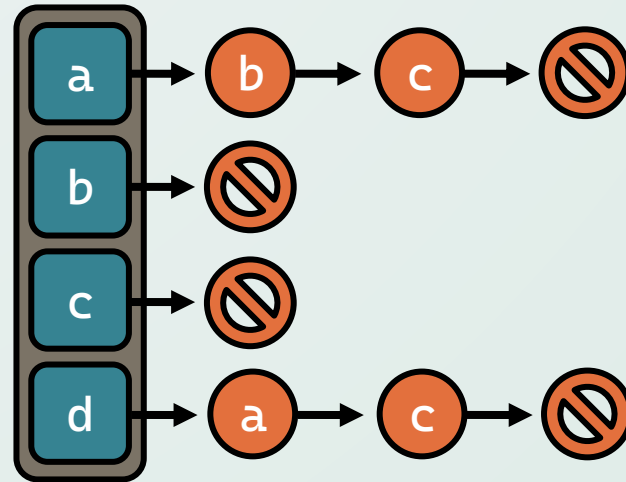
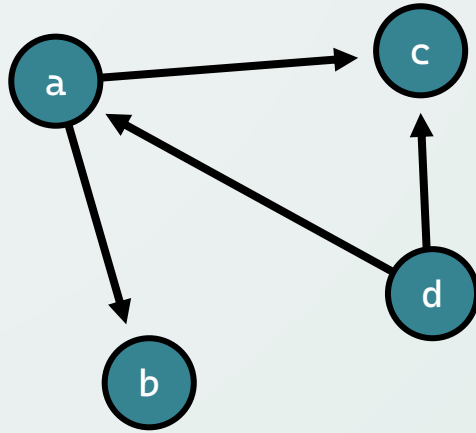
מטריצת שכנויות



	a	b	c	d
a	0	1	1	0
b	0	0	0	0
c	0	0	0	0
d	1	0	1	0

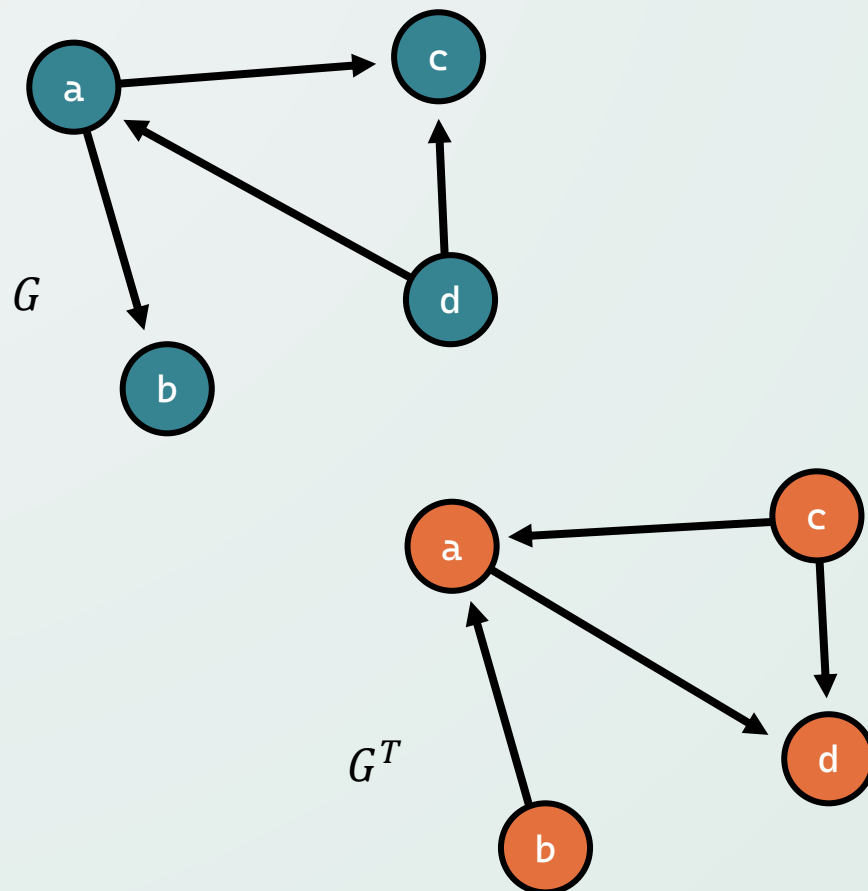
- דרך נפוצה לייצג גרף בעל n צמתים היא בעזרת מטריצה $A_{n \times n}$.
- התא A_{ij} יהיה 1 אם ישנה הקשת (i,j) או 0 אם הקשת אינה קיימת.
- אם הגרף ממושקל, במטריצה יופיע משקל הקשת.
- לייצוג זה יש מספר יתרונות.
- ניתן לבדוק האם קיימת קשת מסוימת ב- $\mathcal{O}(1)$.
- ייצוג בעזרת מטריצה מאפשר ביצוע של פעולות אלגבריות, ויכול להיות שימושי באלגוריתמים מסוימים.
- אך לייצוג זה גם חסרונות.
- גודל המטריצה הינו $\mathcal{O}(n^2)$, גם בעבור גרפים דלילים.
- לא ניתן לעבור על כל השכנים של צומת בצורה פשוטה.

רשימת שכנויות



- נוכל לצמצם את המטריצה.
- במקום לשמור את כל הזוגות (u, v) , נוכל לשמור מערך של רשימות שכנויות.
- המערך במיקום ה- i יכיל רשימה של כל השכנים של הצומת ה- i .
- לייצוג זה יתרונות רבים.
- גודל הייצוג לינארי - $O(n + m)$.
- ניתן לעבור על השכנים של כל צומת בקלות.
- מהצד השני, ישנם גם מספר חסרונות.
- לא ניתן לבדוק האם קיימת קשת מסוימת בזמן קבוע.

משחקים עם ייצוגים

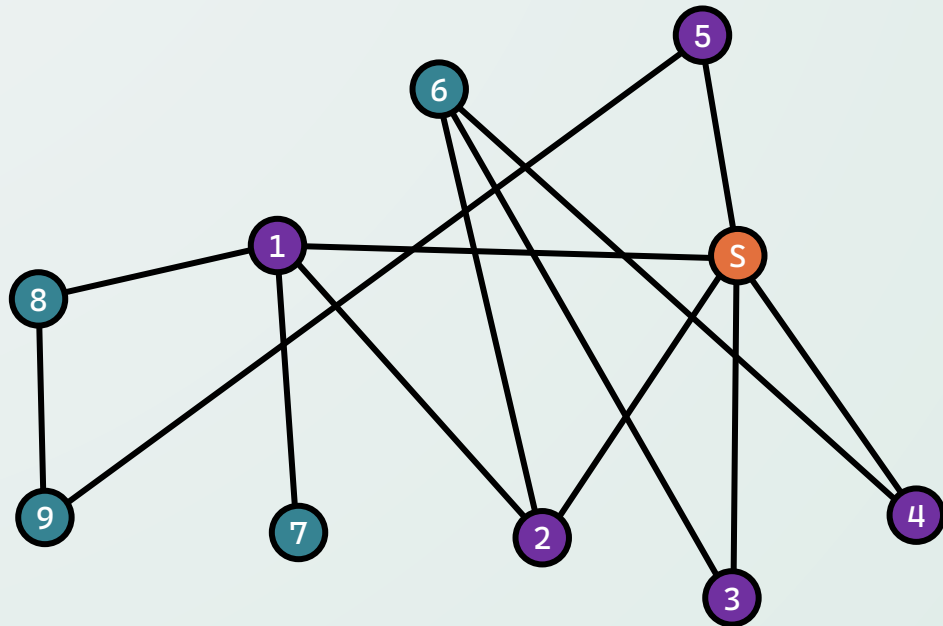


- נתון גרף מכוון $G = (V, E)$.
- תארו אלגוריתם הבונה את הגרף $G^T = (V, E^R)$ שהוא הגרף שבו כל הקשתות מכוונות הפוך.
- תארו אלגוריתם המוחק את הקשתות הכפולות בגרף. כלומר שכל קשת (u, v) תופיע בייצוג לכל היותר פעם אחת.
- תארו אלגוריתם המקבל קשת $(u, v) \in E$ ומכווץ אותה. כלומר, לאחר הפרוצדורה יתאחדו הצמתים u, v לצומת אחת.

אלגוריתם החיפוש לרוחב

Breadth-first Search (BFS)

אלגוריתם החיפוש לרוחב

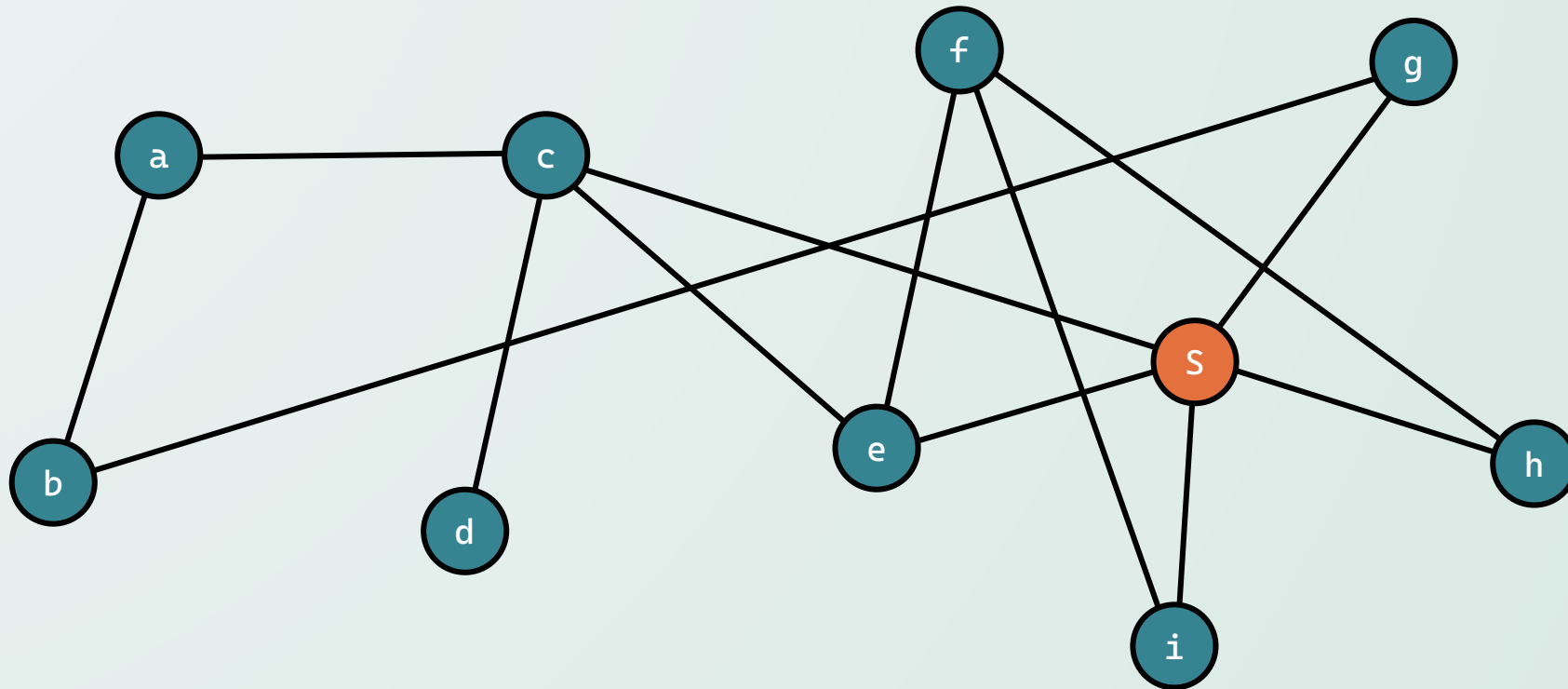


- אלגוריתם ה-BFS הוא אלגוריתם המשמש לסריקת גרף.
- האלגוריתם מקבל כקלט גרף G וצומת מקור S .
- האלגוריתם משריש סדר על הצמתים.
- הוא סורק תחילה את הצמתים שנמצאים במרחק צומת S מ- S (שכנים ישירים), ורק לאחר מכן צמתים במרחק 2 מ- S , וכך הלאה, עד שנסרק כל רכיב הקשירות של S .
- זמן הריצה של האלגוריתם הוא לינארי ואופטימלי, כלומר $O(n + m)$.
- השימוש הנפוץ באלגוריתם הוא מציאת המרחקים הקצרים ביותר של כל צומת בגרף מהצומת S .

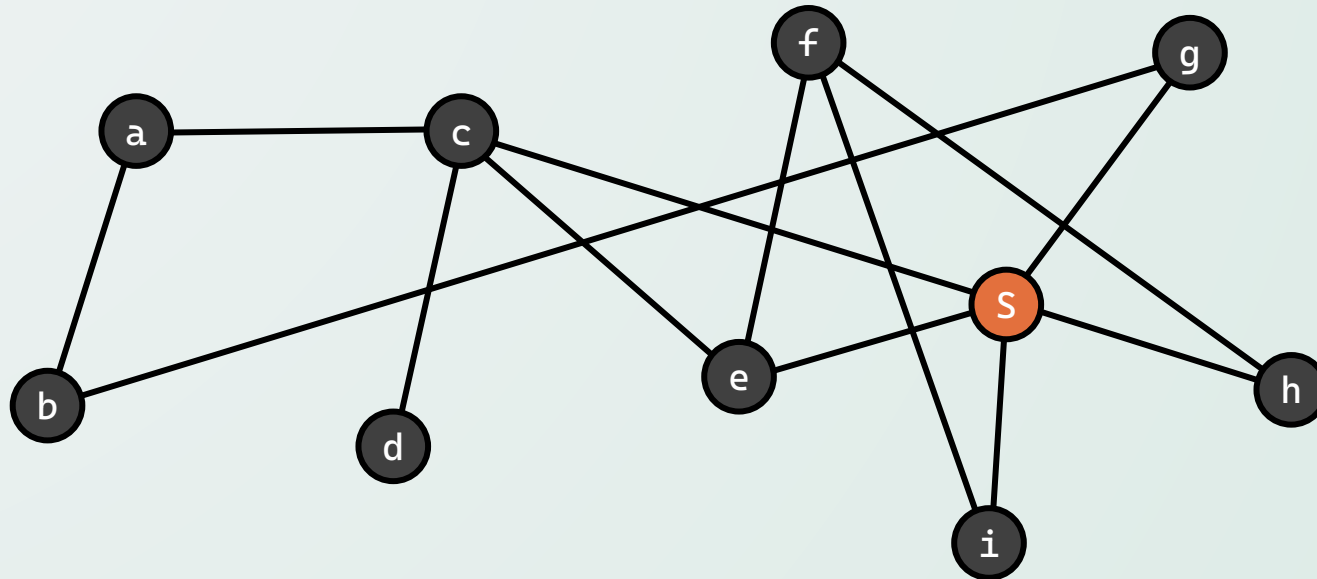
אלגוריתם החיפוש לרוחב

1. Function BFS(G, s):
 2. `queue.push(s)`
 3. `dist[s] = 0`
 4. While queue not empty:
 5. `v = queue.pop()`
 6. For each (v, u) in G:
 7. If `dist[u]` is ∞ :
 8. `dist[u] = dist[v]+1`
 9. `queue.push(u)`
 - האלגוריתם משתמש בתור (queue) כדי לקבוע באיזו צומת לבקר בשלב הבא.
 - בכל פעם שנבקר בצומת, נוסיף את כל השכנים שלה שעוד לא ראינו לתור.
 - בדרך זו, מובטח כי האלגוריתם יסרוק את הצמתים בסדר שנקבע על פי מרחקם מהצומת ההתחלתי.
 - זאת מכיוון שצומת שנכנס לתור יצא רק לאחר שכל הצמתים שהיו בו קודם יצאו, וכי כל הצמתים שנוספים לתור רחוקים יותר מהצומת הנוכחית שמעובדת.

דוגמת הרצה של אלגוריתם החיפוש לרוחב

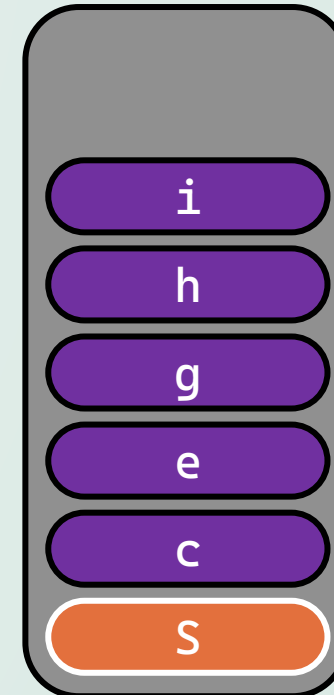
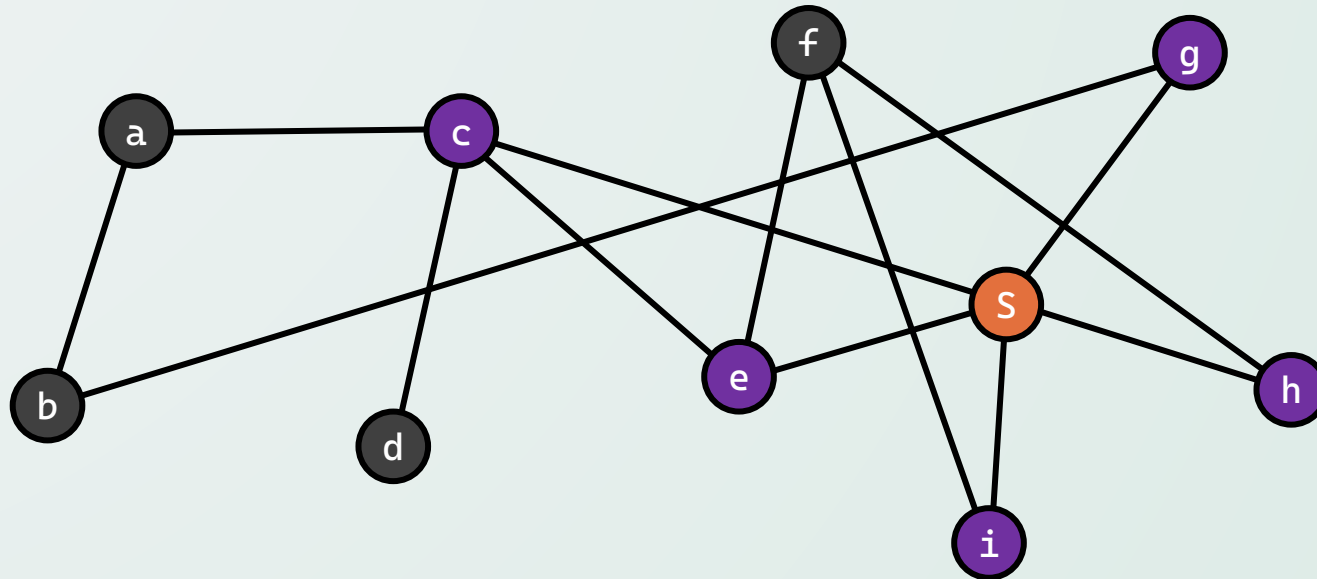


דוגמת הרצה של אלגוריתם החיפוש לרוחב



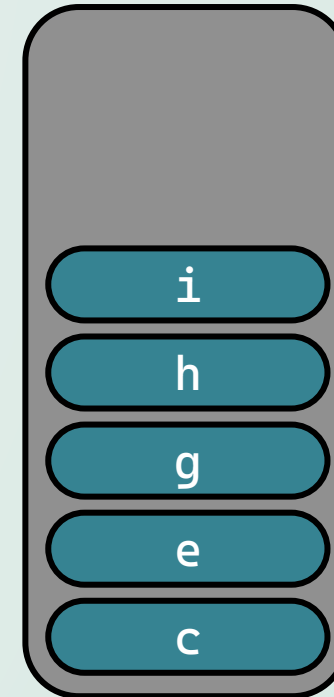
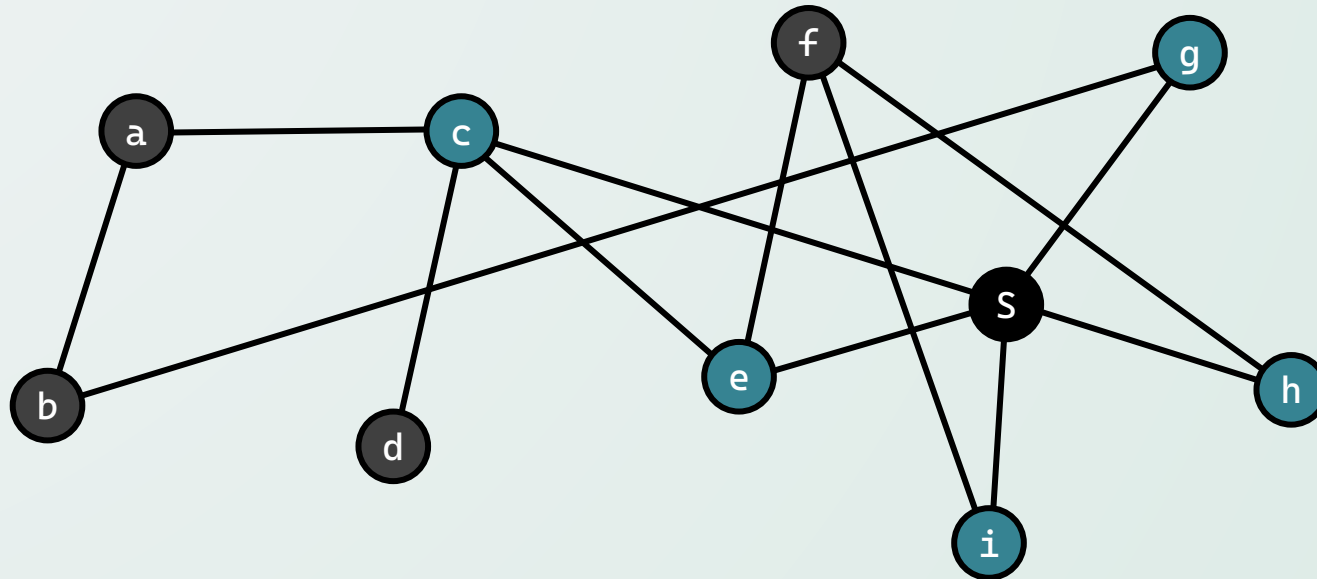
a	b	c	d	e	f	g	h	i	S
∞	∞	∞	∞	∞	∞	∞	∞	∞	0

דוגמת הרצה של אלגוריתם החיפוש לרוחב



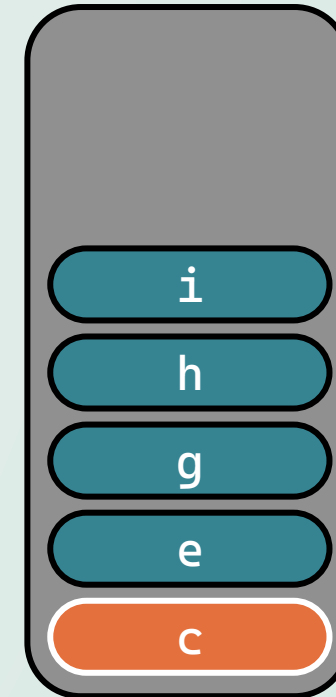
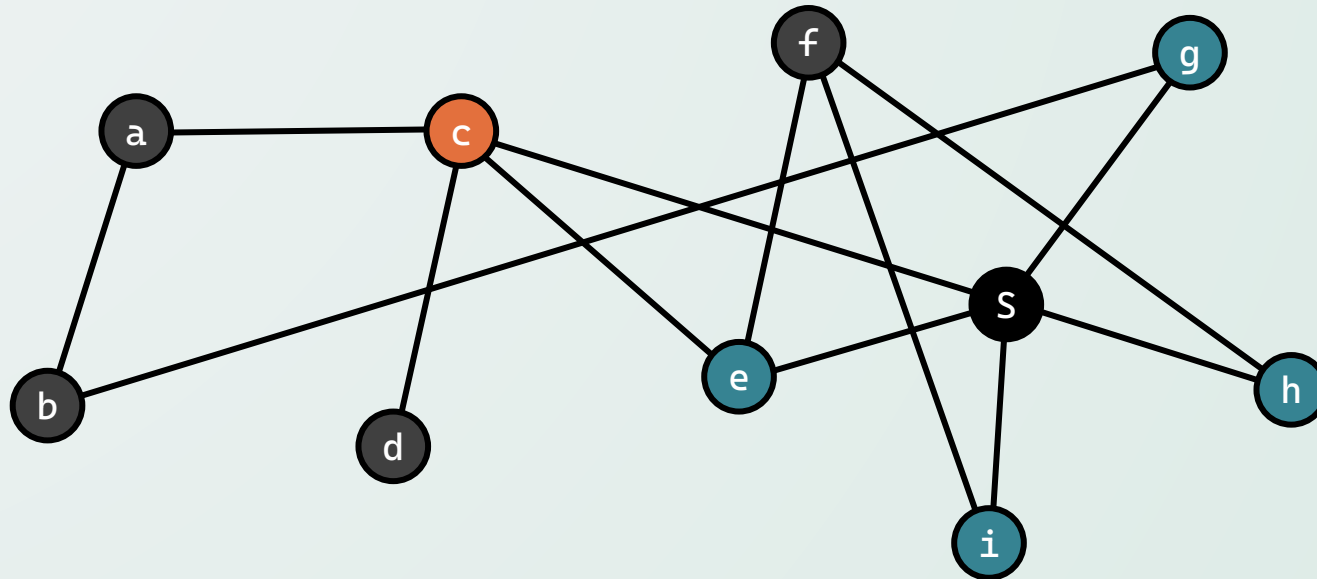
a	b	c	d	e	f	g	h	i	s
∞	∞	1	∞	1	∞	1	1	1	0

דוגמת הרצה של אלגוריתם החיפוש לרוחב



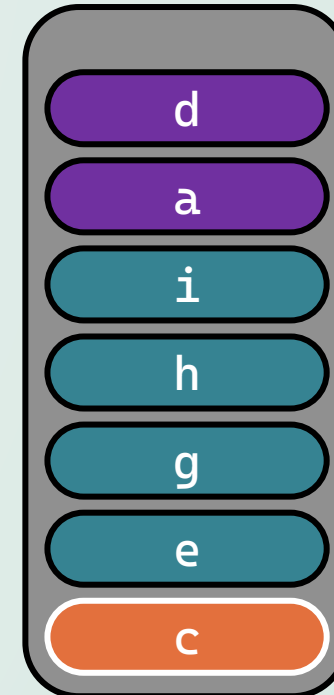
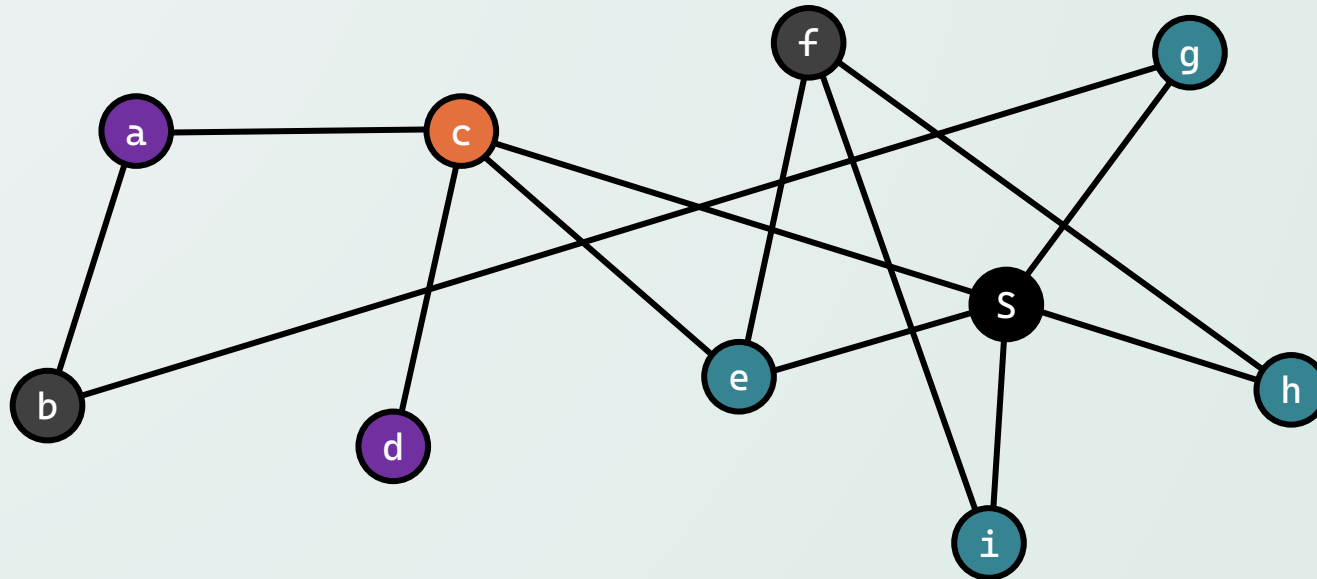
a	b	c	d	e	f	g	h	i	s
∞	∞	1	∞	1	∞	1	1	1	0

דוגמת הרצה של אלגוריתם החיפוש לרוחב



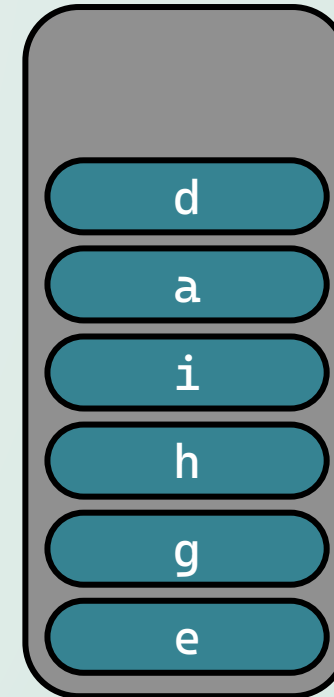
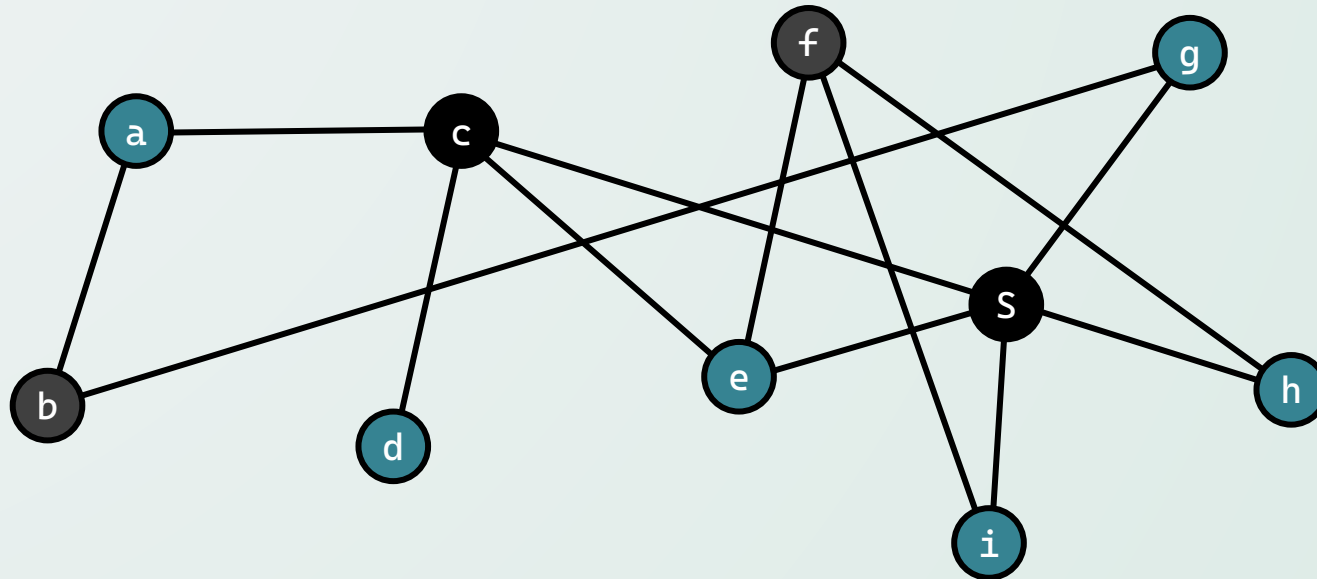
a	b	c	d	e	f	g	h	i	s
∞	∞	1	∞	1	∞	1	1	1	0

דוגמת הרצה של אלגוריתם החיפוש לרוחב



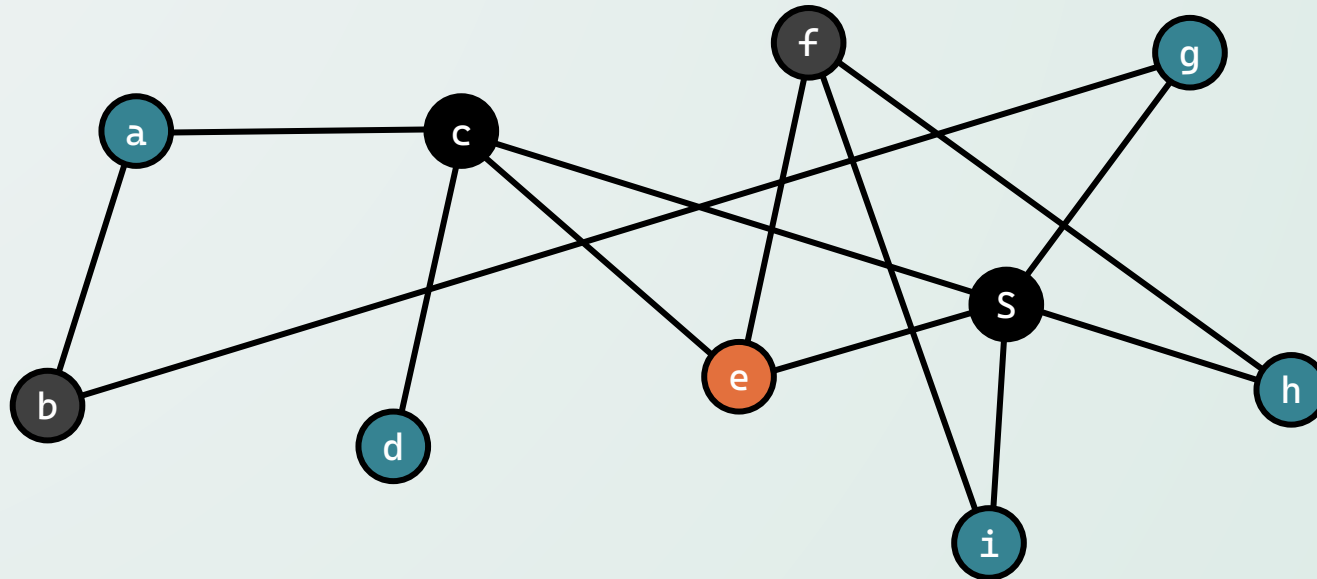
a	b	c	d	e	f	g	h	i	s
2	∞	1	2	1	∞	1	1	1	0

דוגמת הרצה של אלגוריתם החיפוש לרוחב



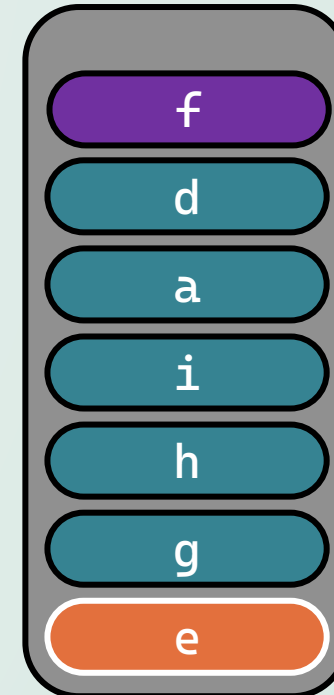
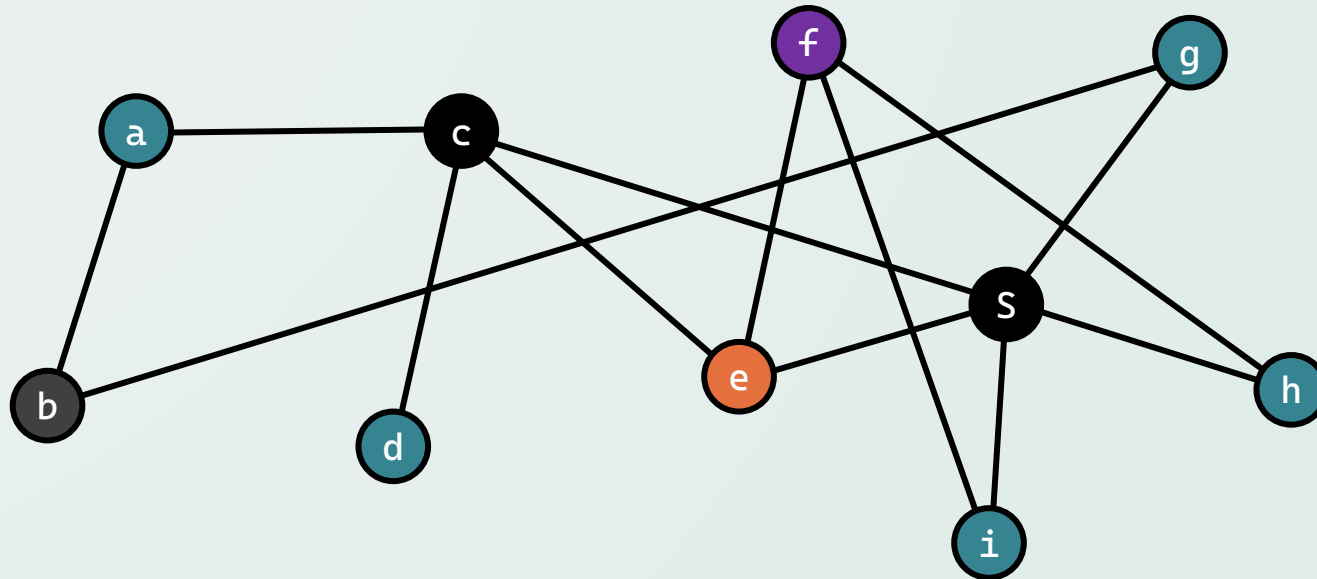
a	b	c	d	e	f	g	h	i	s
2	∞	1	2	1	∞	1	1	1	0

דוגמת הרצה של אלגוריתם החיפוש לרוחב



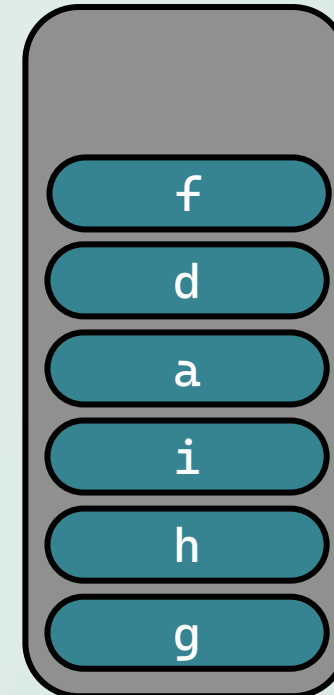
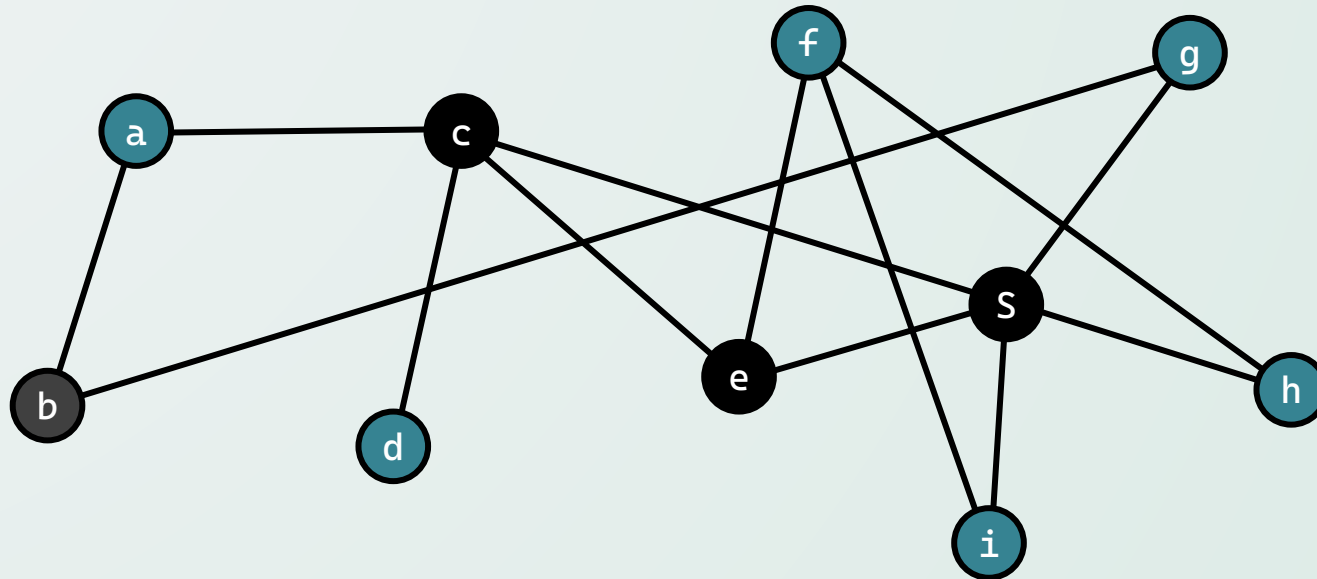
a	b	c	d	e	f	g	h	i	s
2	∞	1	2	1	∞	1	1	1	0

דוגמת הרצה של אלגוריתם החיפוש לרוחב



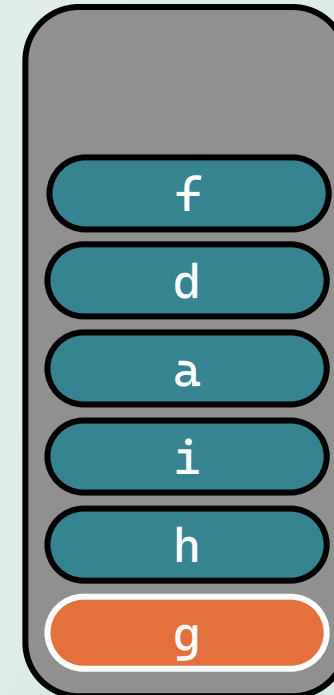
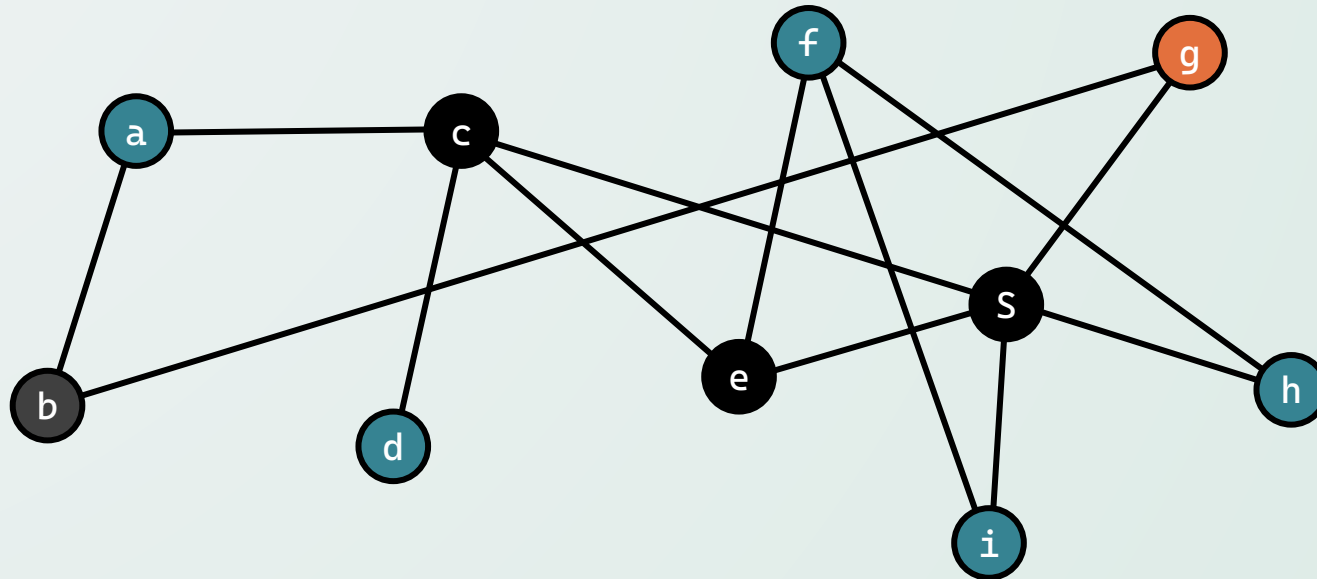
a	b	c	d	e	f	g	h	i	s
2	∞	1	2	1	2	1	1	1	0

דוגמת הרצה של אלגוריתם החיפוש לרוחב



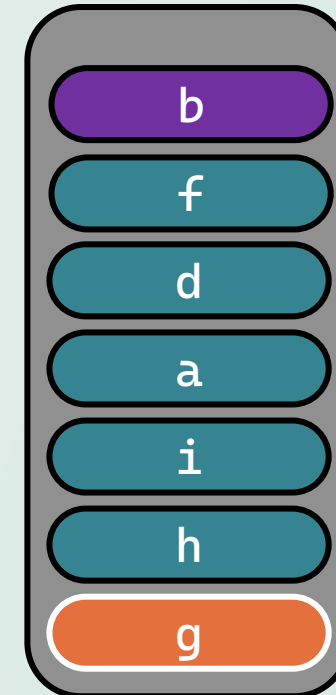
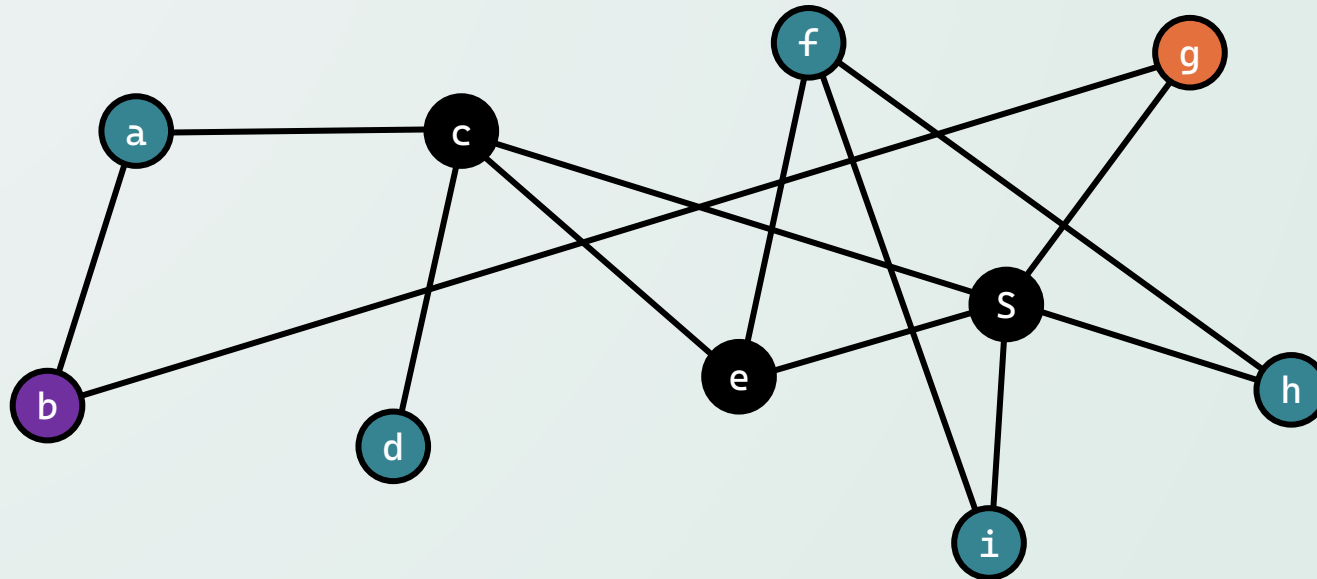
a	b	c	d	e	f	g	h	i	s
2	∞	1	2	1	2	1	1	1	0

דוגמת הרצה של אלגוריתם החיפוש לרוחב



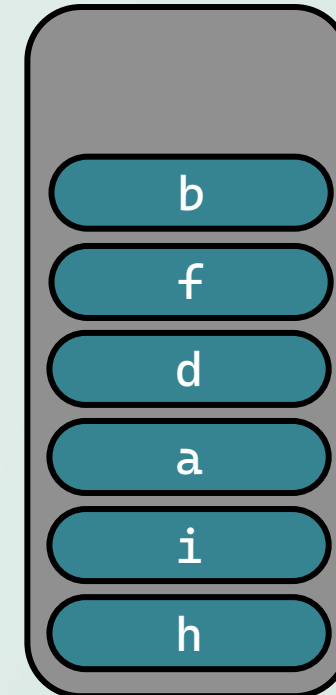
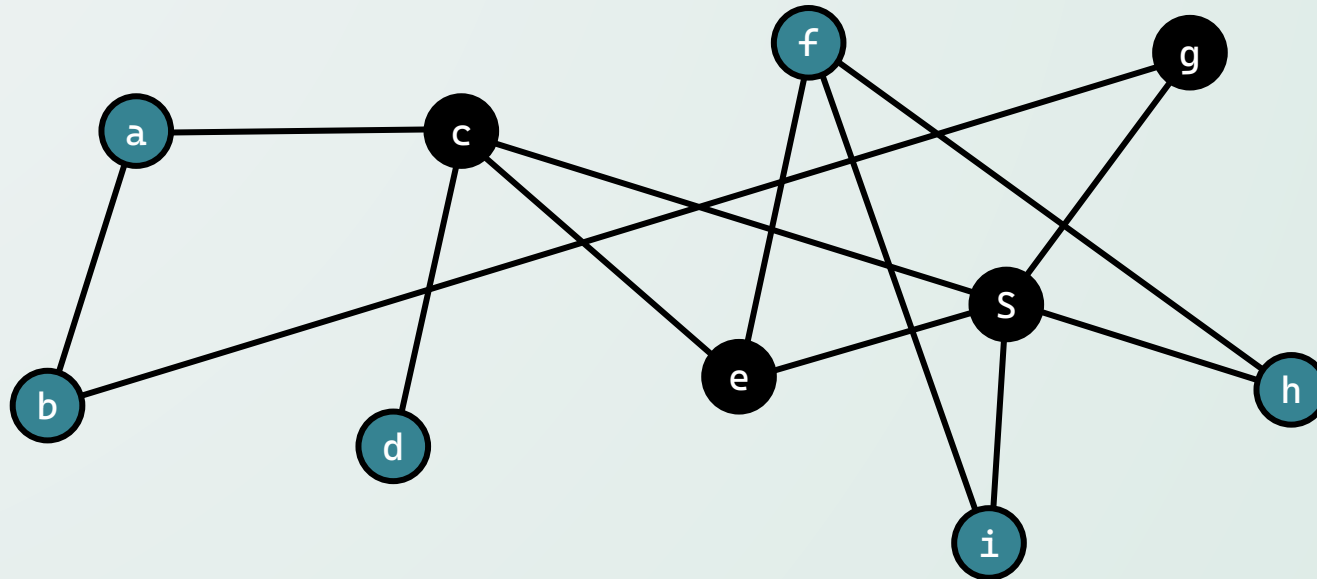
a	b	c	d	e	f	g	h	i	s
2	∞	1	2	1	2	1	1	1	0

דוגמת הרצה של אלגוריתם החיפוש לרוחב



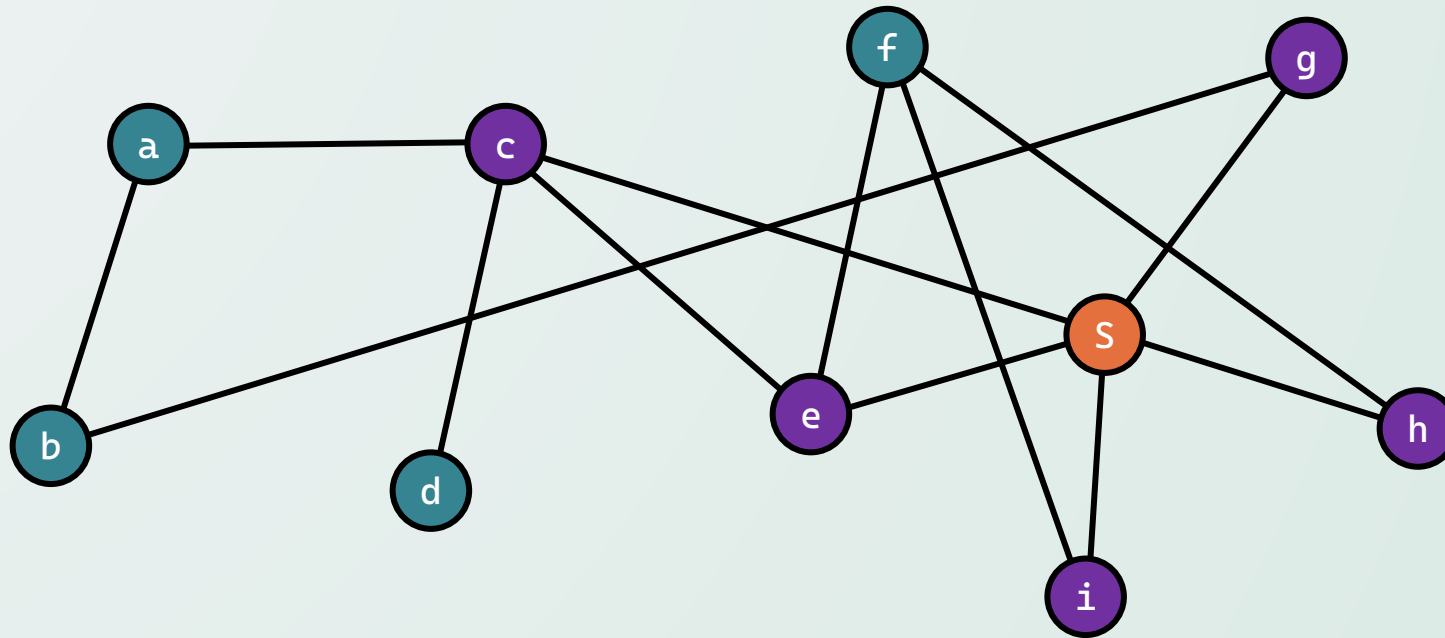
a	b	c	d	e	f	g	h	i	s
2	2	1	2	1	2	1	1	1	0

דוגמת הרצה של אלגוריתם החיפוש לרוחב



a	b	c	d	e	f	g	h	i	s
2	2	1	2	1	2	1	1	1	0

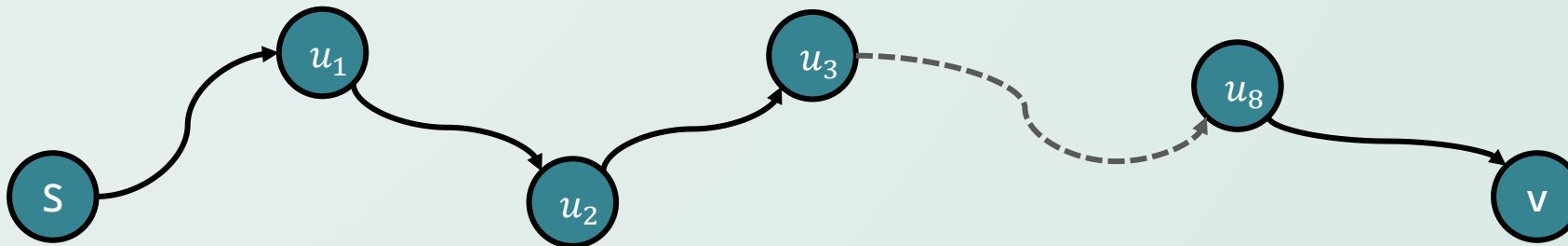
דוגמת הרצה של אלגוריתם החיפוש לרוחב



a	b	c	d	e	f	g	h	i	s
2	2	1	2	1	2	1	1	1	0

הוכחת נכונות

- נסמן $d(u,v)$ כמרחק הקצר ביותר מהצומת u לצומת v (בקשתות).
- **למה 1 (קשירות):** אם $d(S,v) < \infty$ אז גם $\text{dist}[v] < \infty$. כלומר, אם ניתן להגיע ל- v אז נגיע אליה.
- נניח בשלילה שקיימת צומת v כך ש- $d(S,v) < \infty$ אבל $\text{dist}[v] = \infty$. בפרט נסתכל על הצומת הכי קרובה שמקיימת תכונה זו.
- נסתכל על המסלול הקצר ביותר אל צומת זו $S \rightarrow u_1 \rightarrow u_2 \rightarrow \dots \rightarrow u_{k-1} \rightarrow v$ שאורכו בדיוק k .
- נתבונן על הצומת הלפני אחרונה במסלול זה. היא יותר קרובה מ- v למקור, ולכן הגענו אליה ועיבדנו אותה.
- אך כאשר האלגוריתם יעבד אותה, הוא יגלה את הצומת v שכן ישנה קשת (u_{k-1}, v) .



הוכחת נכונות

- **למה 2 (סדר):** ברגע שאנו מעבדים צומת v שמרחקה $d(S, v) = k$, כל הצמתים u שמרחקם $d(S, u) \leq k$ כבר התגלו.
- נניח בשלילה שקיימת צומת u כך ש- $k' = d(S, u) \leq k$ אך היא עדיין לא גולתה. בפרט נסתכל על הצומת הראשונה שמקיימת את התכונה הזו בריצת האלגוריתם.



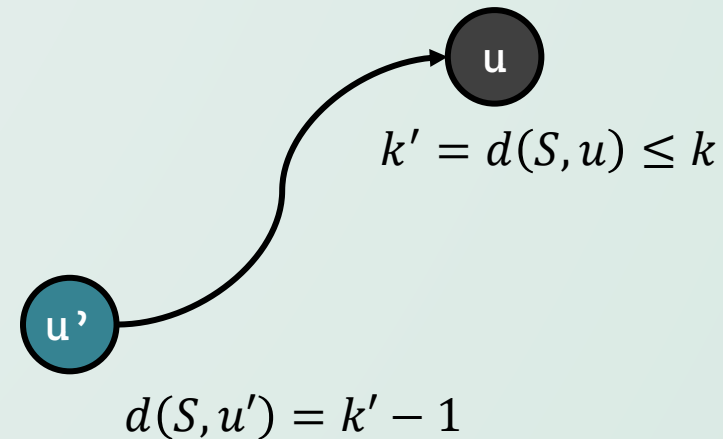
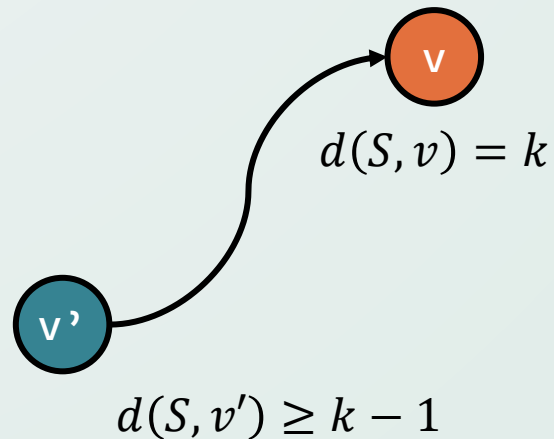
$$d(S, v) = k$$



$$k' = d(S, u) \leq k$$

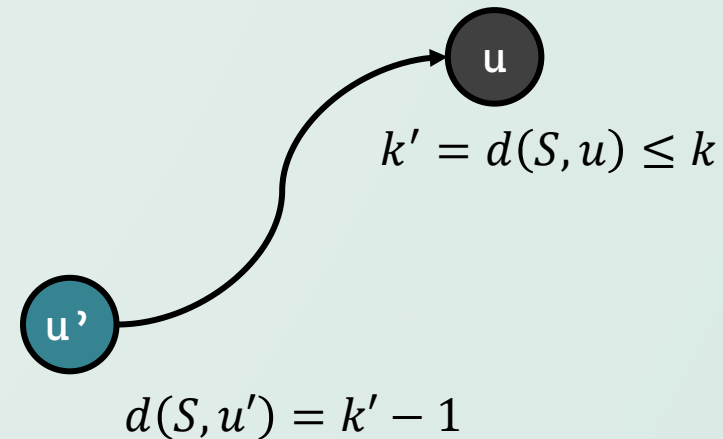
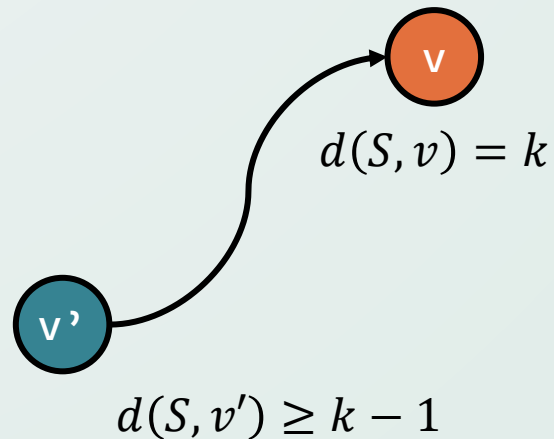
הוכחת נכונות

- **למה 2 (סדר):** ברגע שאנו מעבדים צומת v שמרחקה $d(S, v) = k$, כל הצמתים u שמרחקם $d(S, u) \leq k$ כבר התגלו.
- נתבונן על הצומת v' שבעת העיבוד שלה האלגוריתם גילה את הצומת v . המרחק שלה מהמקור הוא לכל הפחות $k - 1$, שכן אם הוא $k - 1$, מהסלול הקצר ביותר ל- v מרחקו $k > k - 1$.
- בנוסף נתבונן על הצומת u' שהיא הצומת הלפני אחרונה במסלול הקצר ביותר מהמקור ל- u . מרחקה מהמקור הוא בדיוק $d(S, u') = d(S, u) - 1$.



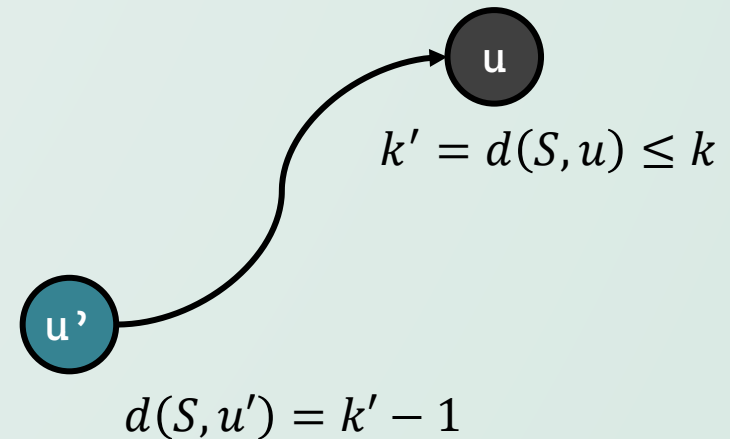
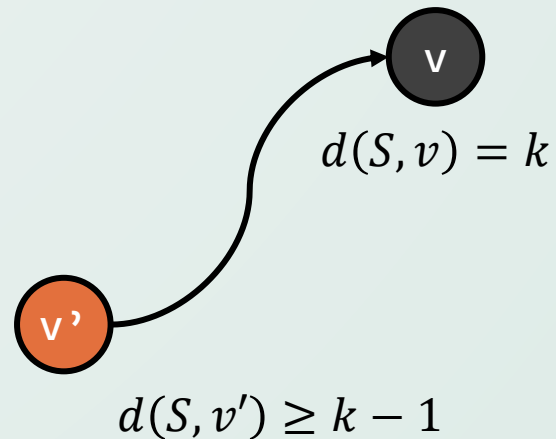
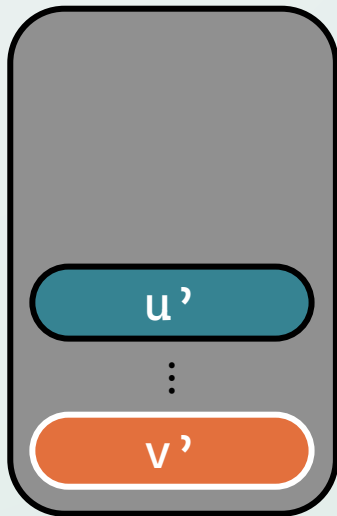
הוכחת נכונות

- **למה 2 (סדר):** ברגע שאנו מעבדים צומת v שמרחקה $d(S, v) = k$, כל הצמתים u שמרחקם $d(S, u) \leq k$ כבר התגלו.
- אנו מניחים כי $k' \leq k$ ולכן $k' - 1 \leq k - 1$.
- נשים לב כי הטענה מתקיימת בעבור u', v' שכן בחרנו את v, u להיות זוג הצמתים הראשון שלו הטענה לא מתקיימת.
- לכן, לפי הטענה, בעת עיבוד הצומת v' , הצומת u' כבר הייתה אמורה להתגלות.



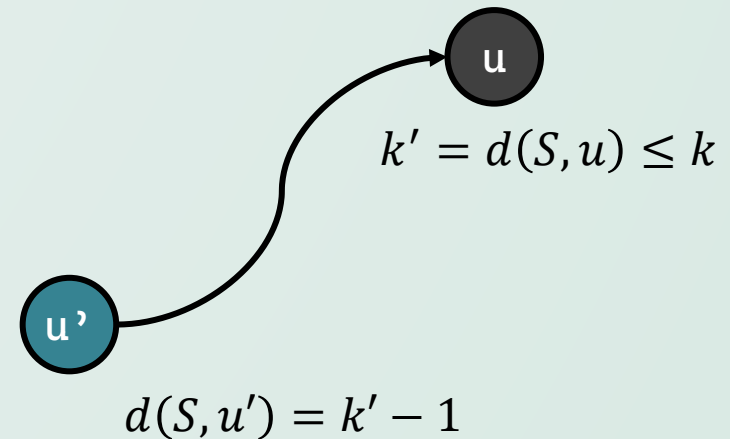
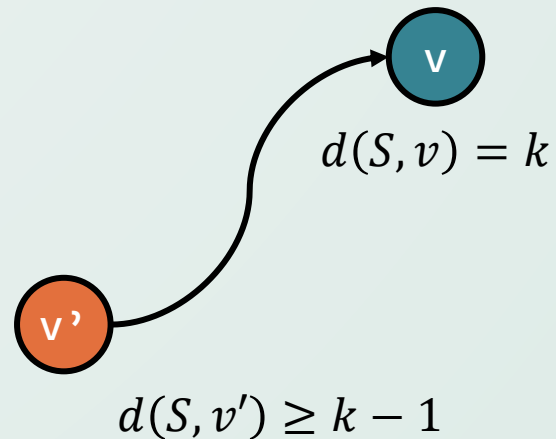
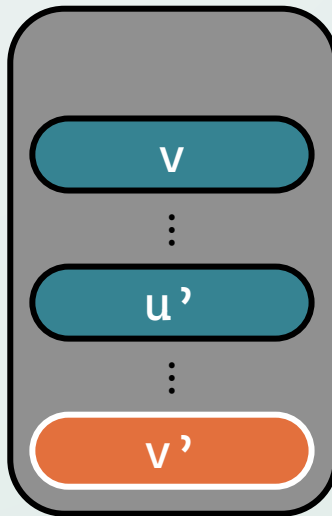
הוכחת נכונות

- **למה 2 (סדר):** ברגע שאנו מעבדים צומת v שמרחקה $d(S, v) = k$, כל הצמתים u שמרחקם $d(S, u) \leq k$ כבר התגלו.
- לכן, לפי הטענה, בעת עיבוד הצומת v' , הצומת u' כבר הייתה אמורה להתגלות.
- בואו נתבונן על מקרה זה! נקבל שבעת עיבוד v הצומת u כבר גולתה, בסתירה להנחה המקורית. ■



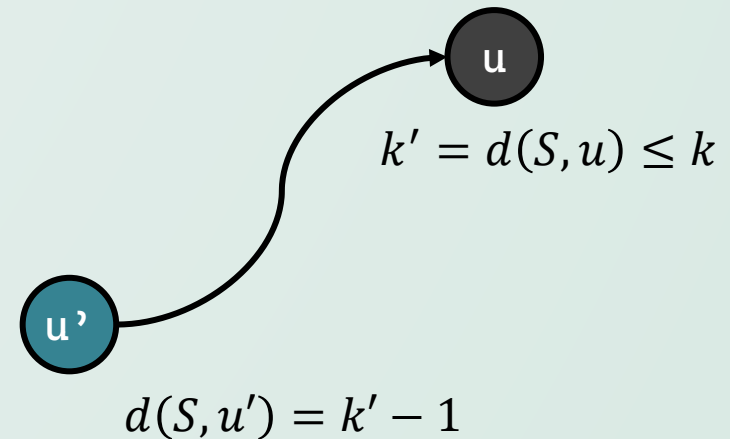
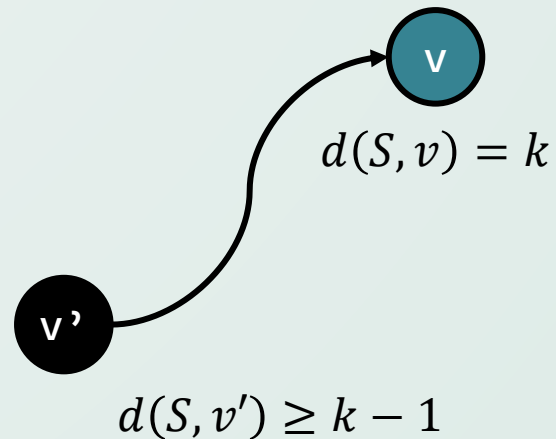
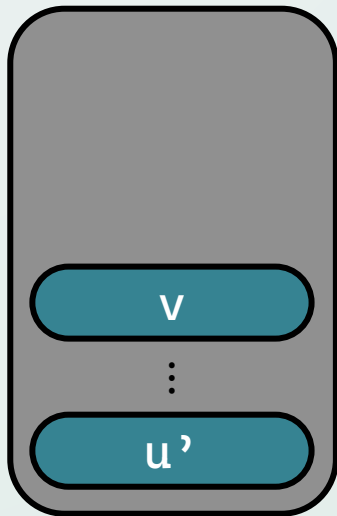
הוכחת נכונות

- **למה 2 (סדר):** ברגע שאנו מעבדים צומת v שמרחקה $d(S, v) = k$, כל הצמתים u שמרחקם $d(S, u) \leq k$ כבר התגלו.
- לכן, לפי הטענה, בעת עיבוד הצומת v' , הצומת u' כבר הייתה אמורה להתגלות.
- בואו נתבונן על מקרה זה! נקבל שבעת עיבוד v הצומת u כבר גולתה, בסתירה להנחה המקורית. ■



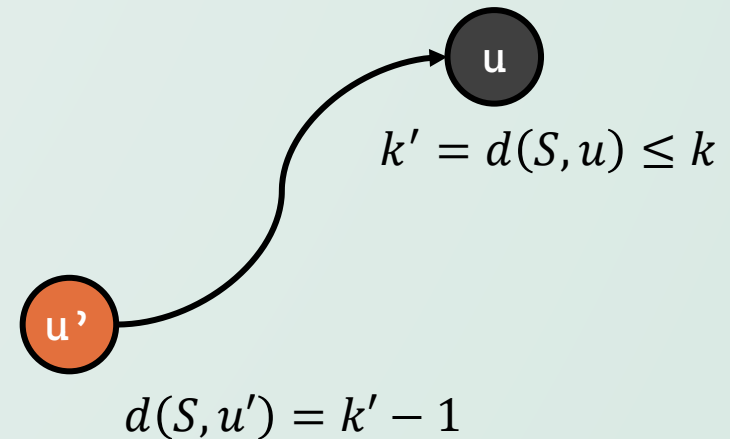
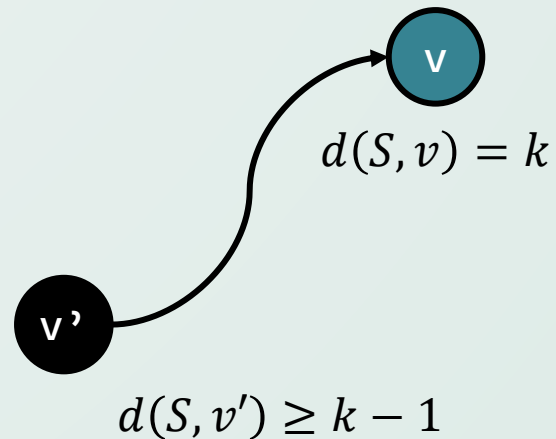
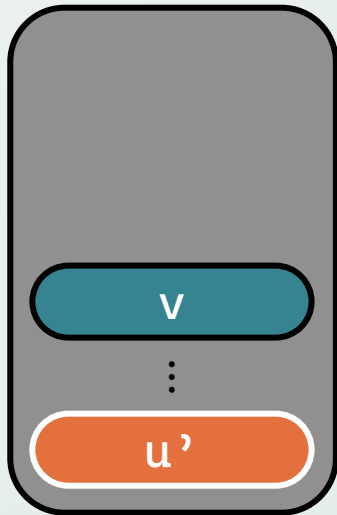
הוכחת נכונות

- **למה 2 (סדר):** ברגע שאנו מעבדים צומת v שמרחקה $d(S, v) = k$, כל הצמתים u שמרחקם $d(S, u) \leq k$ כבר התגלו.
- לכן, לפי הטענה, בעת עיבוד הצומת v' , הצומת u' כבר הייתה אמורה להתגלות.
- בואו נתבונן על מקרה זה! נקבל שבעת עיבוד v הצומת u כבר גולתה, בסתירה להנחה המקורית. ■



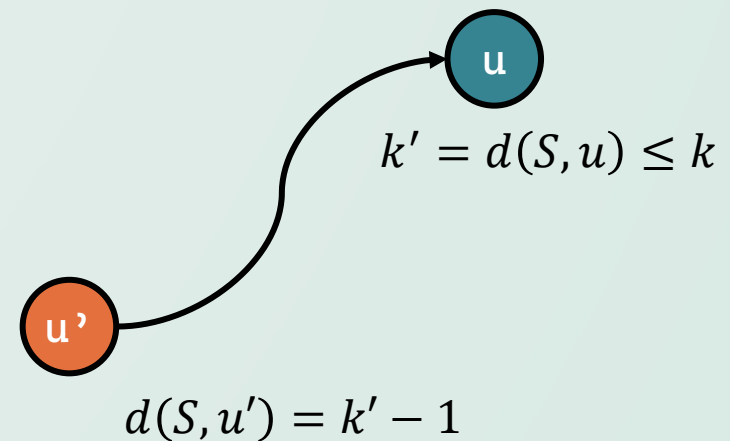
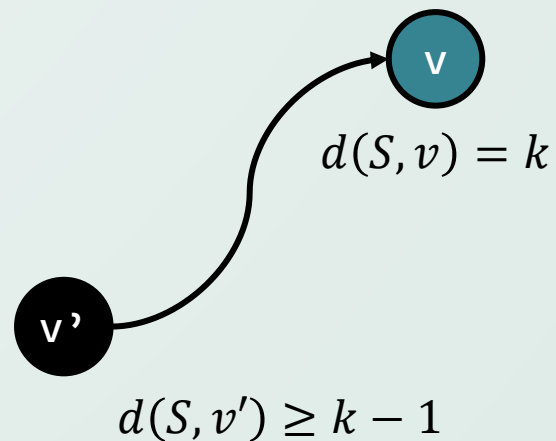
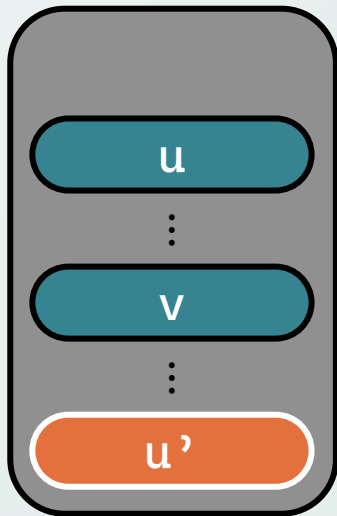
הוכחת נכונות

- **למה 2 (סדר):** ברגע שאנו מעבדים צומת v שמרחקה $d(S, v) = k$, כל הצמתים u שמרחקם $d(S, u) \leq k$ כבר התגלו.
- לכן, לפי הטענה, בעת עיבוד הצומת v' , הצומת u' כבר הייתה אמורה להתגלות.
- בואו נתבונן על מקרה זה! נקבל שבעת עיבוד v הצומת u כבר גולתה, בסתירה להנחה המקורית. ■



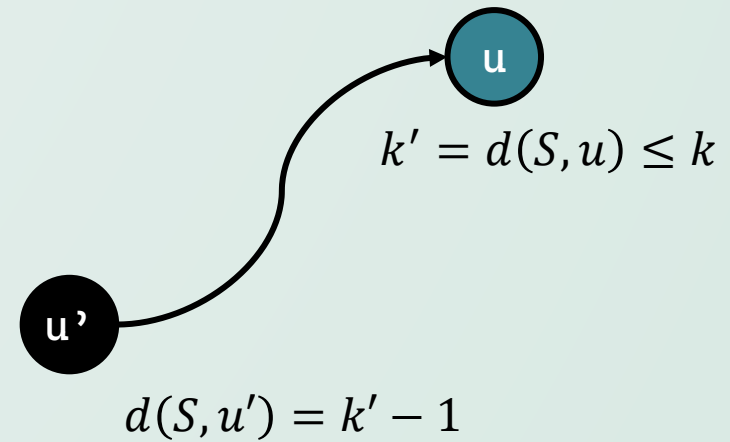
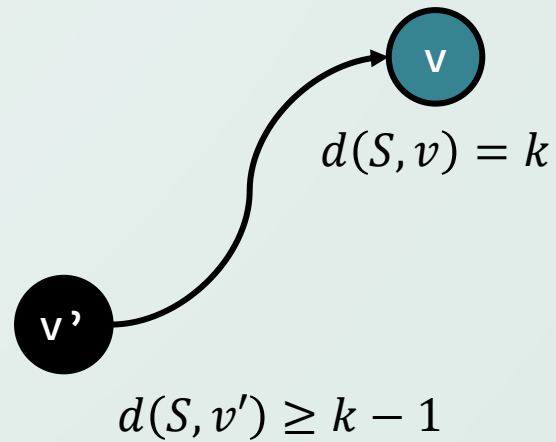
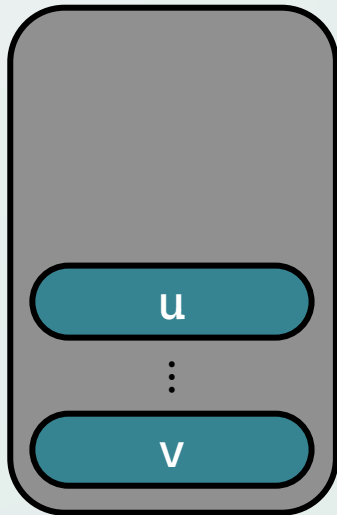
הוכחת נכונות

- **למה 2 (סדר):** ברגע שאנו מעבדים צומת v שמרחקה $d(S, v) = k$, כל הצמתים u שמרחקם $d(S, u) \leq k$ כבר התגלו.
- לכן, לפי הטענה, בעת עיבוד הצומת v' , הצומת u' כבר הייתה אמורה להתגלות.
- בואו נתבונן על מקרה זה! נקבל שבעת עיבוד v הצומת u כבר גולתה, בסתירה להנחה המקורית. ■



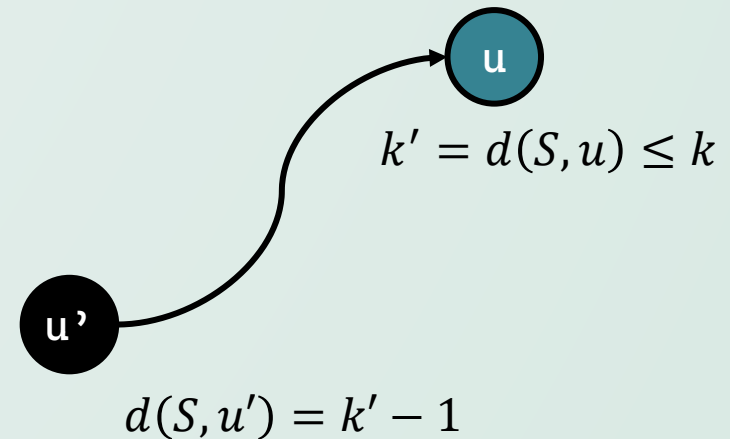
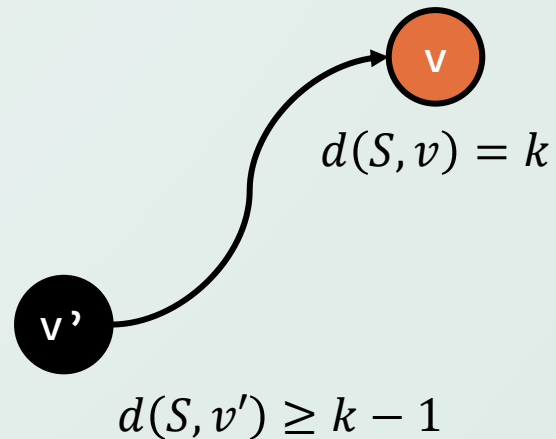
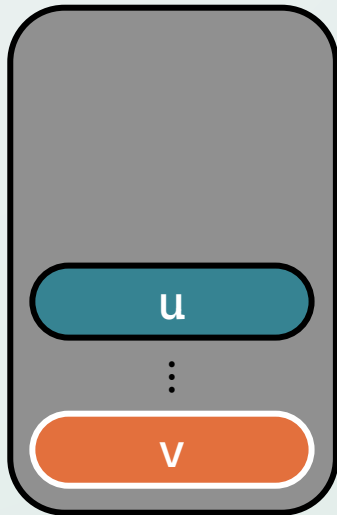
הוכחת נכונות

- **למה 2 (סדר):** ברגע שאנו מעבדים צומת v שמרחקה $d(S, v) = k$, כל הצמתים u שמרחקם $d(S, u) \leq k$ כבר התגלו.
- לכן, לפי הטענה, בעת עיבוד הצומת v' , הצומת u' כבר הייתה אמורה להתגלות.
- בואו נתבונן על מקרה זה! נקבל שבעת עיבוד v הצומת u כבר גולתה, בסתירה להנחה המקורית. ■



הוכחת נכונות

- **למה 2 (סדר):** ברגע שאנו מעבדים צומת v שמרחקה $d(S, v) = k$, כל הצמתים u שמרחקם $d(S, u) \leq k$ כבר התגלו.
- לכן, לפי הטענה, בעת עיבוד הצומת v' , הצומת u' כבר הייתה אמורה להתגלות.
- בואו נתבונן על מקרה זה! נקבל שבעת עיבוד v הצומת u כבר גולתה, בסתירה להנחה המקורית. ■



הוכחת נכונות

- נקרא לצמתים שהאלגוריתם עוד לא הגיע אליהם ככאלה שלא התגלו. נקרא לצומת שהתגלתה אבל עוד לא עובדה (נמצאת בתור) בתור צומת חיה. לאחר שצומת עובדה, נאמר שהיא מתה.
- **למה 3 (תור):** בכל רגע נתון, התור יהיה חצוי לשניים: צמתים v המקיימים $d(S, v) = k$ וצמתים v המקיימים $d(S, v) = k + 1$ (אם בכלל).
- נסו להוכיח טענה זו בעצמכם!



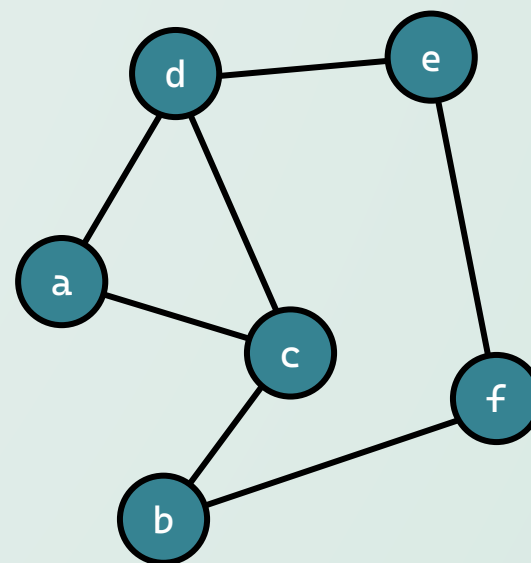
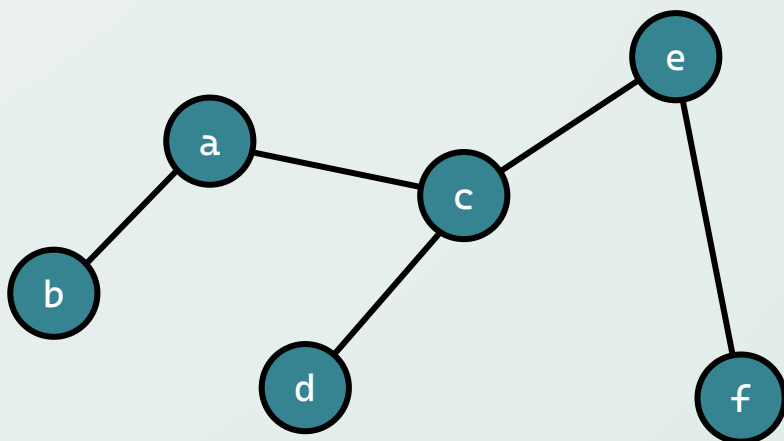
הוכחת נכונות

- **משפט:** לאחר שהצומת v מעובדת מתקיים $\text{dist}[v] = d(S, v)$. נוכיח טענה זו באינדוקציה.
- **בסיס האינדוקציה:** לאחר ש- S מעובדת מתקיים $d(S, S) = \text{dist}[S] = 0$ כנדרש.
- **צעד האינדוקציה:** נניח כי הטענה נכונה בעבור כל הצמתים u כך ש- $d(S, u) < k$. נוכיח את הטענה בעבור צומת v כך ש- $d(S, v) = k$.
- נניח שהצומת v התגלתה במהלך העיבוד של u על ידי הקשת (u, v) . קיימת צומת כזו לפי **למה 1**.
- מצד אחד, מתקיים $d(S, u) < d(S, v) = k$, לפי **למה 2**.
- מהצד השני, מתקיים $d(S, v) \leq d(S, u) + 1$ לפי **למה 3**.
- כלומר סה"כ $k - 1 \leq d(S, u) < k$ ולכן $\text{dist}[u] = k - 1 = d(S, u)$.
- בעצם הוכחנו שאם ישנה צומת שמרחקה k מהמקור, היא מתגלה ע"י צומת שמרחקה $k - 1$ מהמקור.
- לכן האלגוריתם יקבע $\text{dist}[v] = \text{dist}[u] + 1 = k = d(S, v)$ כנדרש. ■

בדיקת גרף נתון כעץ

בדיקת גרף נתון כעץ

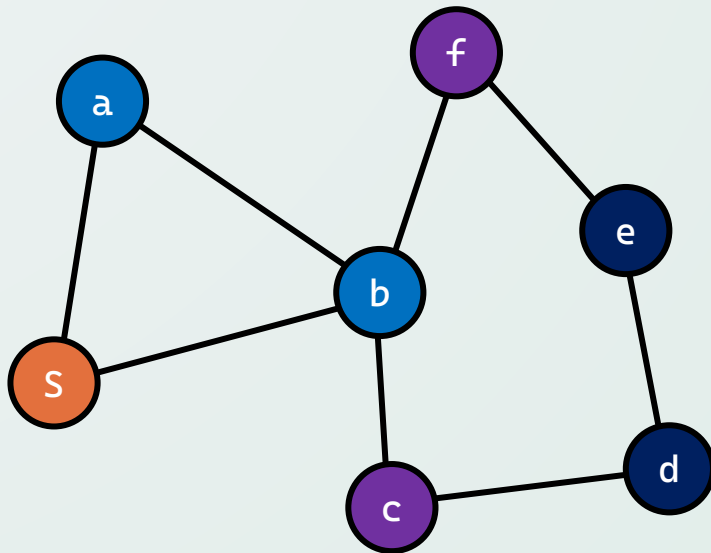
- נתון גרף לא מכוון $G = (V, E)$ שאינו בהכרח קשיר.
- תארו אלגוריתם, בסיבוכיות טובה ככל יכולתכם, המכריע האם הגרף הנתון הוא עץ. הוכיחו את סיבוכיותו ונכונותו של האלגוריתם שלכם.



הפתרון הפשוט

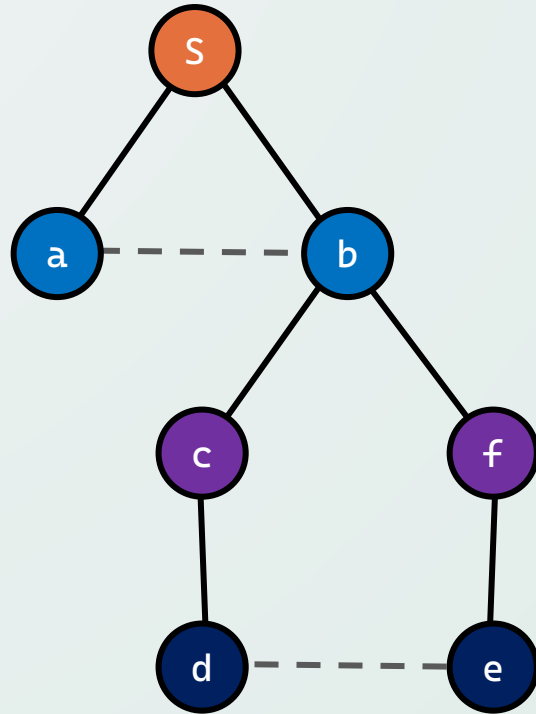
- כדי להכריע האם הגרף $G = (V, E)$ הוא עץ מספיק לוודא שהוא קשיר ו- $|E| + 1 = |V|$.
- **מלמה 1**, אנו יודעים כי נגיע לכל הצמתים ברכיב הקשירות של המקור.
- אם **נבחר מקור שרירותית**, נוכל לבדוק בסיום הריצה האם הגענו לכל הצמתים.
- בנוסף, נוכל לספור את מספר הקשתות והצמתים ולוודא שהתנאי השני מתקיים.
- ומה אם אנחנו מתעקשים, ובכל זאת רוצים לדעת האם קיים מעגל בגרף?

ניתוח בעזרת עץ החיפוש לרוחב



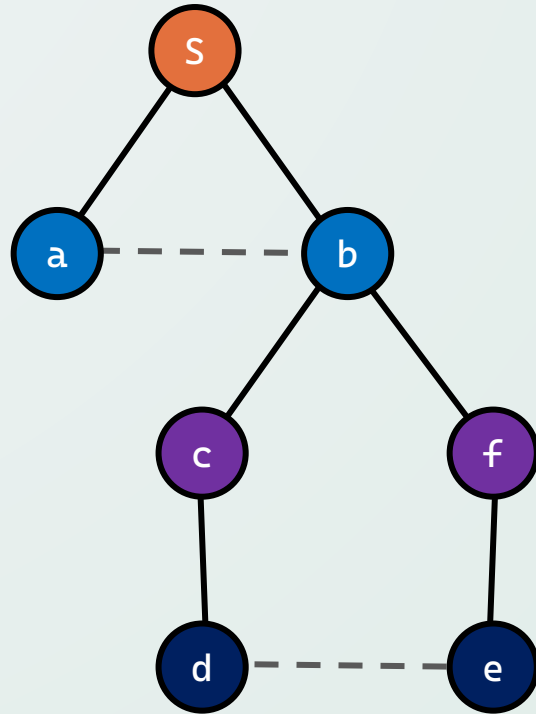
- לשם הפשטות, נתבונן על גרף לא מכוון וקשיר.
- עץ ה-BFS יכיל את כל הקשתות שבהם השתמשנו בטיול ה-BFS על גרף זה, כאשר הרמה ה- i בו תכיל את כל הצמתים במרחק i מהמקור.
- קשתות הגרף המקורי שאינן נמצאות בעץ הן קשתות המחברות שתי רמות זהות או סמוכות.
- אם הייתה קשת המחברת שתי רמות שמרחקן 2 ומעלה, הרי שאלגוריתם ה-BFS היה משתמש בה ומקרב את כל הצמתים ברמה התחתונה לרמה גבוהה יותר.
- קשת לא תופיע בעץ ה-BFS אם בעת עיבוד צומת אחת שלה, הצומת השנייה כבר עובדה קודם לכן.

ניתוח בעזרת עץ החיפוש לרוחב



- לשם הפשטות, נתבונן על גרף לא מכוון וקשיר.
- עץ ה-BFS יכיל את כל הקשתות שבהם השתמשנו בטיול ה-BFS על גרף זה, כאשר הרמה ה- i בו תכיל את כל הצמתים במרחק i מהמקור.
- קשתות הגרף המקורי שאינן נמצאות בעץ הן קשתות המחברות שתי רמות זהות או סמוכות.
- אם הייתה קשת המחברת שתי רמות שמרחקן 2 ומעלה, הרי שאלגוריתם ה-BFS היה משתמש בה ומקרב את כל הצמתים ברמה התחתונה לרמה גבוהה יותר.
- קשת לא תופיע בעץ ה-BFS אם בעת עיבוד צומת אחת שלה, הצומת השנייה כבר עובדה קודם לכן.

מציאת מעגל בגרף בעזרת BFS



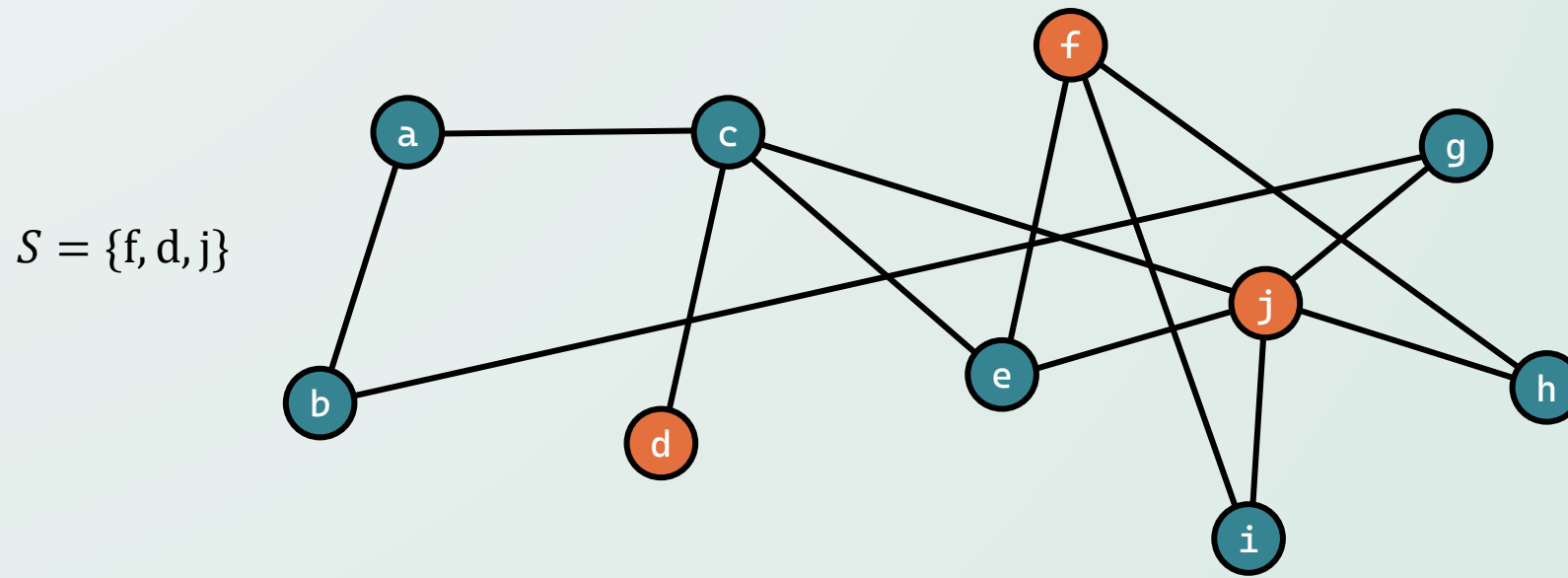
- כדי למצוא מעגל, מספיק להתבונן על עץ ה-BFS.
- אם כל הקשתות של הגרף נמצאות בעץ ה-BFS, הרי שהגרף המקורי הוא עץ בפני עצמו.
- אחרת, קיימת קשת אחת לפחות שהיא קשת חוצה ולא נמצאת בעץ ה-BFS.
- קל לראות שכל קשת חוצה כזאת יוצרת מעגל אחד לפחות בגרף המקורי.
- נשמור לכל צומת שאנו מגלים, את השכן שלה שממנו הגענו אליה.
- כאשר אנו מטפלים בצומת v , אם יש לה שכן u (שאינו השכן שממנו הגענו אליה) שכבר טיפלנו בו, הרי שהקשת (u,v) היא קשת חוצה ולכן יוצרת מעגל.

מסלולים קצרים מקבוצת קודקודים

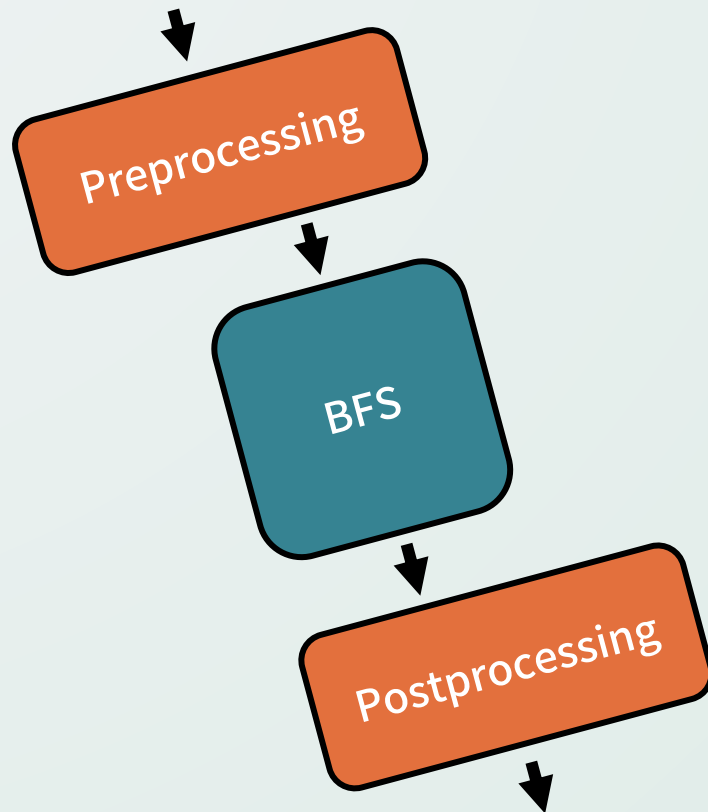
Shortest path from a group of vertices

מסלולים קצרים מקבוצת קודקודים

- נתון גרף $G = (V, E)$ וקבוצת קודקודים $S \subseteq V$.
- נסמן את המרחק המינימלי של צומת מקבוצת קודקודים S כ- $d(S, v)$ בעבור $v \in V$ בתור $\min\{d(u, v) \mid u \in S\}$.
- תארו אלגוריתם, בסיבוכיות טובה ככל שתוכלו, המחשב את המרחק המינימלי של כל הצמתים בגרף מקבוצת הקודקודים S . הוכיחו את סיבוכיותו ונכונותו של האלגוריתם שלכם.

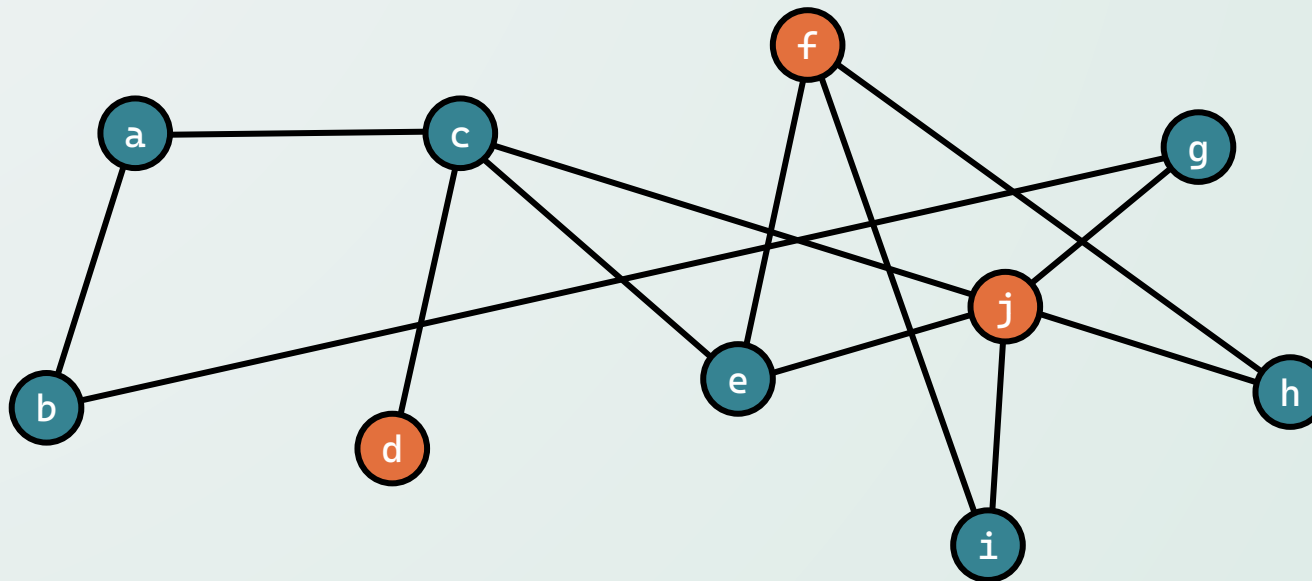


רדוקציה קלה להוכחה



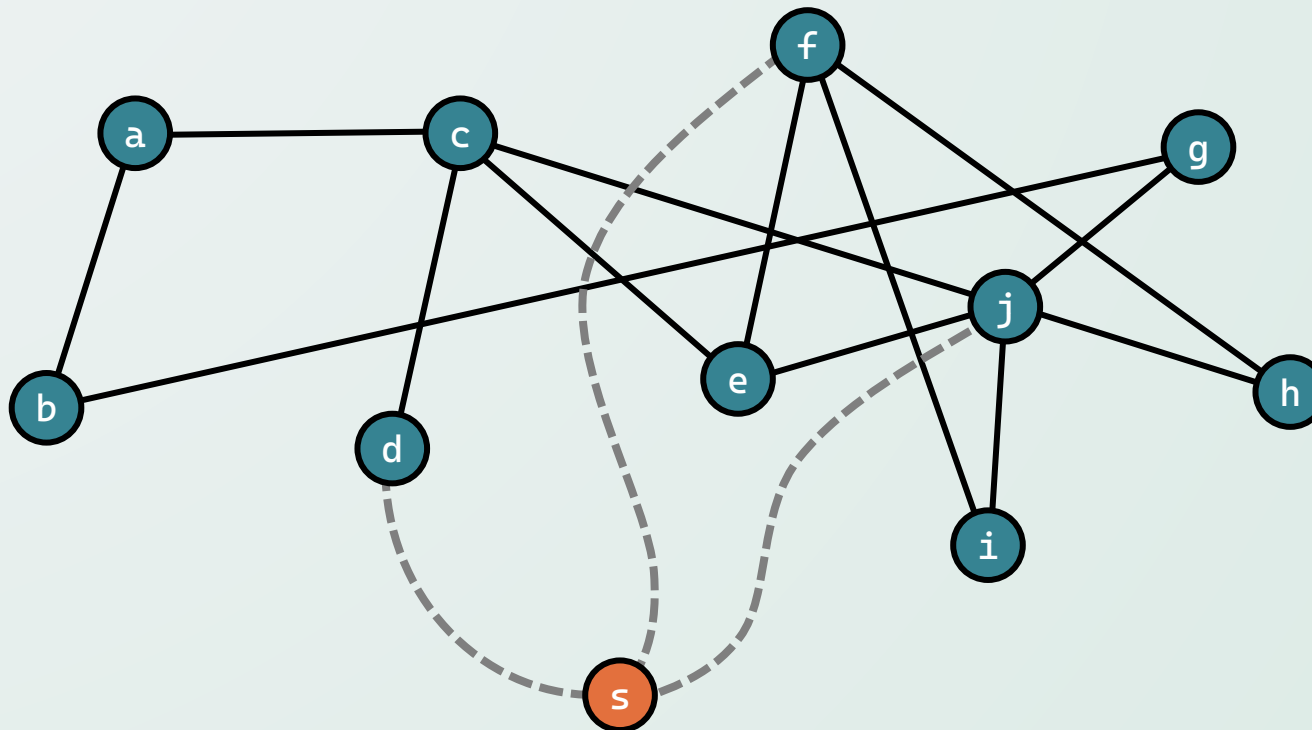
- הפתרון שקובע את המרחק של כל הצמתים מהקבוצה S לאפס ומכניס את כולם לתור, ואז מתחיל בריצת ה-BFS - נכון.
- עם זאת, הוכחת הנכונות שלו **דורשת עבודה**.
- היינו רוצים להשתמש באלגוריתם ה-BFS בתור **קופסא שחורה**, ולשנות רק את הקלט והפלט שלו.
- וכך לא נצטרך להוכיח מחדש את נכונות BFS עצמו.
- כן נצטרך להוכיח את נכונות השינויים שעשינו לקלט, ולפלט ה-BFS ולוודא שהאלגוריתם הכולל אכן פולט את התוצאה הרצויה.

הפתרון



- אלגוריתם ה-BFS יודע לקבל צומת מקור אחת בלבד.
- נוכל לחבר את כל הצמתים ב- S לצומת חדשה s , ולקבוע אותה כמקור לאלגוריתם ה-BFS.
- בסיום ריצת האלגוריתם נפחית 1 ממרחקי כל הצמתים.
- קל לראות ולהוכיח שהפתרון הנ"ל נכון. הוכיחו בעצמכם!

הפתרון



- אלגוריתם ה-BFS יודע לקבל צומת מקור אחת בלבד.
- נוכל לחבר את כל הצמתים ב- S לצומת חדשה s , ולקבוע אותה כמקור לאלגוריתם ה-BFS.
- בסיום ריצת האלגוריתם נפחית 1 ממרחקי כל הצמתים.
- קל לראות ולהוכיח שהפתרון הנ"ל נכון. הוכיחו בעצמכם!

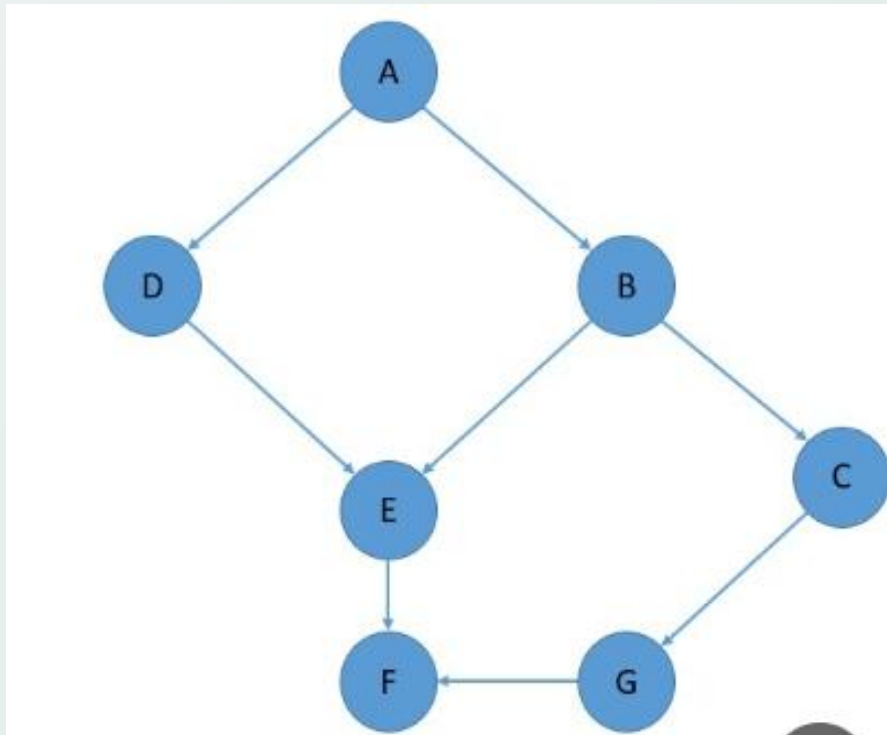
מיון טופולוגי

Topological Sorting

תזכורת – DAG

• גרף $G = (V, E)$ יקרא DAG (Directed Acyclic Graph) אם"מ הוא מכוון ואין בו מעגלים.

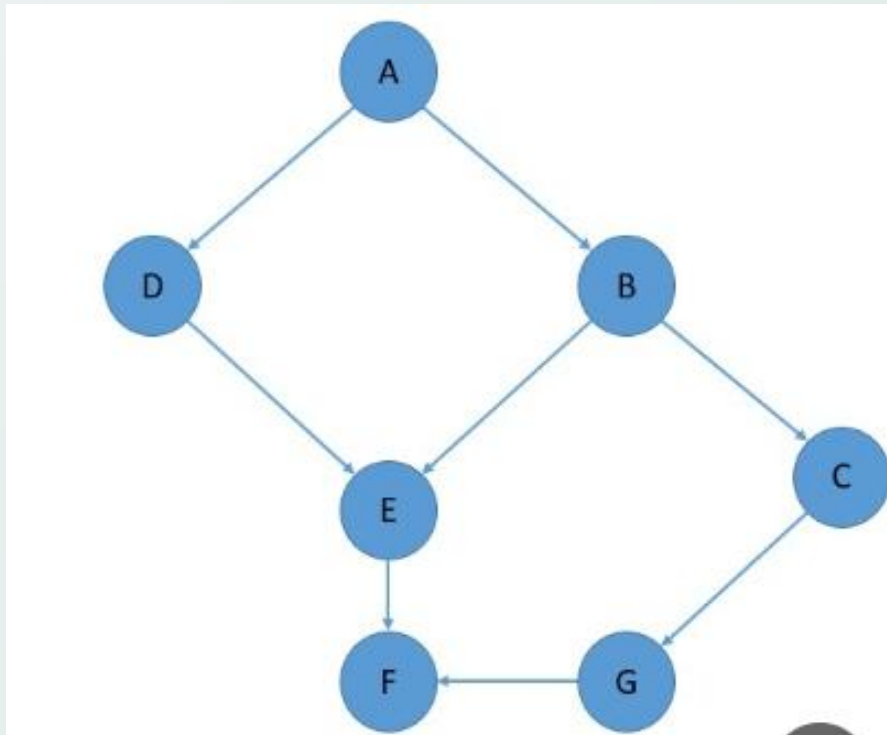
- DAG:



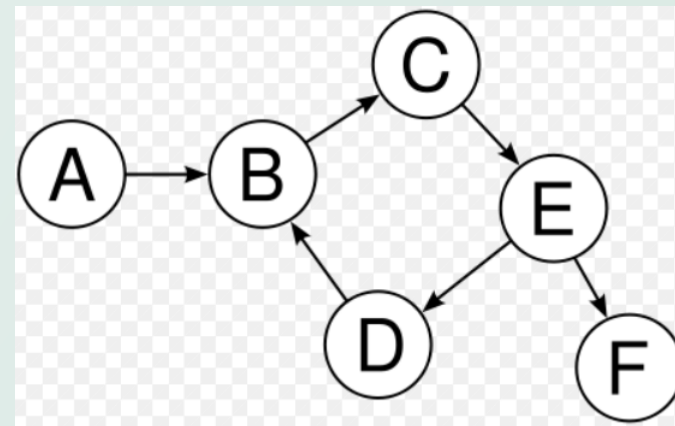
תזכורת – DAG

• גרף $G = (V, E)$ יקרא DAG (Directed Acyclic Graph) אם"מ הוא מכוון ואין בו מעגלים.

• DAG:



לא DAG:



מיון טופולוגי

- בהינתן DAG G , עם קבוצת קודקודים $V = \{v_1 \dots v_n\}$, מיון טופולוגי של הגרף, הוא פונקציה $f: V \rightarrow [n]$ המקיימת: $(u, v) \in E \Rightarrow f(u) < f(v)$. בפרט, כל מסלול בגרף יהיה מונוטוני עולה ממש.

מיון טופולוגי

- בהינתן DAG G , עם קבוצת קודקודים $V = \{v_1 \dots v_n\}$, מיון טופולוגי של הגרף, הוא פונקציה $f: V \rightarrow [n]$ המקיימת: $(u, v) \in E \Rightarrow f(u) < f(v)$. בפרט, כל מסלול בגרף יהיה מונוטוני עולה ממש.
- פחות בפורמליות, המיון הטופולוגי ממספר את הקודקודים כך שקשתות הולכות מקטן לגדול.

מיון טופולוגי

- בהינתן DAG G , עם קבוצת קודקודים $V = \{v_1 \dots v_n\}$, מיון טופולוגי של הגרף, הוא פונקציה $f: V \rightarrow [n]$ המקיימת: $(u,v) \in E \Rightarrow f(u) < f(v)$. בפרט, כל מסלול בגרף יהיה מונוטוני עולה ממש.
- פחות בפורמליות, המיון הטופולוגי ממספר את הקודקודים כך שקשתות הולכות מקטן לגדול.
- נרצה למצוא מיון טופולוגי לגרף DAG (הוא לא קיים באף גרף אחר), בזמן לינארי. בכיתה עם עמית ראיתם אלגוריתם לעשות את זה, שהשתמש בכלים די כבדים (DFS), להפוך את הגרף ושוב להפעיל (DFS). פה נראה אלגוריתם הרבה יותר פשוט למיון.

מיון טופולוגי

- בהינתן DAG G , עם קבוצת קודקודים $V = \{v_1 \dots v_n\}$, מיון טופולוגי של הגרף, הוא פונקציה $f: V \rightarrow [n]$ המקיימת: $(u,v) \in E \Rightarrow f(u) < f(v)$. בפרט, כל מסלול בגרף יהיה מונוטוני עולה ממש.
- פחות בפורמליות, המיון הטופולוגי ממספר את הקודקודים כך שקשתות הולכות מקטן לגדול.
- נרצה למצוא מיון טופולוגי לגרף DAG (הוא לא קיים באף גרף אחר), בזמן לינארי. בכיתה עם עמית ראיתם אלגוריתם לעשות את זה, שהשתמש בכלים די כבדים (DFS), להפוך את הגרף ושוב להפעיל (DFS). פה נראה אלגוריתם הרבה יותר פשוט למיון.
- המיון הטופולוגי הוא אחד הכלים השימושיים כשעובדים עם DAG, ותמיד כדאי לחשוב עליו.

אבחנה על DAG

- נתבונן באבחנה הבאה – אם גרף G הוא DAG, אזי קיים בו קודקוד $v \in V$ עם דרגת כניסה אפס, כלומר אין אף קשת (u, v) .

אבחנה על DAG

- נתבונן באבחנה הבאה – אם גרף G הוא DAG, אזי קיים בו קודקוד $v \in V$ עם דרגת כניסה אפס, כלומר אין אף קשת (u, v) .
- למה זה נכון? נניח בשלילה שלא. אזי נסתכל על קודקוד שרירותי v . בפרט יש צומת עם קשת ל- v . נסתכל על הצומת הזאת, ועם אותו הטיעון בדיוק נטען שיש צומת הקודמת לה.

אבחנה על DAG

- נתבונן באבחנה הבאה – אם גרף G הוא DAG, אזי קיים בו קודקוד $v \in V$ עם דרגת כניסה אפס, כלומר אין אף קשת (u, v) .
- למה זה נכון? נניח בשלילה שלא. אזי נסתכל על קודקוד שרירותי v . בפרט יש צומת עם קשת ל- v . נסתכל על הצומת הזאת, ועם אותו הטיעון בדיוק נטען שיש צומת הקודמת לה.
- אחרי שהמשכנו ככה $(+1)|V|$ פעמים, בכל פעם הטיעון היה תקף, והייתה קשת שמובילה לקודקוד. כיוון שיש לנו $|V|$ קודקודים, בוודאות לפי עקרון שובך היונים עברנו על צומת פעמיים. אם עברנו פעמיים, זה אומר שיש מעגל, והגרף לא DAG, בסתירה להנחה.
- כלומר אכן קיים קודקוד עם דרגת כניסה אפס.

אלגוריתם (KAHN)

- לפי האבחנה, קיים קודקוד עם דרגת כניסה אפס. כיוון שאין קשתות שמגיעות אליו, נמספר אותו במספר 1.

אלגוריתם (KAHN)

- לפי האבחנה, קיים קודקוד עם דרגת כניסה אפס. כיוון שאין קשתות שמגיעות אליו, נמספר אותו במספר 1.
- כעת נמחק את הקודקוד הזה. הגרף שנשאר הוא עדיין DAG, ואותו נרצה למספר במספרים $2 \dots n$. בפרט, אם נקבל מיון חוקי של הגרף אחרי הסתרת הקודקוד, המיון יהיה חוקי גם ללא הסתרת הקודקוד, כיוון שהדבר היחיד שיכול להשתנות אלה הקשתות מהקודקוד הראשון לשאר. אמנם, סימנו אותו בתור 1, ולכן לכל קודקוד אחר המספור שלו יהיה גבוה יותר והמיון אכן חוקי.

אלגוריתם (KAHN)

- לפי האבחנה, קיים קודקוד עם דרגת כניסה אפס. כיוון שאין קשתות שמגיעות אליו, נמספר אותו במספר 1.
- כעת נמחק את הקודקוד הזה. הגרף שנשאר הוא עדיין DAG, ואותו נרצה למספר במספרים $2 \dots n$. בפרט, אם נקבל מיון חוקי של הגרף אחרי הסתרת הקודקוד, המיון יהיה חוקי גם ללא הסתרת הקודקוד, כיוון שהדבר היחיד שיכול להשתנות אלה הקשתות מהקודקוד הראשון לשאר. אמנם, סימנו אותו בתור 1, ולכן לכל קודקוד אחר המספור שלו יהיה גבוה יותר והמיון אכן חוקי.
- במימוש נאיבי, נצטרך לחפש כל פעם קודקוד עם דרגת כניסה אפס, בזמן לינארי. זה יביא לסיבוכיות ריצה של $O(|V||E|)$. ננסה להוריד את הסיבוכיות ללינארית.

שיפור לזמן לינארי

- בתור עיבוד מקדים, נוכל לשמור רשימה של כל הצמתים עם דרגת כניסה אפס. זה יקח זמן לינארי, אם נשתמש ברשימת שכנויות. בנוסף, לכל צומת נשמור את דרגת הכניסה, גם כן בזמן לינארי.

שיפור לזמן לינארי

- בתור עיבוד מקדים, נוכל לשמור רשימה של כל הצמתים עם דרגת כניסה אפס. זה יקח זמן לינארי, אם נשתמש ברשימת שכנויות. בנוסף, לכל צומת נשמור את דרגת הכניסה, גם כן בזמן לינארי.
- מתוך הרשימה נבחר שרירותית קודקוד, נמספר אותו באחד. כשנסתיר אותו, נצטרך להוריד אחד מדרגת הכניסה של כל הקודקודים שיש מהקודקוד קשת אליהם. אם לאחד מהצמתים הללו דרגת הכניסה הפכה ל0, נוסיף אותו לרשימה של הצמתים עם דרגת כניסה אפס.

שיפור לזמן לינארי

- בתור עיבוד מקדים, נוכל לשמור רשימה של כל הצמתים עם דרגת כניסה אפס. זה יקח זמן לינארי, אם נשתמש ברשימת שכנויות. בנוסף, לכל צומת נשמור את דרגת הכניסה, גם כן בזמן לינארי.
- מתוך הרשימה נבחר שרירותית קודקוד, נמספר אותו באחד. כשנסתיר אותו, נצטרך להוריד אחד מדרגת הכניסה של כל הקודקודים שיש מהקודקוד קשת אליהם. אם לאחד מהצמתים הללו דרגת הכניסה הפכה ל0, נוסיף אותו לרשימה של הצמתים עם דרגת כניסה אפס.
- נחזור ונבחר קודקוד עם דרגת כניסה אפס, נמספר אותו ב2 ונחזור על התהליך.

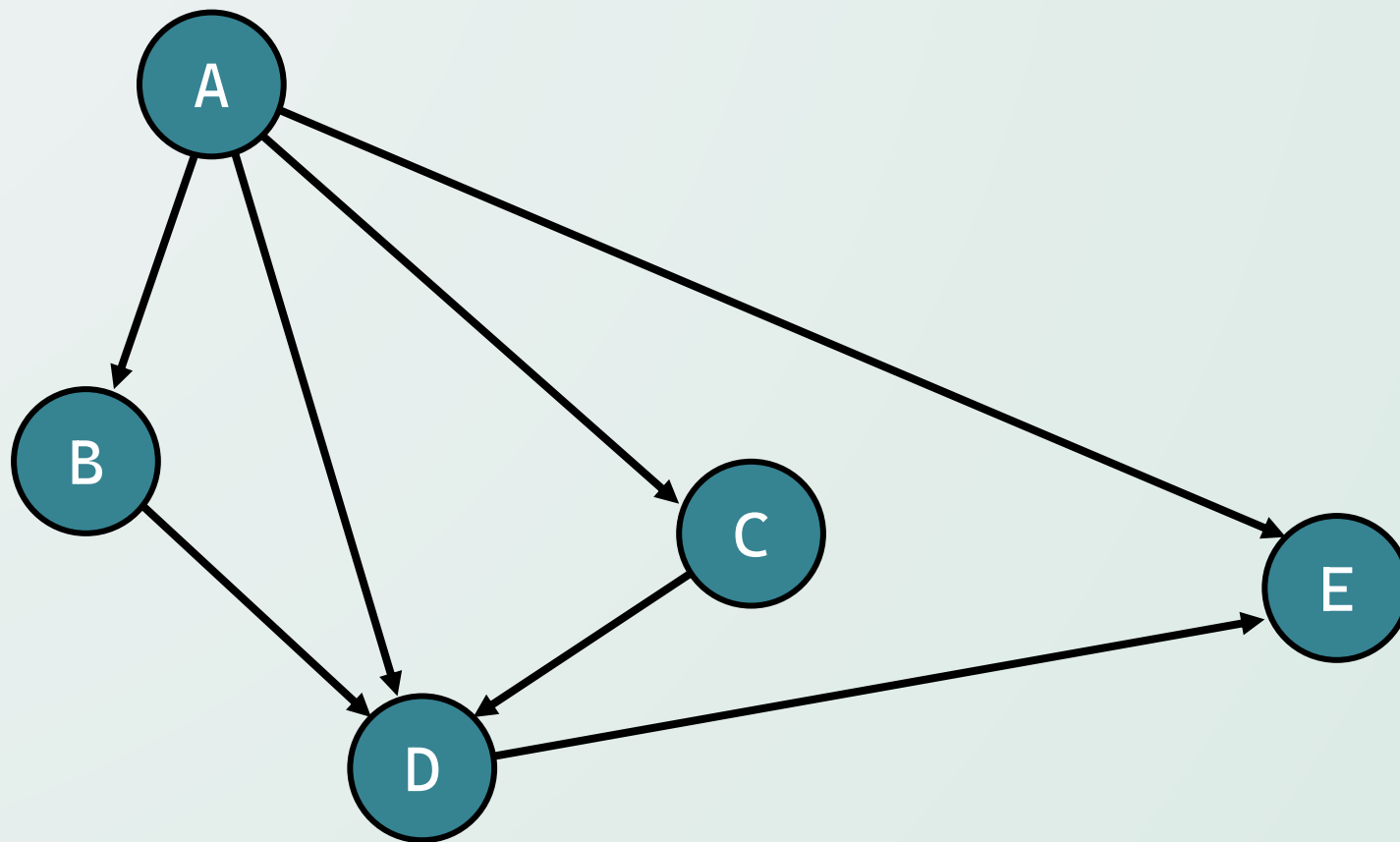
שיפור לזמן לינארי

- בתור עיבוד מקדים, נוכל לשמור רשימה של כל הצמתים עם דרגת כניסה אפס. זה יקח זמן לינארי, אם נשתמש ברשימת שכנויות. בנוסף, לכל צומת נשמור את דרגת הכניסה, גם כן בזמן לינארי.
- מתוך הרשימה נבחר שרירותית קודקוד, נמספר אותו באחד. כשנסתיר אותו, נצטרך להוריד אחד מדרגת הכניסה של כל הקודקודים שיש מהקודקוד קשת אליהם. אם לאחד מהצמתים הללו דרגת הכניסה הפכה ל0, נוסיף אותו לרשימה של הצמתים עם דרגת כניסה אפס.
- נחזור ונבחר קודקוד עם דרגת כניסה אפס, נמספר אותו ב2 ונחזור על התהליך.
- **שאלה** – כל פעם שאנחנו מורידים אחד מדרגת הכניסה של כל הקודקודים עם קשת אליהם אנחנו יכולים לעשות $O(V)$ עבודה. איך בסופו של דבר נקבל זמן לינארי?

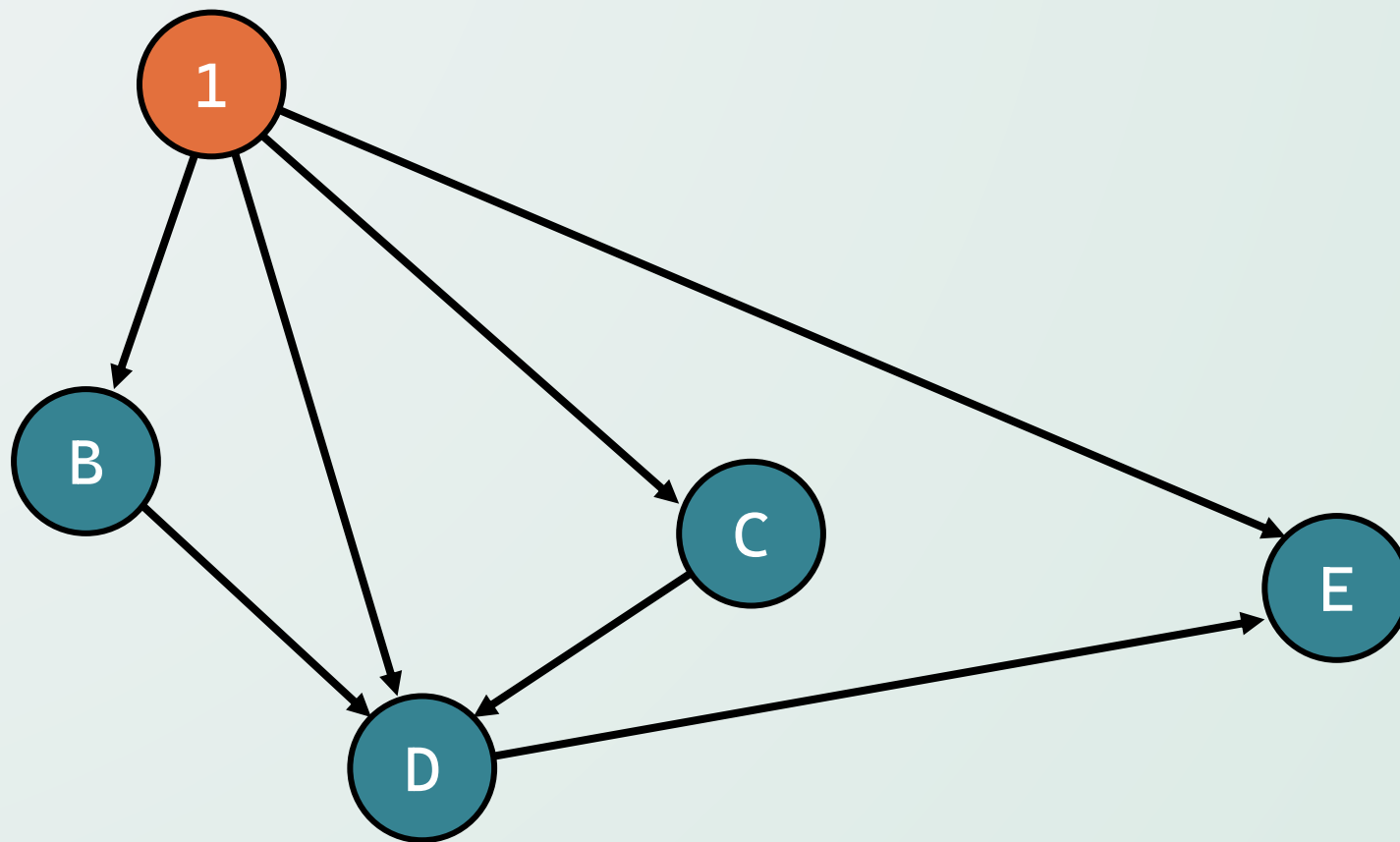
שיפור לזמן לינארי

- בתור עיבוד מקדים, נוכל לשמור רשימה של כל הצמתים עם דרגת כניסה אפס. זה יקח זמן לינארי, אם נשתמש ברשימת שכנויות. בנוסף, לכל צומת נשמור את דרגת הכניסה, גם כן בזמן לינארי.
- מתוך הרשימה נבחר שרירותית קודקוד, נמספר אותו באחד. כשנסתיר אותו, נצטרך להוריד אחד מדרגת הכניסה של כל הקודקודים שיש מהקודקוד קשת אליהם. אם לאחד מהצמתים הללו דרגת הכניסה הפכה ל0, נוסיף אותו לרשימה של הצמתים עם דרגת כניסה אפס.
- נחזור ונבחר קודקוד עם דרגת כניסה אפס, נמספר אותו ב2 ונחזור על התהליך.
- **שאלה** – כל פעם שאנחנו מורידים אחד מדרגת הכניסה של כל הקודקודים עם קשת אליהם אנחנו יכולים לעשות $O(V)$ עבודה. איך בסופו של דבר נקבל זמן לינארי?
- **תשובה** – דבר כזה מתבצע פעם אחת לכל קשת. לכן, לא נעשה זאת יותר מ $|E|$ פעמים. לכן נקבל סיבוכיות $O(V + E)$.

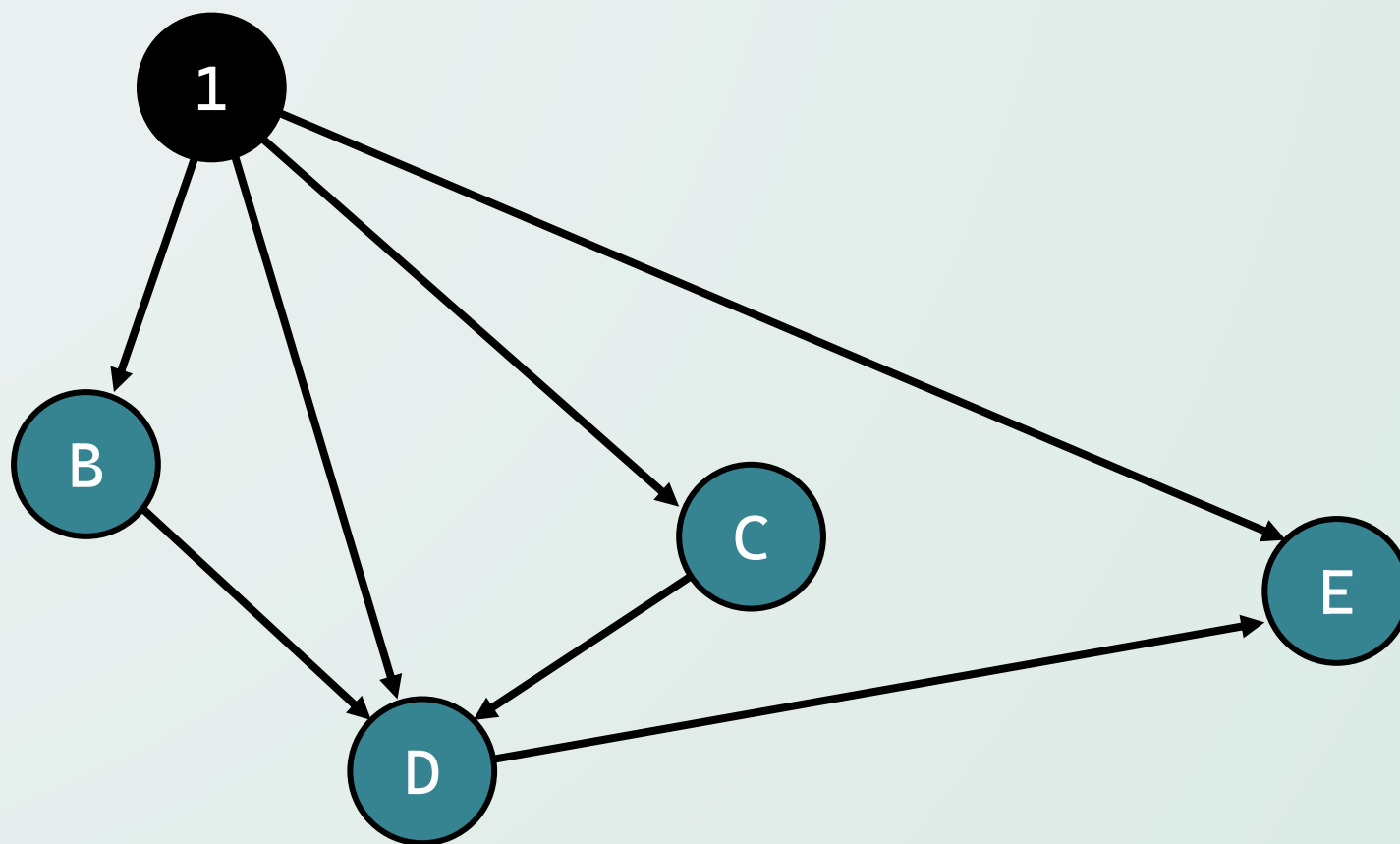
דוגמת הרצה



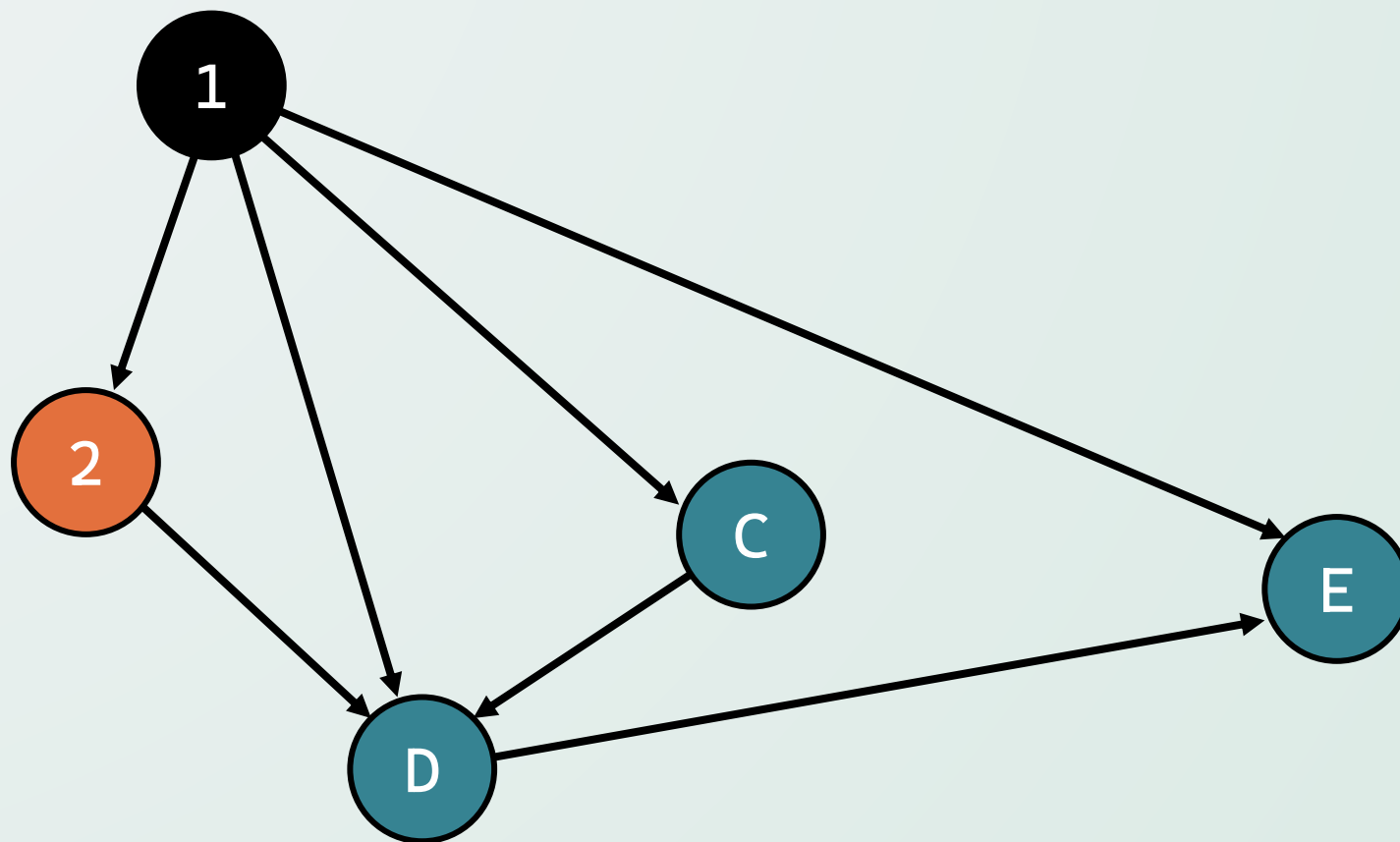
דוגמת הרצה



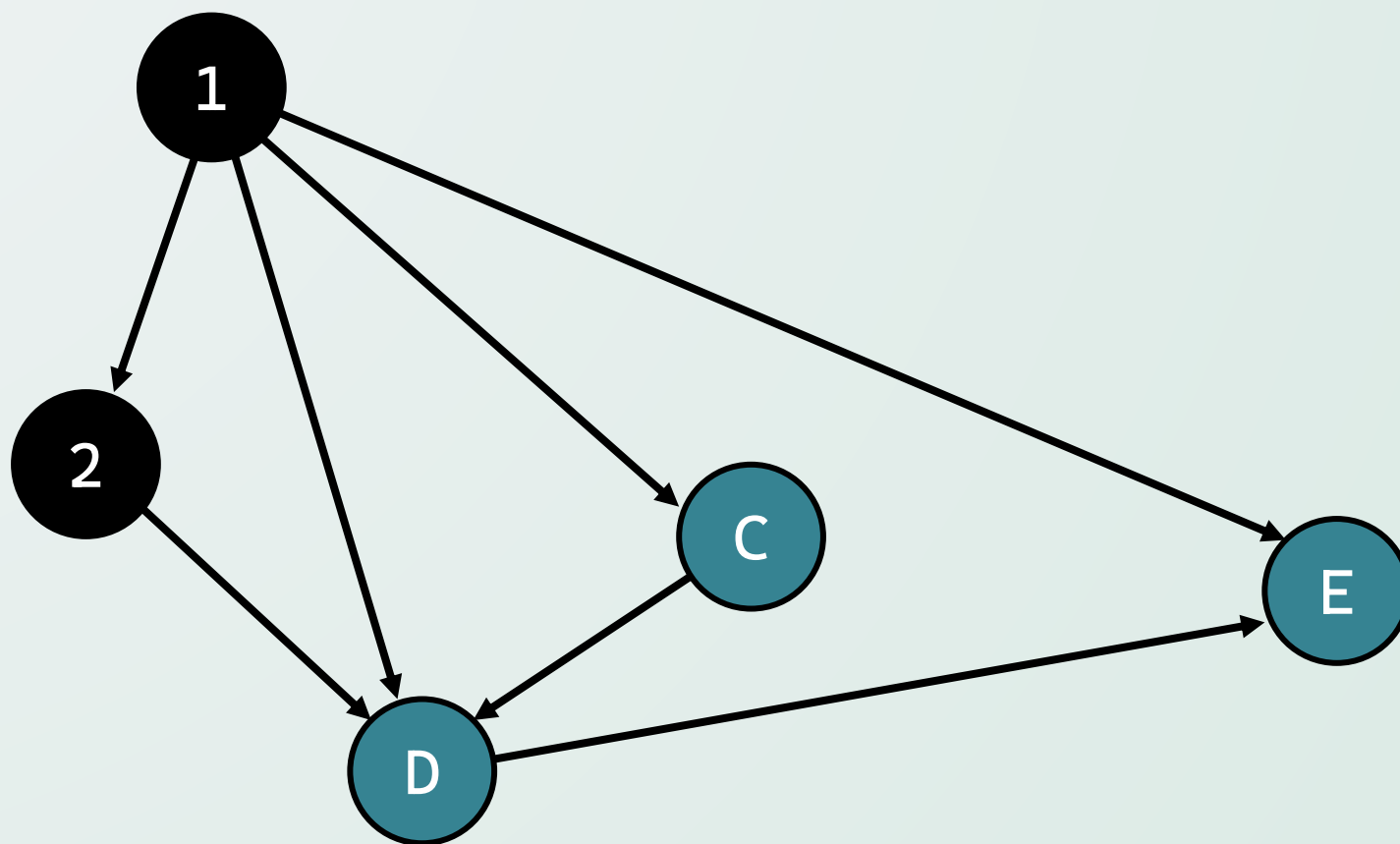
דוגמת הרצה



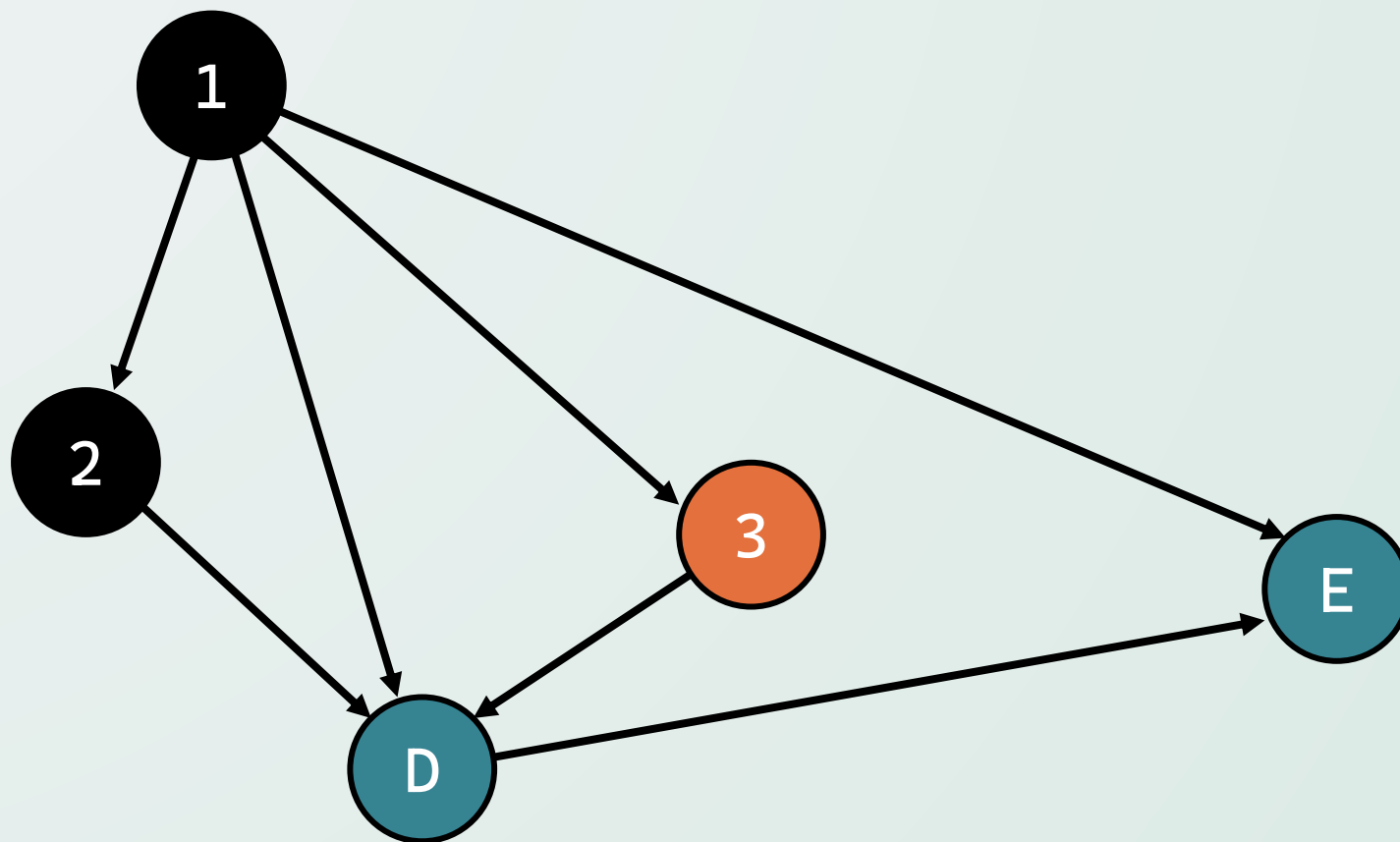
דוגמת הרצה



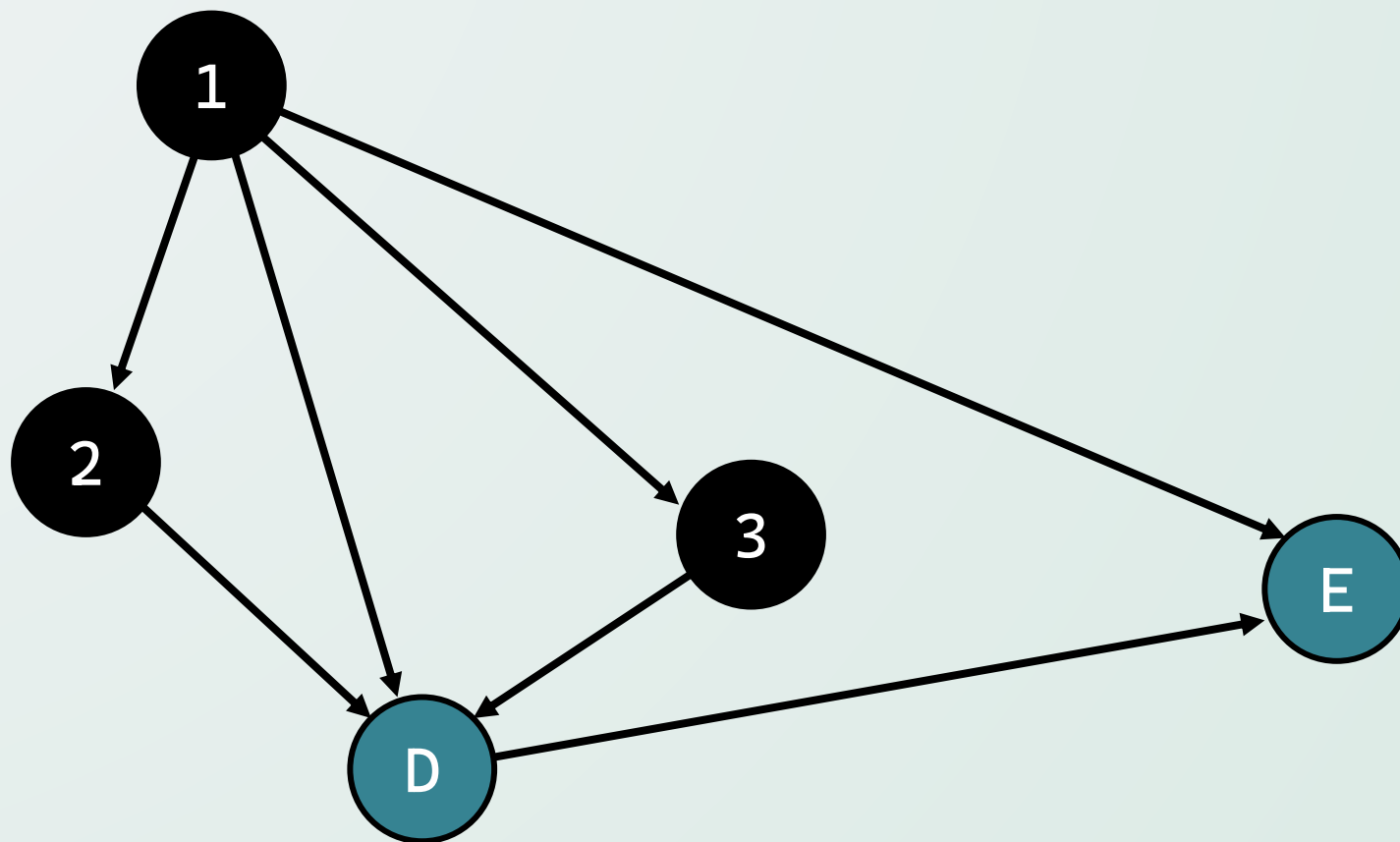
דוגמת הרצה



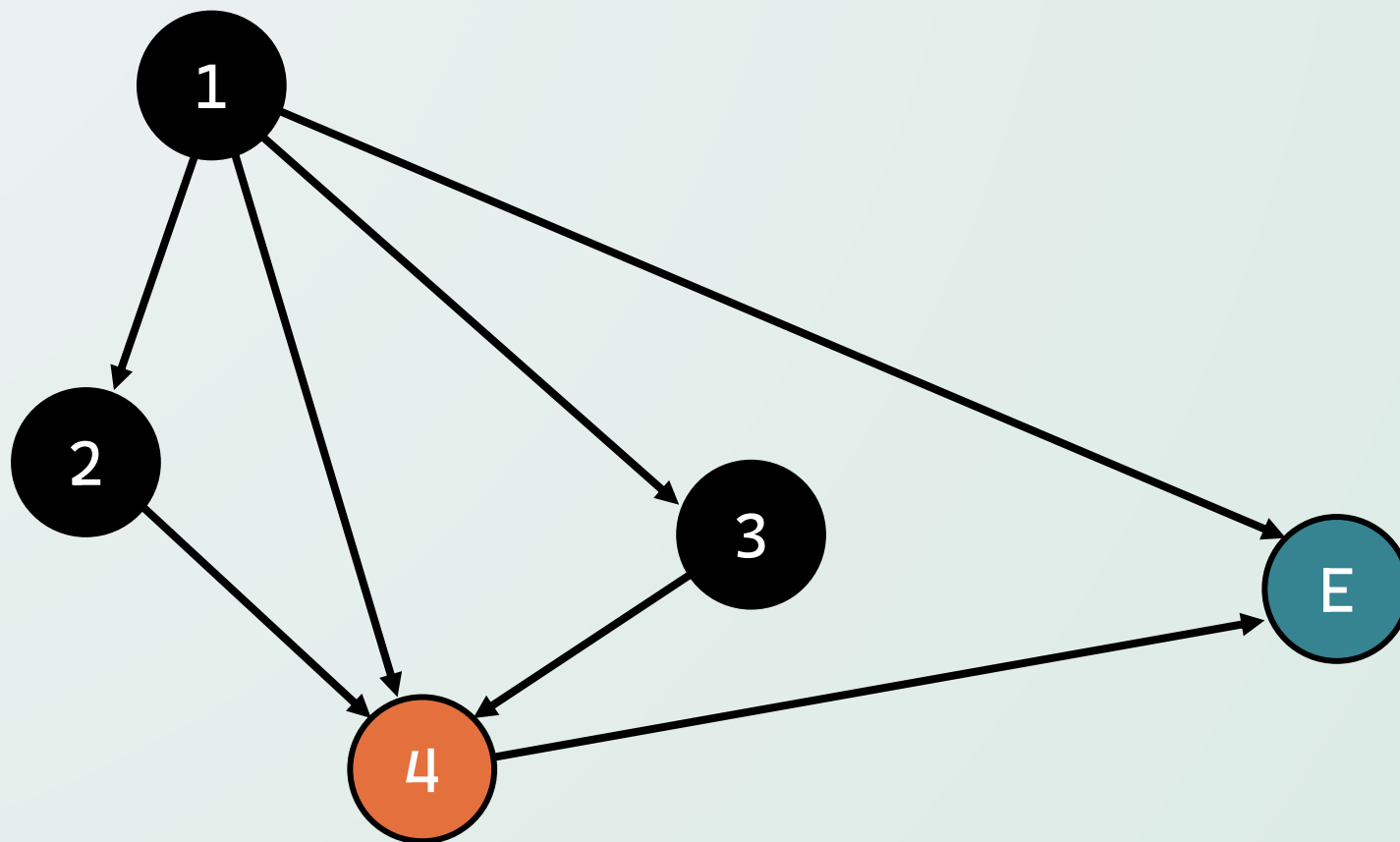
דוגמת הרצה



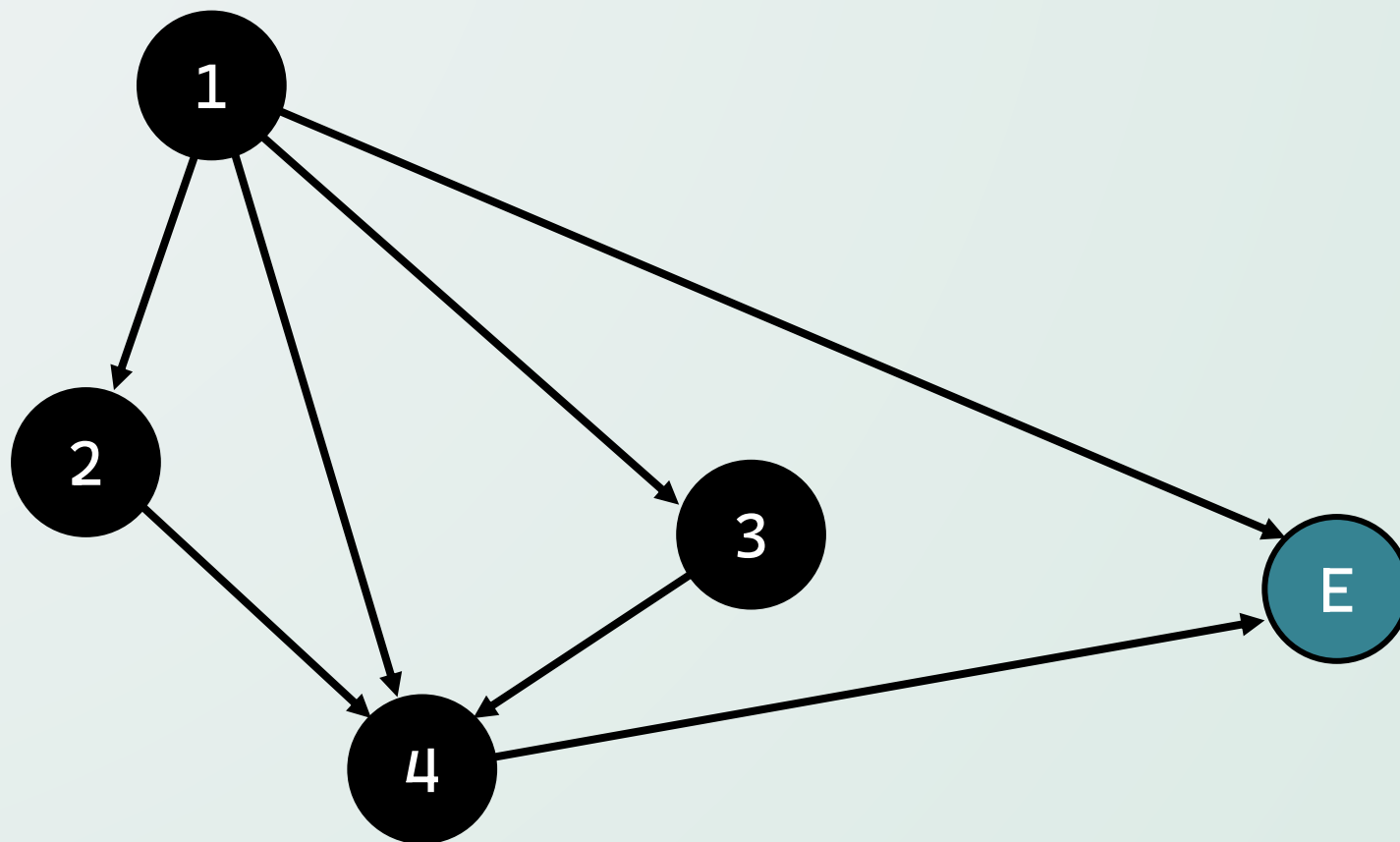
דוגמת הרצה



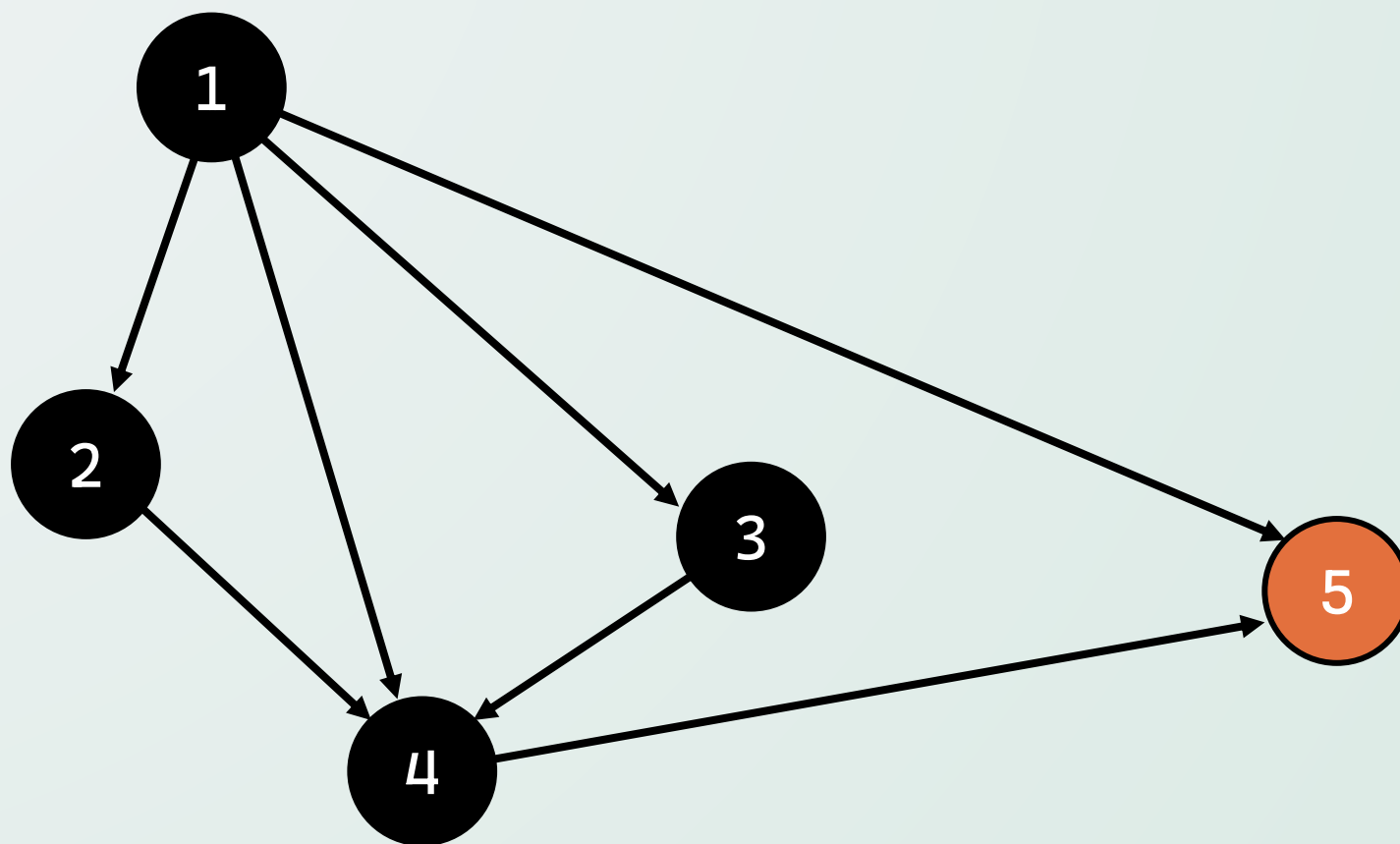
דוגמת הרצה



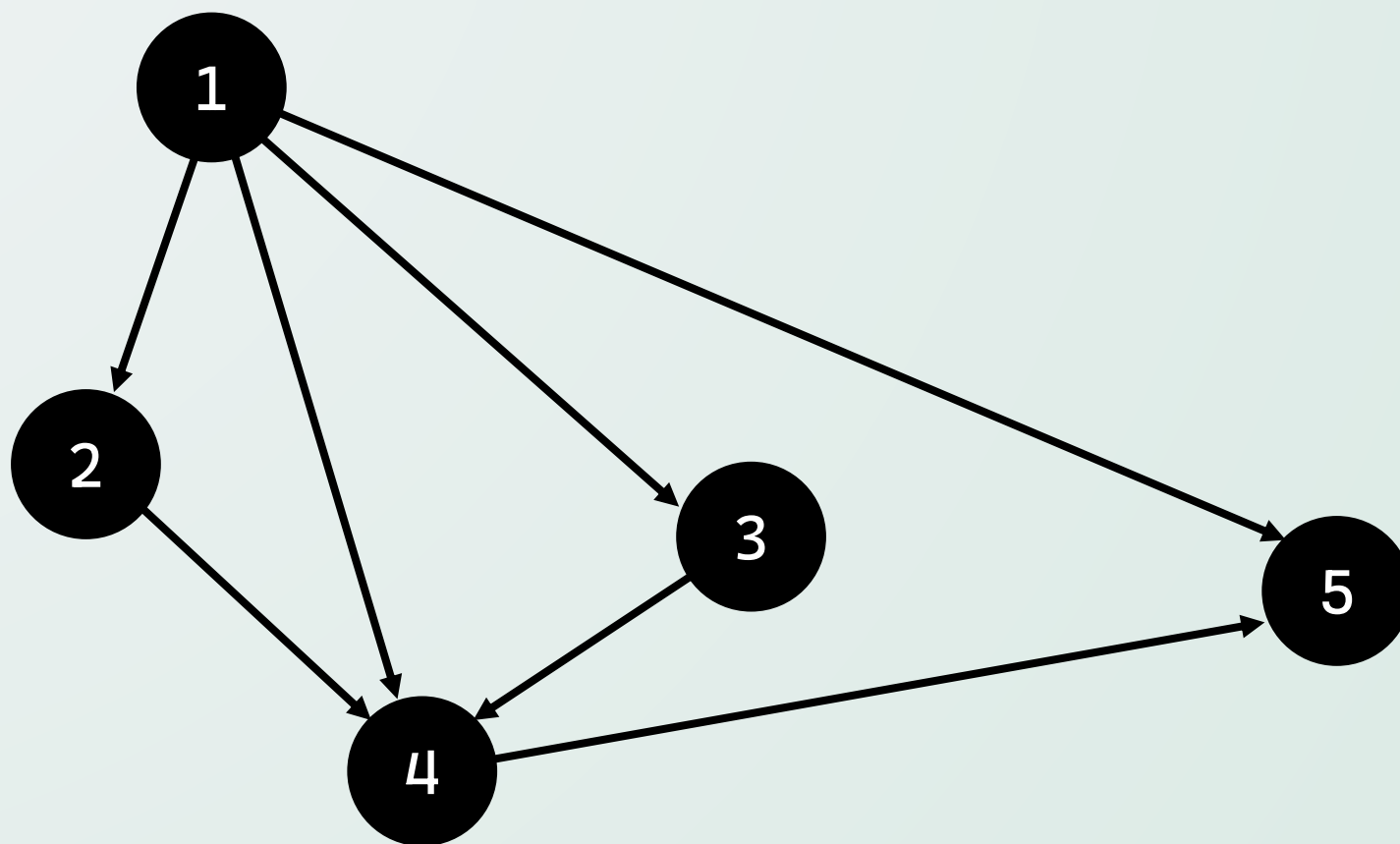
דוגמת הרצה



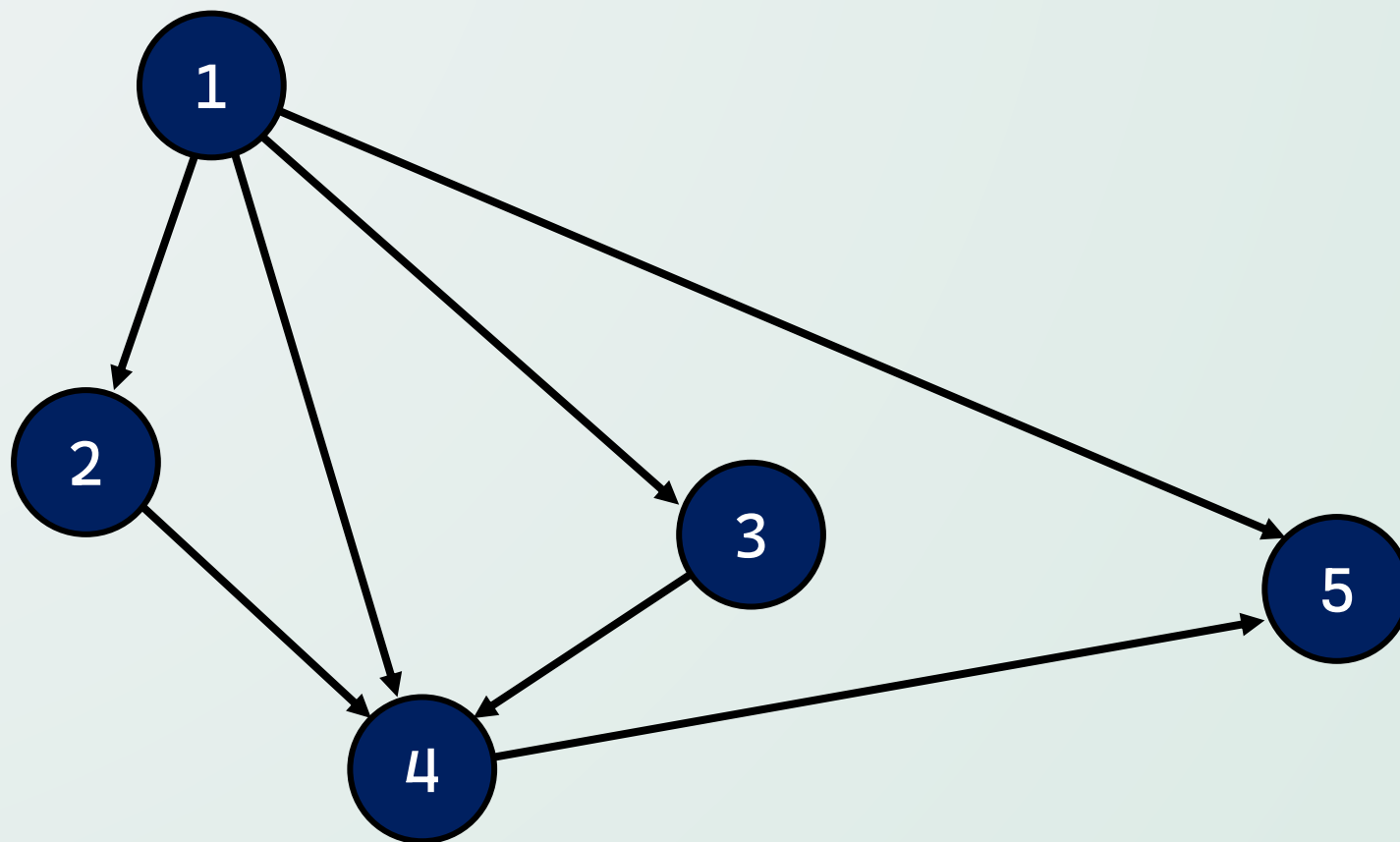
דוגמת הרצה



דוגמת הרצה



דוגמת הרצה

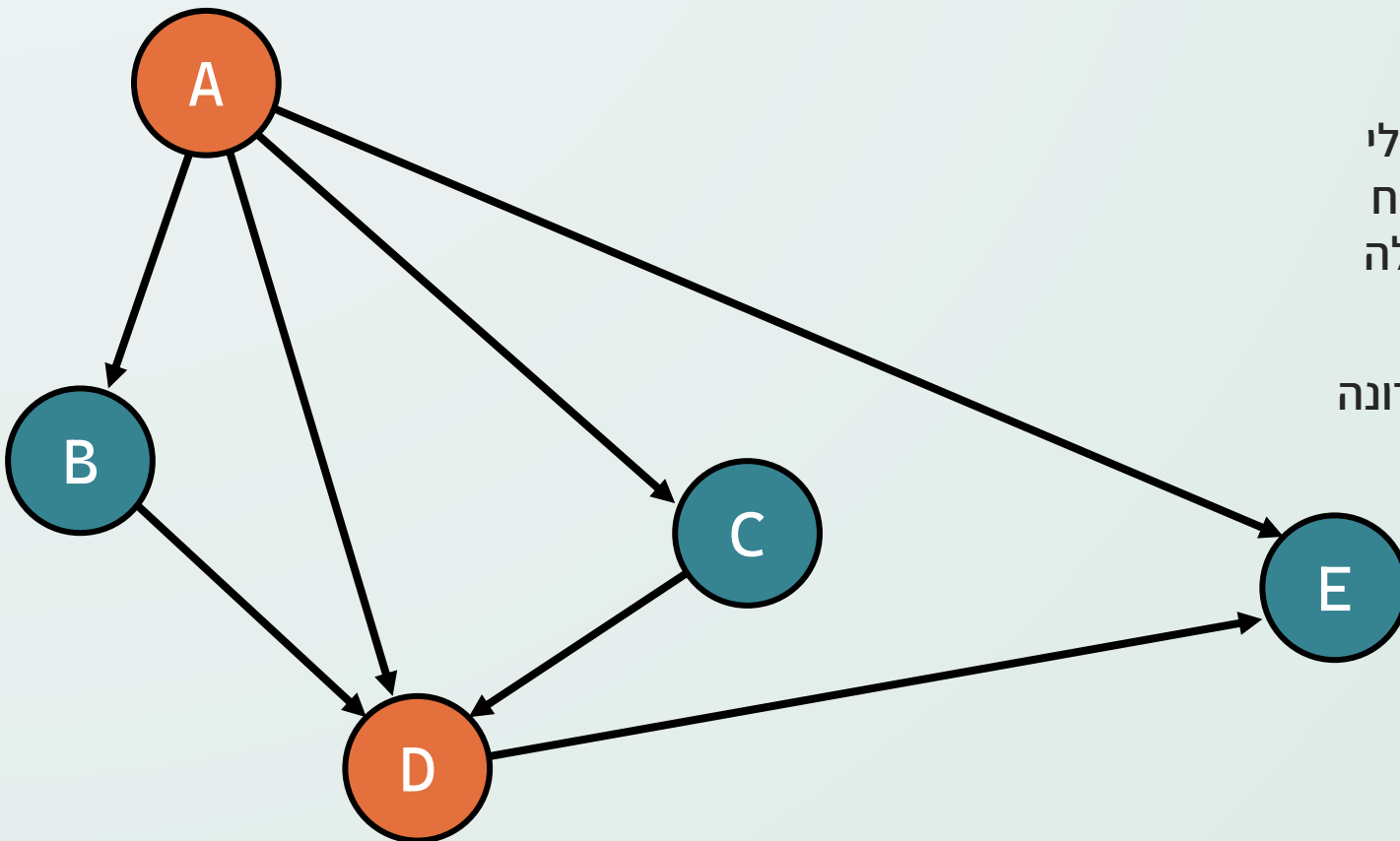


מספר מסלולים ב DAG

Number of Paths in a DAG

מספר מסלולים ב DAG

- שרון המתרגל רוצה לשלוח לחברה שלו שרונה פרחים. לצערנו, הם במערכת יחסים ארוכת טווח (Long distance) והוא לא יכול להביא לה את הפרחים בעצמו 😞
- נתונה לו רשת של נמלים (ports) ומסלולי ספינות חד כיווניים שמחברים ביניהם. נניח שאין מסלול מעגלי, כלומר ספינה לא יכולה לצאת ולחזור מאותו נמל.
- שרון רומנטיקן לא קטן, ורוצה לשלוח לשרונה זר פרחים בכל מסלול אפשרי.
- עזרו לשרון לחשב את מספר הזרים שיצטרך לקנות.



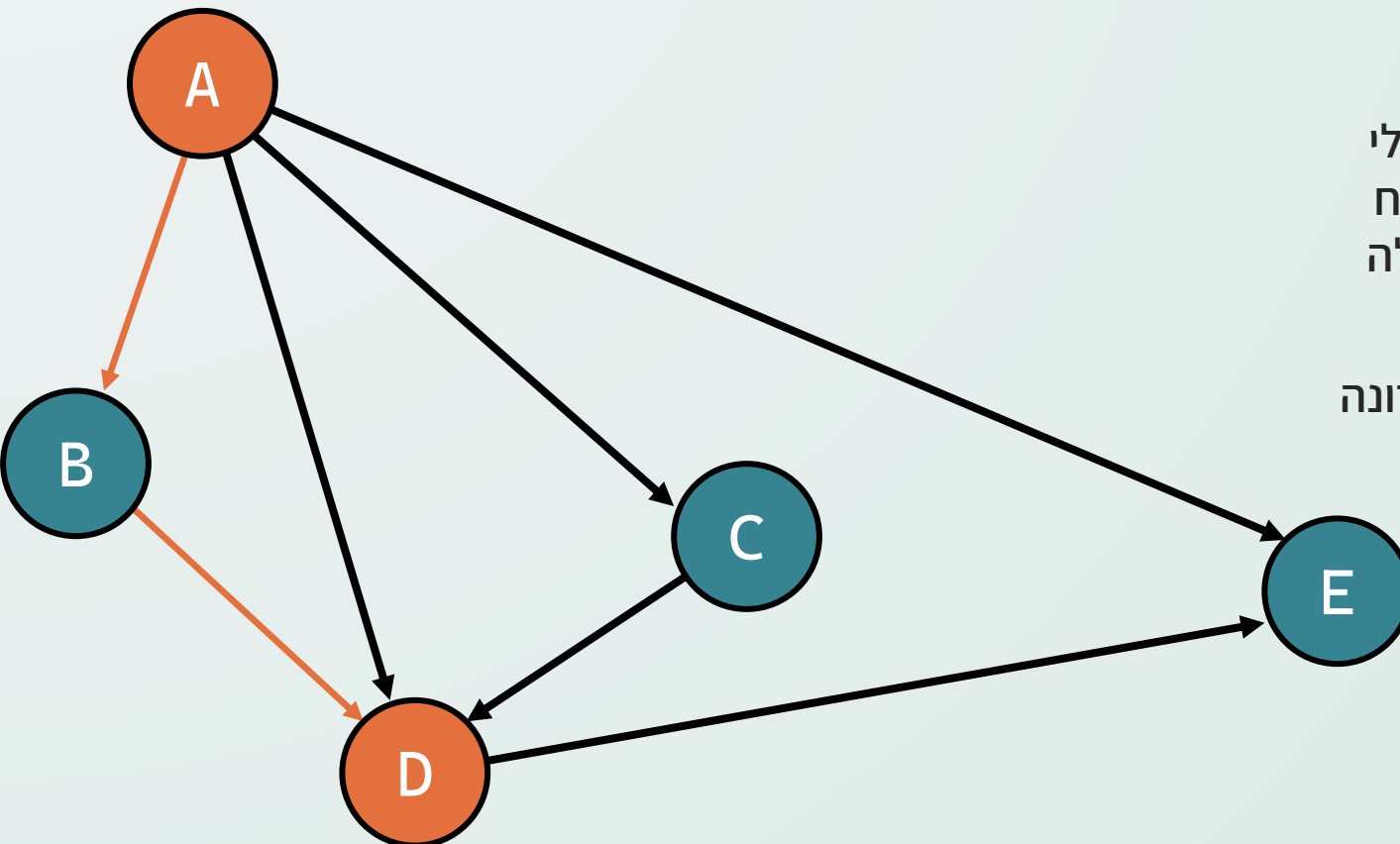
מספר מסלולים ב DAG

- שרון המתרגל רוצה לשלוח לחברה שלו שרונה פרחים. לצערנו, הם במערכת יחסים ארוכת טווח (Long distance) והוא לא יכול להביא לה את הפרחים בעצמו 😞

- נתונה לו רשת של נמלים (ports) ומסלולי ספינות חד כיווניים שמחברים ביניהם. נניח שאין מסלול מעגלי, כלומר ספינה לא יכולה לצאת ולחזור מאותו נמל.

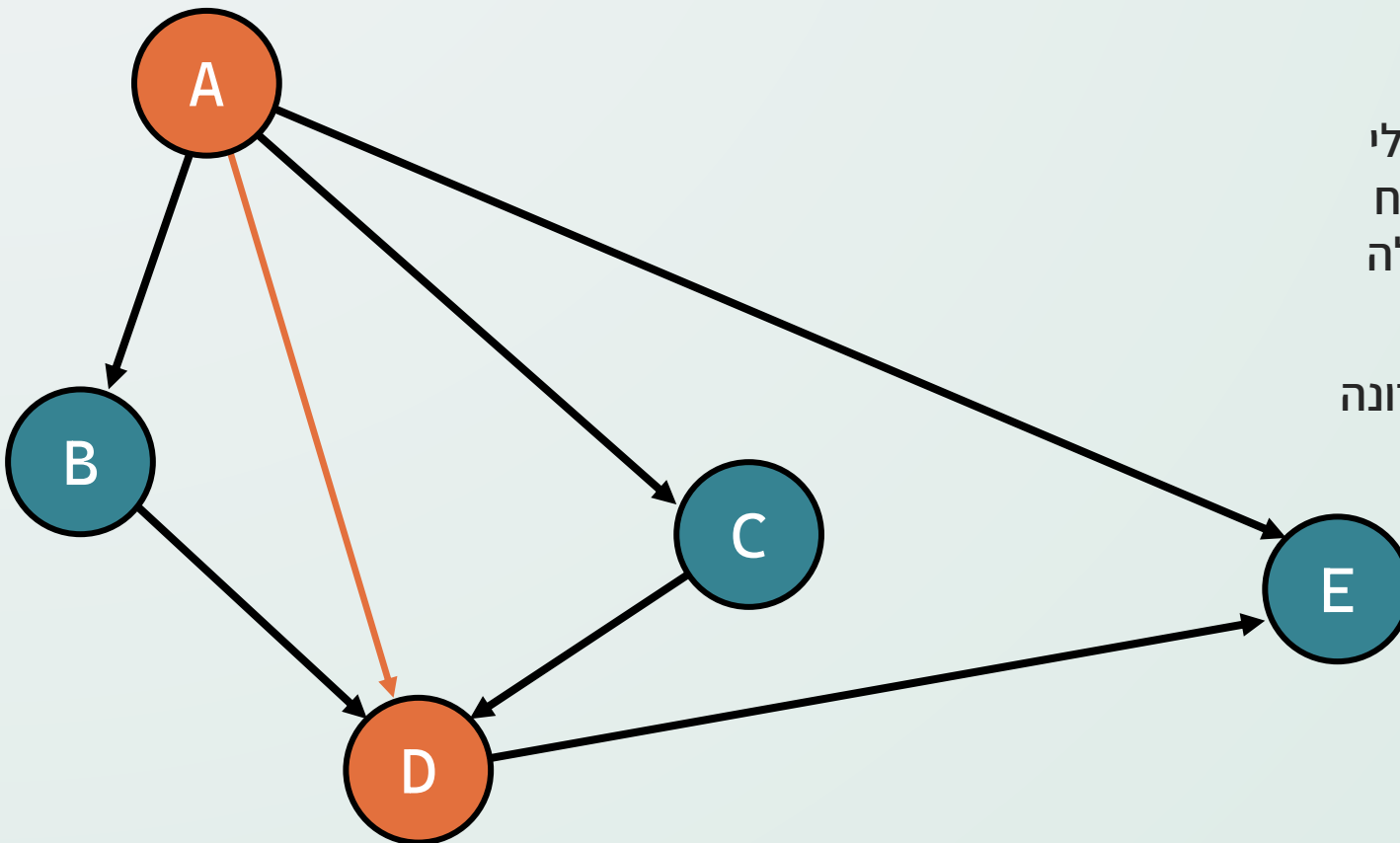
- שרון רומנטיקן לא קטן, ורוצה לשלוח לשרונה זר פרחים בכל מסלול אפשרי.

- עזרו לשרון לחשב את מספר הזרים שיצטרך לקנות.



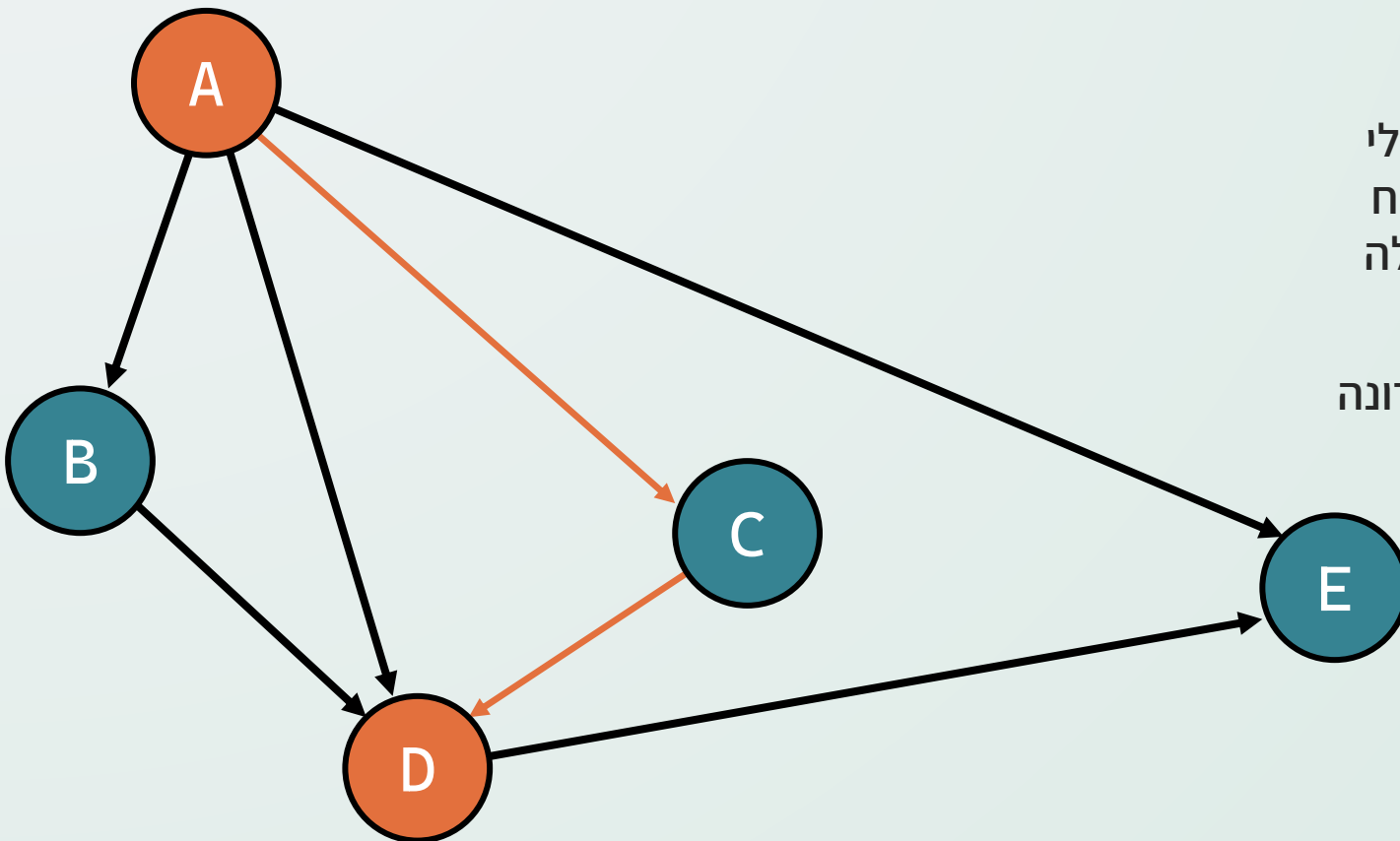
מספר מסלולים ב DAG

- שרון המתרגל רוצה לשלוח לחברה שלו שרונה פרחים. לצערנו, הם במערכת יחסים ארוכת טווח (Long distance) והוא לא יכול להביא לה את הפרחים בעצמו 😞
- נתונה לו רשת של נמלים (ports) ומסלולי ספינות חד כיווניים שמחברים ביניהם. נניח שאין מסלול מעגלי, כלומר ספינה לא יכולה לצאת ולחזור מאותו נמל.
- שרון רומנטיקן לא קטן, ורוצה לשלוח לשרונה זר פרחים בכל מסלול אפשרי.
- עזרו לשרון לחשב את מספר הזרים שיצטרך לקנות.



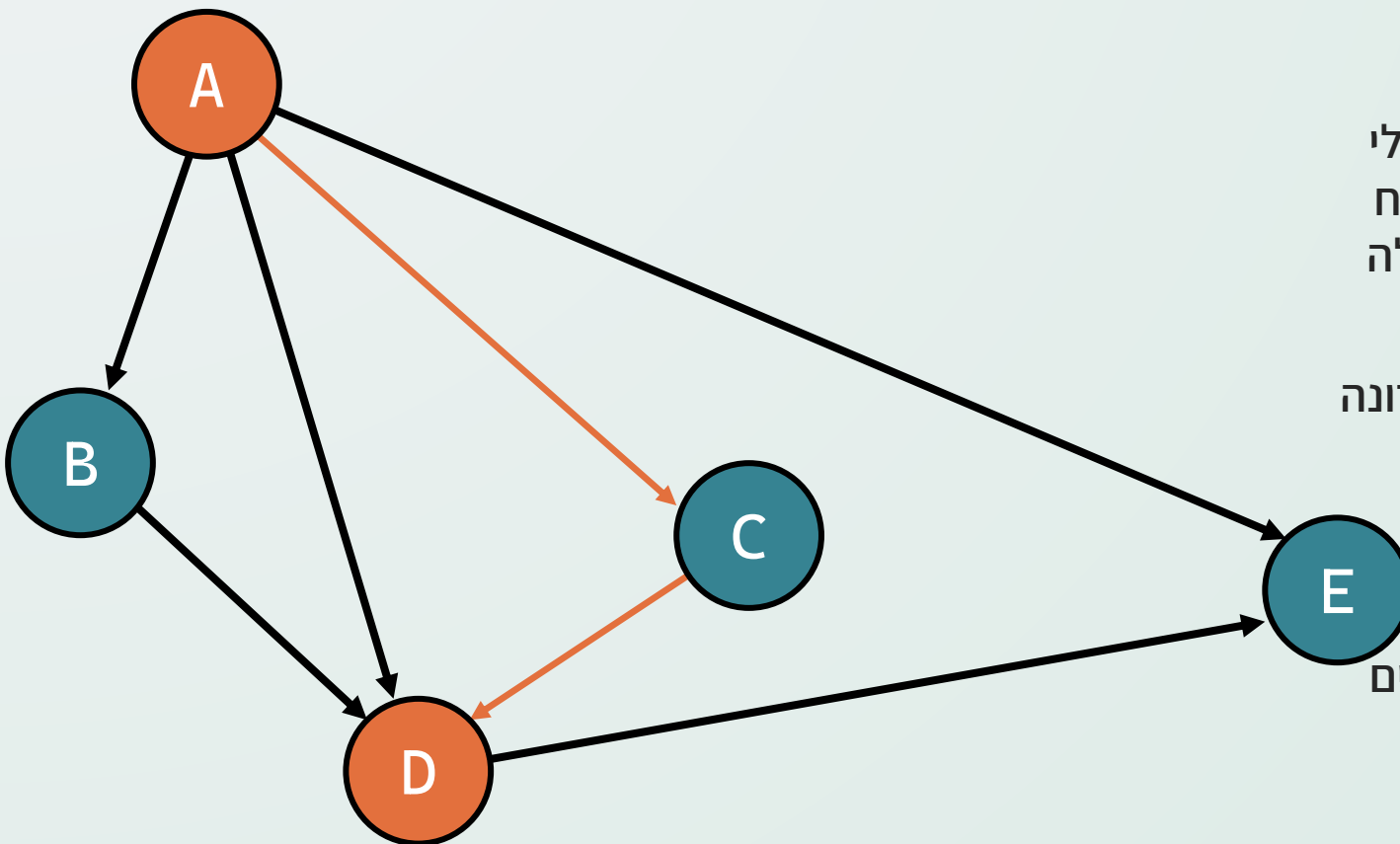
מספר מסלולים ב DAG

- שרון המתרגל רוצה לשלוח לחברה שלו שרונה פרחים. לצערנו, הם במערכת יחסים ארוכת טווח (Long distance) והוא לא יכול להביא לה את הפרחים בעצמו 😞
- נתונה לו רשת של נמלים (ports) ומסלולי ספינות חד כיווניים שמחברים ביניהם. נניח שאין מסלול מעגלי, כלומר ספינה לא יכולה לצאת ולחזור מאותו נמל.
- שרון רומנטיקן לא קטן, ורוצה לשלוח לשרונה זר פרחים בכל מסלול אפשרי.
- עזרו לשרון לחשב את מספר הזרים שיצטרך לקנות.



מספר מסלולים ב DAG

- שרון המתרגל רוצה לשלוח לחברה שלו שרונה פרחים. לצערנו, הם במערכת יחסים ארוכת טווח (Long distance) והוא לא יכול להביא לה את הפרחים בעצמו 😞
- נתונה לו רשת של נמלים (ports) ומסלולי ספינות חד כיווניים שמחברים ביניהם. נניח שאין מסלול מעגלי, כלומר ספינה לא יכולה לצאת ולחזור מאותו נמל.
- שרון רומנטיקן לא קטן, ורוצה לשלוח לשרונה זר פרחים בכל מסלול אפשרי.
- עזרו לשרון לחשב את מספר הזרים שיצטרך לקנות.
- בתכלס, בהינתן DAG ושני צמתים מסומנים u, v מצאו את מספר המסלולים מ u ל v .



מספר מסלולים ב DAG

- ראשית, נמיין את ה DAG מיון טופולוגי. נתייחס ל u, v בתור המספור שלהם במיון. אבחנו – אם $v < u$, מספר המסלולים הוא אפס. זאת מכיוון שמסלול ב DAG הוא מונוטוני עולה ממש, ולכן אי אפשר להגיע מ u ל v .

מספר מסלולים ב DAG

- ראשית, נמיין את ה DAG מיון טופולוגי. נתייחס ל u, v בתור המספור שלהם במיון. אבחנו – אם $v < u$, מספר המסלולים הוא אפס. זאת מכיוון שמסלול ב DAG הוא מונוטוני עולה ממש, ולכן אי אפשר להגיע מ u ל v .
- אחרת, נפתור באמצעות DP. לכל צומת w , נשמור את מספר הדרכים להגיע לצומת u , בתור $ways(w)$. כמובן $ways(u)=1$ – המסלול הטריטוריאלי. נאתחל את השאר לאפס.

מספר מסלולים ב DAG

- ראשית, נמיין את ה DAG מיון טופולוגי. נתייחס ל u, v בתור המספור שלהם במיון. אבחנו – אם $v < u$, מספר המסלולים הוא אפס. זאת מכיוון שמסלול ב DAG הוא מונוטוני עולה ממש, ולכן אי אפשר להגיע מ u ל v .
- אחרת, נפתור באמצעות DP. לכל צומת w , נשמור את מספר הדרכים להגיע לצומת u , בתור $ways(w)$. כמובן $ways(u)=1$ – המסלול הטרוויאלי. נאתחל את השאר לאפס.
- כעת נעבור על הגרף לפי המיון הטופולוגי, אבל נתחיל מ u . כלומר הסדר יהיה $u, u+1 \dots v$.
- לכל צומת w שנעבור, נשנה את $ways(w)$ כך:

$$ways(w) = \sum_{(r,w) \in E} ways(r)$$

מספר מסלולים ב DAG

- ראשית, נמיין את ה DAG מיון טופולוגי. נתייחס ל u, v בתור המספור שלהם במיון. אבחנו – אם $v < u$, מספר המסלולים הוא אפס. זאת מכיוון שמסלול ב DAG הוא מונוטוני עולה ממש, ולכן אי אפשר להגיע מ u ל v .
- אחרת, נפתור באמצעות DP. לכל צומת w , נשמור את מספר הדרכים להגיע לצומת w , בתור $ways(w)$. כמובן $ways(u)=1$ – המסלול הטרוויאלי. נאתחל את השאר לאפס.
- כעת נעבור על הגרף לפי המיון הטופולוגי, אבל נתחיל מ u . כלומר הסדר יהיה $u, u+1 \dots v$.
- לכל צומת w שנעבור, נשנה את $ways(w)$ כך:
$$ways(w) = \sum_{(r,w) \in E} ways(r)$$
- כשנגיע לצומת v , נעצור ונחזיר $ways(v)$.

מספר מסלולים ב DAG

- ראשית, נמיין את ה DAG מיון טופולוגי. נתייחס ל u, v בתור המספור שלהם במיון. אבחנו – אם $v < u$, מספר המסלולים הוא אפס. זאת מכיוון שמסלול ב DAG הוא מונוטוני עולה ממש, ולכן אי אפשר להגיע מ u ל v .
- אחרת, נפתור באמצעות DP. לכל צומת w , נשמור את מספר הדרכים להגיע לצומת u , בתור $ways(w)$. כמובן $ways(u) = 1$ – המסלול הטרוויאלי. נאתחל את השאר לאפס.
- כעת נעבור על הגרף לפי המיון הטופולוגי, אבל נתחיל מ u . כלומר הסדר יהיה $u, u + 1 \dots v$.
- לכל צומת w שנעבור, נשנה את $ways(w)$ כך:
$$ways(w) = \sum_{(r,w) \in E} ways(r)$$
- כשנגיע לצומת v , נעצור ונחזיר $ways(v)$.
- סיבוכיות** – מיון טופולוגי מתבצע בזמן לינארי, ולאחר מכן לכל קשת (לכל היותר) עושים מספר קבוע של פעולות. לכן סה"כ לינארי.

הוכחת נכונות

- נבצע אינדוקציה על האינדקס במיון הטופולוגי. ברור שכל צומת עם אינדקס נמוך מ u אי אפשר להגיע אליו, לכן אפילו לא נסתכל עליו.
- **מקרה בסיס** – הגדרנו את $\text{ways}(u)=1$, כיוון שיש מסלול אחד – לא לזוז. אין מעגלים, ולכן זה המסלול היחיד.

הוכחת נכונות

- נבצע אינדוקציה על האינדקס במיון הטופולוגי. ברור שכל צומת עם אינדקס נמוך מ u אי אפשר להגיע אליו, לכן אפילו לא נסתכל עליו.
- **מקרה בסיס** – הגדרנו את $ways(u)=1$, כיוון שיש מסלול אחד – לא לזוז. אין מעגלים, ולכן זה המסלול היחיד.
- **צעד האינדוקציה** – נניח לכל $j < i$ אכן תקין, ונוכיח ש $ways(i)$ תקין.

הוכחת נכונות

- נבצע אינדוקציה על האינדקס במיון הטופולוגי. ברור שכל צומת עם אינדקס נמוך מ- u אי אפשר להגיע אליו, לכן אפילו לא נסתכל עליו.
- **מקרה בסיס** – הגדרנו את $ways(u)=1$, כיוון שיש מסלול אחד – לא לזוז. אין מעגלים, ולכן זה המסלול היחיד.
- **צעד האינדוקציה** – נניח לכל $j < i$ $ways(j)$ אכן תקין, ונוכיח ש $ways(i)$ תקין.
- מסלולים הם מונוטוניים עולים ממש, לכן הדרך היחידה להגיע ל- i היא מצמתים הקטנים מ- i . לפי הגדרת המיון הטופולוגי, בכל פעם שנשתמש ב- $ways(\cdot)$, הערך יהיה מעודכן.

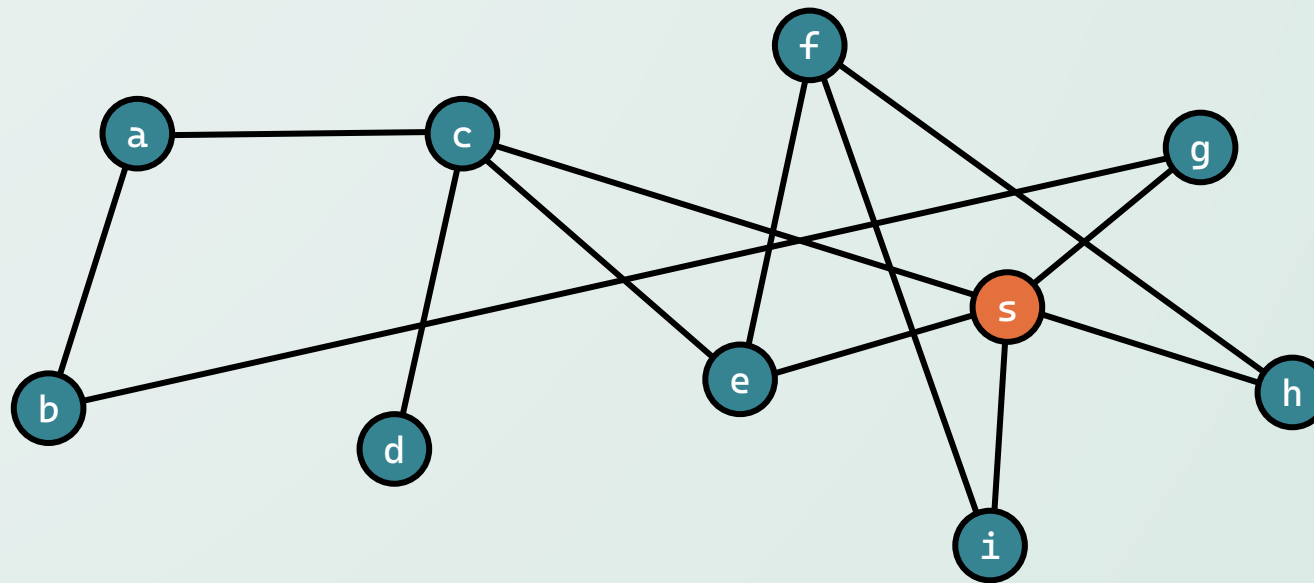
הוכחת נכונות

- נבצע אינדוקציה על האינדקס במיון הטופולוגי. ברור שכל צומת עם אינדקס נמוך מ- u אי אפשר להגיע אליו, לכן אפילו לא נסתכל עליו.
- **מקרה בסיס** – הגדרנו את $ways(u)=1$, כיוון שיש מסלול אחד – לא לזוז. אין מעגלים, ולכן זה המסלול היחיד.
- **צעד האינדוקציה** – נניח לכל $j < i$ $ways(j)$ אכן תקין, ונוכיח ש $ways(i)$ תקין.
- מסלולים הם מונוטוניים עולים ממש, לכן הדרך היחידה להגיע ל- i היא מצמתים הקטנים מ- i . לפי הגדרת המיון הטופולוגי, בכל פעם שנשתמש ב- $ways(\cdot)$, הערך יהיה מעודכן.
- מספר הדרכים להגיע לצומת כלשהי, זה מספר הדרכים להגיע לכל צומת קודמת ואז אליה, ובפרט הנוסחא שכתבנו לחישוב $ways$ נכונה. לפי הנחת האינדוקציה, אנחנו משתמשים בערכים תקינים, ולכן גם $ways(i)$ יהיה תקין.

מסלולים זוגיים קצרים ביותר

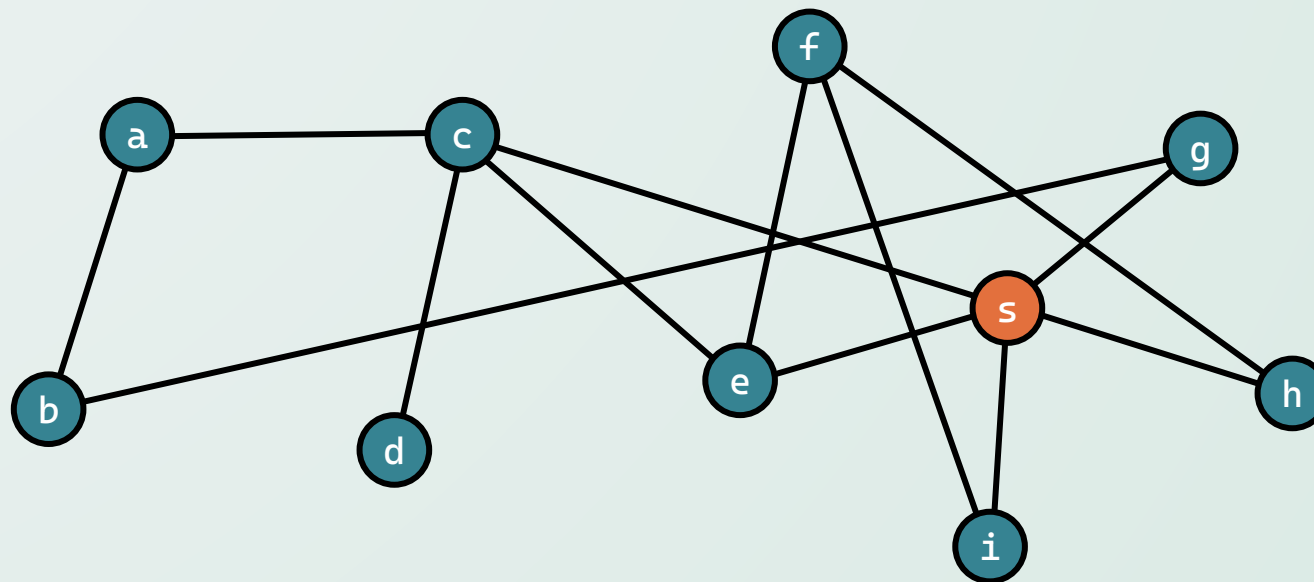
מסלולים זוגיים קצרים ביותר

- מיכל נמצאת ברשת כבישי אגרה המחברים בין ערים. להיכנס לכביש אגרה עולה 50 שקלים, ללא תלות באורך הכביש. למיכל יש רק שטרות של 100 שקלים, ולצערנו לא מחזירים עודף בכבישי אגרה.
- עזרו למיכל למצוא את המחיר המינימלי שתשלם עבור נסיעה לכל עיר, כך שלא יוותר עודף.



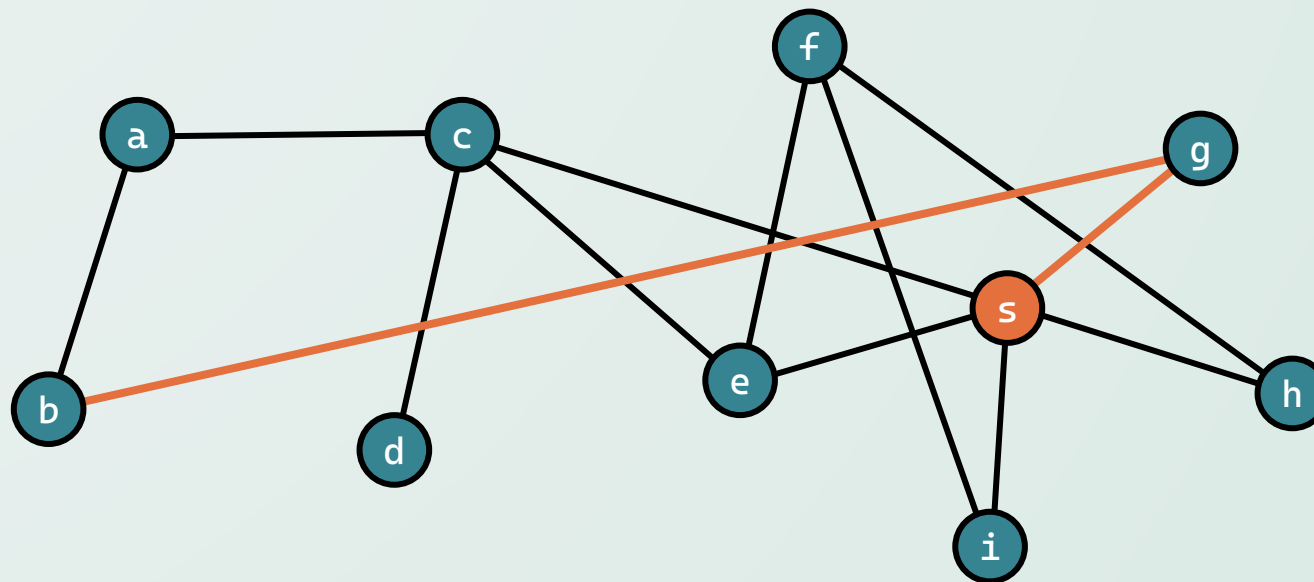
מסלולים זוגיים קצרים ביותר

- נתון גרף $G = (V, E)$ וצומת מקור $s \in V$.
- תארו אלגוריתם, בסיבוכיות טובה ככל שתוכלו, שמוצא לכל צומת $v \in V$ את **המסלול הקצר ביותר שאורכו זוגי מ- s ל- v** . הוכיחו את סיבוכיותו ונכונותו של האלגוריתם שלכם.



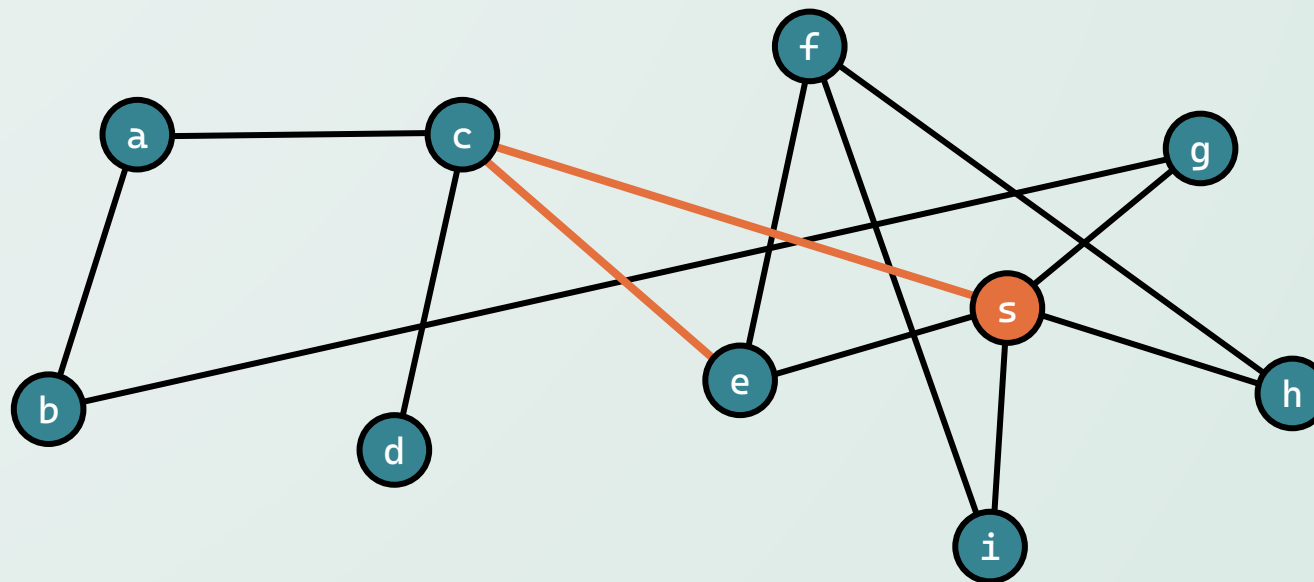
מסלולים זוגיים קצרים ביותר

- נתון גרף $G = (V, E)$ וצומת מקור $s \in V$.
- תארו אלגוריתם, בסיבוכיות טובה ככל שתוכלו, שמוצא לכל צומת $v \in V$ את **המסלול הקצר ביותר שאורכו זוגי מ- s ל- v** . הוכיחו את סיבוכיותו ונכונותו של האלגוריתם שלכם.



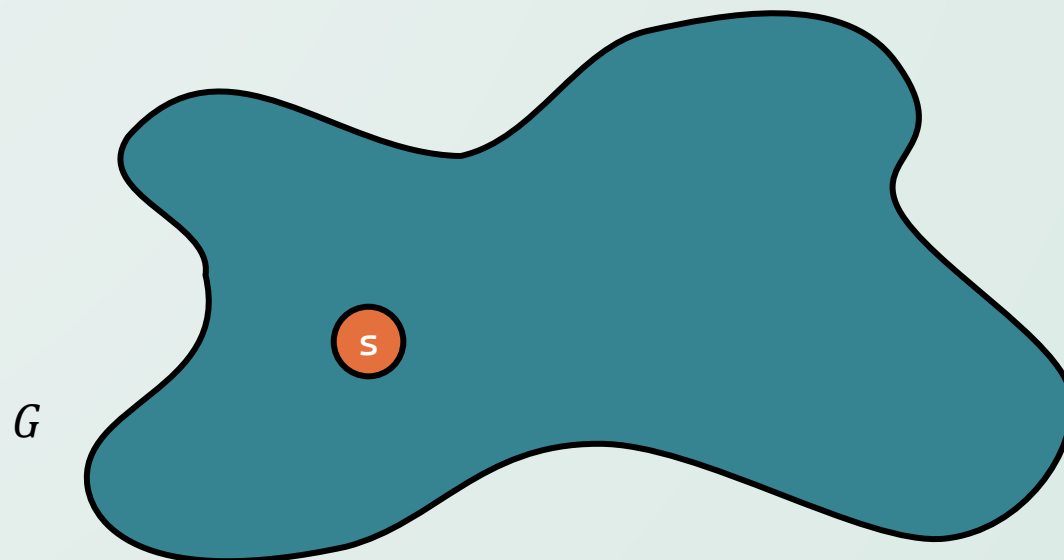
מסלולים זוגיים קצרים ביותר

- נתון גרף $G = (V, E)$ וצומת מקור $s \in V$.
- תארו אלגוריתם, בסיבוכיות טובה ככל שתוכלו, שמוצא לכל צומת $v \in V$ את **המסלול הקצר ביותר שאורכו זוגי מ- s ל- v** . הוכיחו את סיבוכיותו ונכונותו של האלגוריתם שלכם.



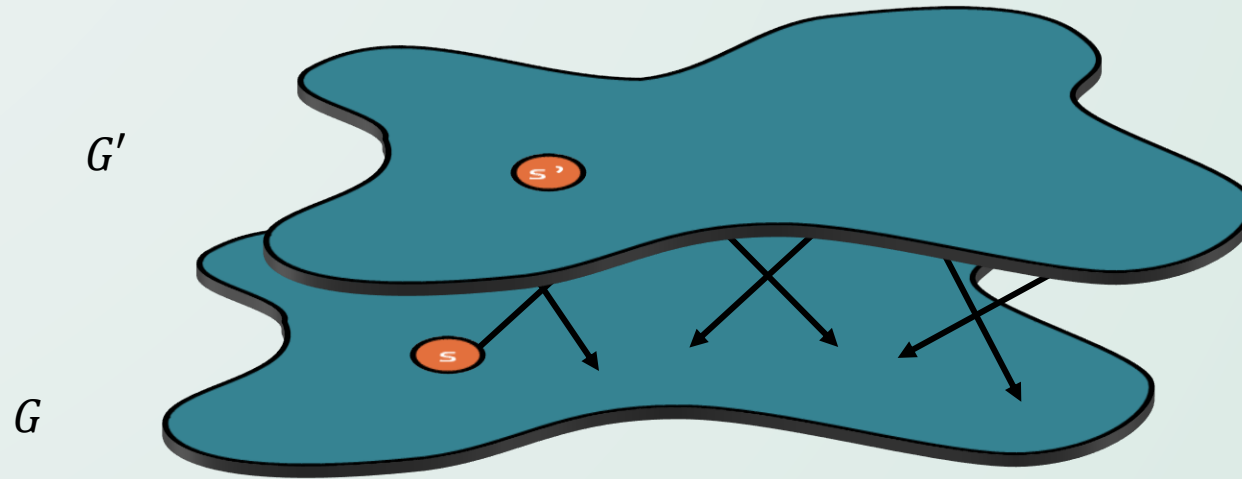
גרף שכבות

- בהינתן גרף G נבנה גרף חדש G' שעליו נריץ את אלגוריתם ה-BFS.
- נשכפל את כל הצמתים בגרף, כך שלכל צומת מקורית v תהיה גם צומת תואמת חדשה v' .
- כל קשת (u, v) תופיע בגרף החדש פעמיים בתור (u, v') ובתור (u', v) בלבד.



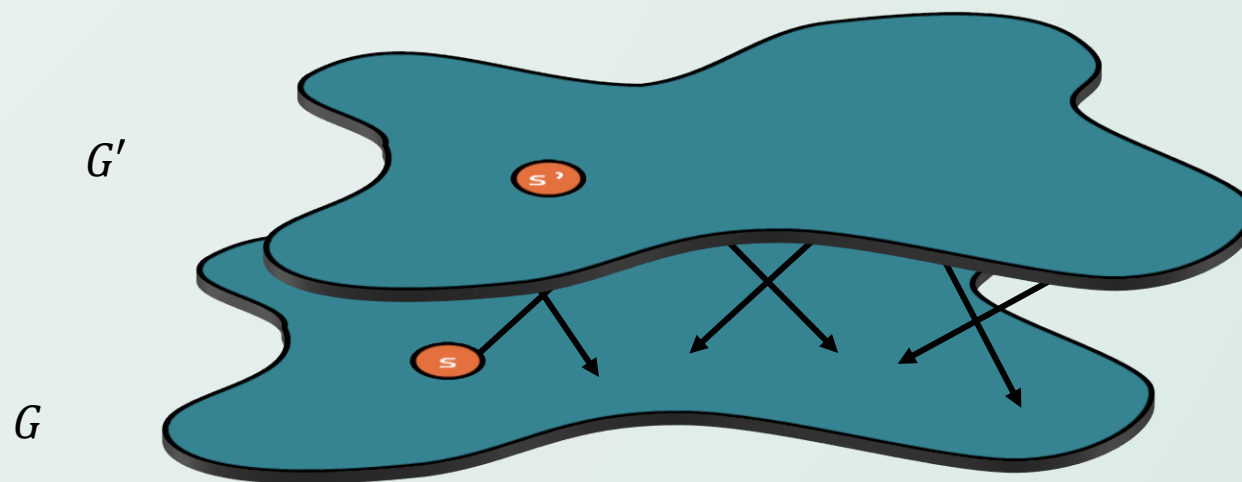
גרף שכבות

- בהינתן גרף G נבנה גרף חדש G' שעליו נריץ את אלגוריתם ה-BFS.
- נשכפל את כל הצמתים בגרף, כך שלכל צומת מקורית v תהיה גם צומת תואמת חדשה v' .
- כל קשת (u, v) תופיע בגרף החדש פעמיים בתור (u, v') ובתור (u', v) בלבד.

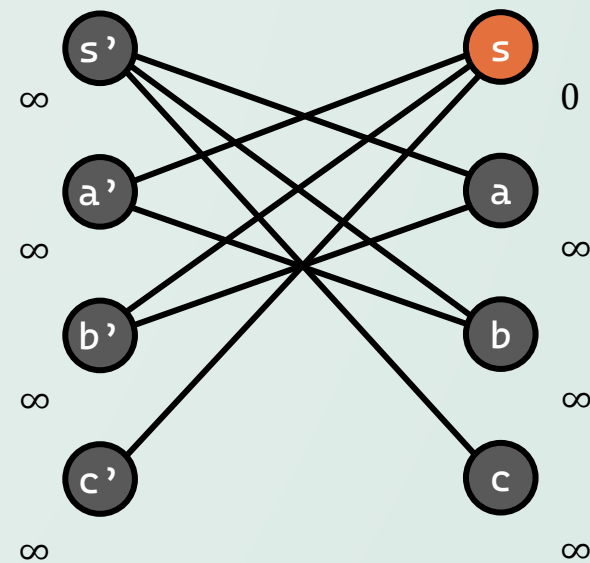
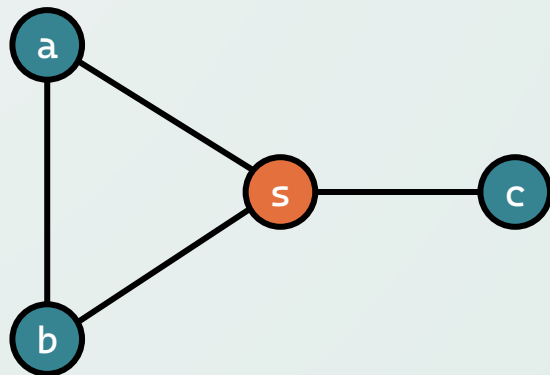


גרף שכבות

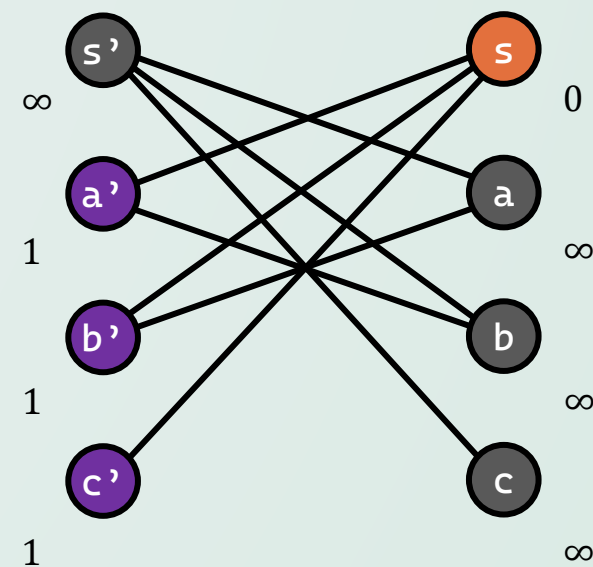
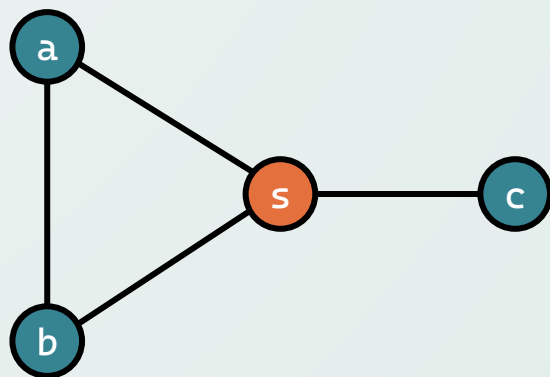
- על הגרף החדש, בעל שתי השכבות, נריץ BFS עם הצומת s כמקור.
- המסלולים הזוגיים הקצרים ביותר יהיו אלו המסתעפים בשכבה התחתונה (באותה השכבה עם s).



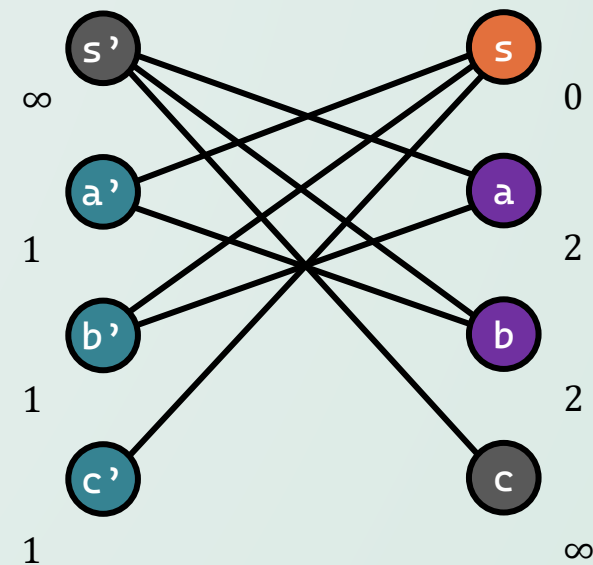
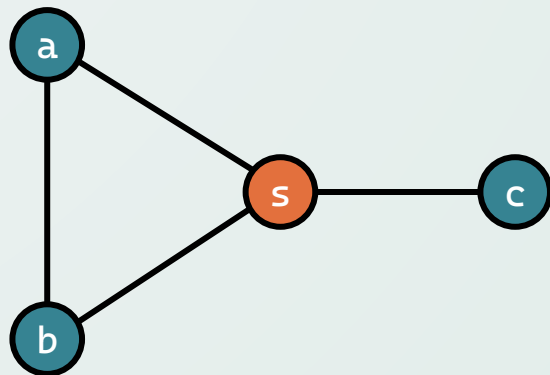
דוגמת הרצה



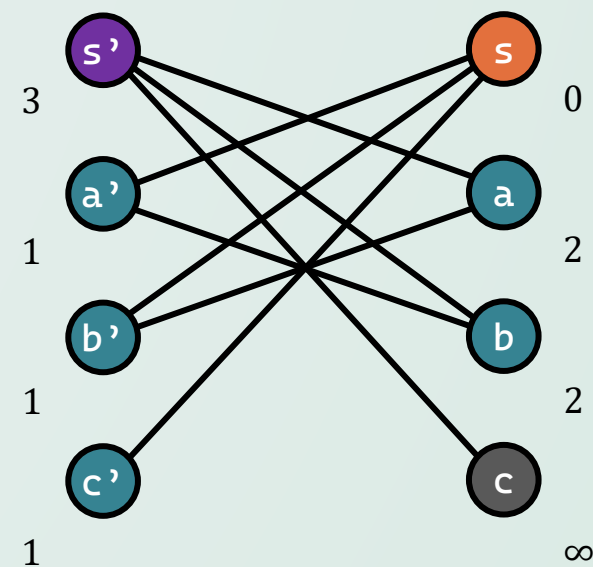
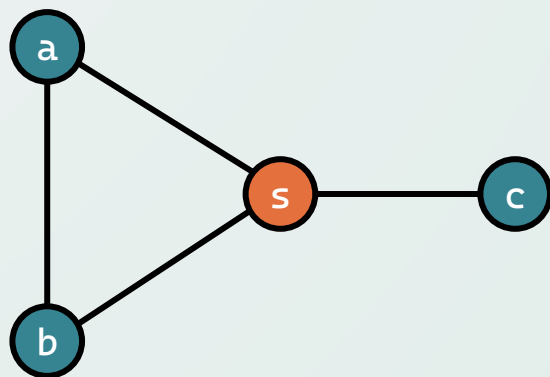
דוגמת הרצה



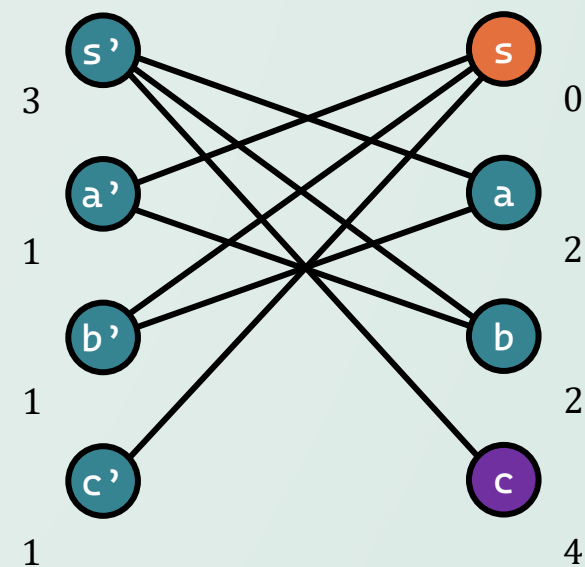
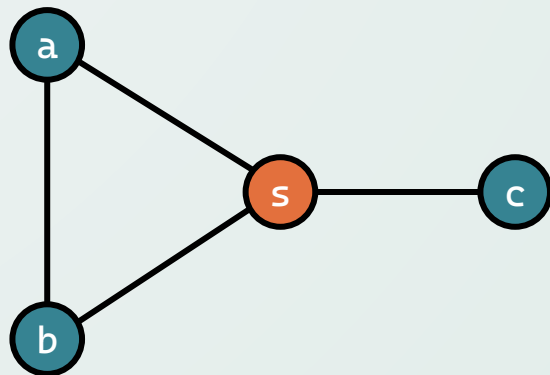
דוגמת הרצה



דוגמת הרצה



דוגמת הרצה



הוכחת נכונות

- נרצה לטעון כי $\text{dist}[v]$ הוא אורך המסלול הזוגי הקצר ביותר מ- s ל- v ב- G המקורי.
- לשם כך, נצטרך להראות מספר דברים:
 1. אם $\text{dist}[v] = k < \infty$:
 - I. אז k זוגי.
 - II. קיים מסלול באורך בדיוק k מ- s ל- v ב- G המקורי.
 - III. לכל מסלול באורך זוגי ℓ מ- s ל- v מתקיים $k \leq \ell$.
 2. אם $\text{dist}[v] = \infty$ אז לא קיים מסלול באורך זוגי מ- s ל- v ב- G המקורי.

הוכחת נכונות

- אם $\text{dist}[v] = k$ אז מנכונות BFS ומדרך בניית הגרף קיים מסלול קצר ביותר שאורכו k מ- s ל- v .
- מסלול זה מדלג בין שתי השכבות: כלומר, נוכל לסמן את המסלול בתור $s \rightarrow a' \rightarrow b \rightarrow c' \rightarrow \dots \rightarrow v$.
- כל קשת מדלגת בין השכבות, ו- s, v נמצאות באותה השכבה, ולכן ברור ש- k זוגי, כנדרש.
- נתבונן על המסלול המקביל בגרף המקורי $s \rightarrow a \rightarrow b \rightarrow c \rightarrow \dots \rightarrow v$. זהו מסלול חוקי, ואורכו k .
- כלומר, קיים מסלול חוקי בגרף המקורי שאורכו בדיוק k , כנדרש.
- יהי $s \rightarrow x \rightarrow y \rightarrow z \rightarrow \dots \rightarrow v$ מסלול מאורך זוגי ℓ בגרף המקורי.
- על פי בניית גרף השכבות, קיים בו המסלול $s \rightarrow x' \rightarrow y \rightarrow z' \rightarrow \dots \rightarrow v$ שאורכו ℓ גם הוא.
- בגרף זה, אנו יודעים כי המסלול הקצר ביותר בין s ל- v הוא k , ולכן $\ell \geq k$ כנדרש.
- נסתכל על המצב שבו $\text{dist}[v] = \infty$ נניח בשלילה שקיים מסלול זוגי מ- s ל- v בגרף המקורי.
- אם כך, בדומה להוכחת הסעיף הקודם, קיים המסלול המתאים לו בגרף השכבות.
- מנכונות BFS נובע כי אם קיים מסלול סופי שכזה מתקיים $\text{dist}[v] < \infty$, בסתירה.